Embedded Systems with the MSP430G2553

Sam Shue

August 2026

Table of contents

Preface

This book is an introduction to embedded systems programming using the Texas Instruments MSP430G2553 microcontroller and the MSP-EXP430G2ET LaunchPad development board.

It is intended for readers who are new to embedded systems, and covers the fundamentals of microcontrollers, C programming for resource-constrained targets, general purpose input/output (GPIO), embedded programming patterns, timers, analog-to-digital conversion, and serial communication.

How this book is organized

The book is split into parts, each grouping related chapters:

- **Foundations** — what an embedded system is, and the C programming concepts needed to work with one.
- **Digital I/O and Program Structure** — controlling GPIO pins and structuring larger embedded programs (hardware abstraction layers, state machines, interrupts).
- **Peripherals** — timers, analog-to-digital conversion (ADC), and serial communication.

Source material

This book is adapted from an earlier Microsoft Word draft of the same material. The original drafts and supporting diagram source files (`.pptx`) are kept for reference under `Archive/TI-MSP430G2553/`. See the project README for details on how this Quarto version was produced.

Part I.

Foundations

1. Introduction

1.1. Learning Objectives

In this chapter we will learn:

- What an embedded system is
- What is a microcontroller
- Why embed a computer
- What constraints do embedded systems have
- What is a development board?

1.2. What is an Embedded System?

An embedded system refers to a digital computer that is… embedded into another system! Think about all the devices you interact with daily that have been "digitized" by the inclusion of a small computer integrated into them to provide some sort of control or automation. Did you make coffee this morning with a coffee maker that automatically poured the perfect amount of coffee for your cup? Did you drive to work in a car that had a "push-to-start" button? Did you wash your clothes this week with a washing machine that knew how long to run each wash cycle based on the load size? These are all examples of systems that can automate many tasks due to their embedded computer!

On the other hand, what is not an embedded system? An embedded system is integrated into another device to provide some sort of control or automation. A computer that we use for general computing purposes, such as a laptop computer, would not be considered an embedded system.

A great rule of thumb for determining whether a computer is an embedded system or not, is to consider what is the purpose of the computer. If the device contains a computer, but is purchased or sold for some functionality other than computation, then it is an embedded system. If the device is purchased or sold because the product *is* a computer, then it is not an embedded system. For example, if you bought a coffee maker that has an embedded computer, do you care how much memory it has or how fast the processor is? However, when you bought your latest smartphone, did you exam the specifications of that device (e.g., processor speed, memory size, storage capacity)?

1.3. Embedded Systems and Microcontrollers

The subject of embedded systems is A **microcontroller** is a device that contains an entire computer on a single chip, making it very easy to integrate with other systems. How are we able to fit an entire computer onto a single chip? Given the perspective of viewing a computer as something powerful like a laptop or desktop, this concept might feel a little far-fetched, but keep in mind we don't need our coffee maker to run "photoshop".

Because of the limited needs of the application, the computer might not need very many resources, allowing it to be stripped of any excess memory and processing power and shrunk down to a very small size, perfect for integration. Many microcontrollers will have memory on the scale of Megabytes while your laptop is easily on the Gigabyte scale. Microcontrollers come in a variety of different specifications - and it is up to the engineer to select the one most appropriate for the system in which it's being embedded, based on processing power, memory, and input/output demands.

Circuit board containing microcontrollers

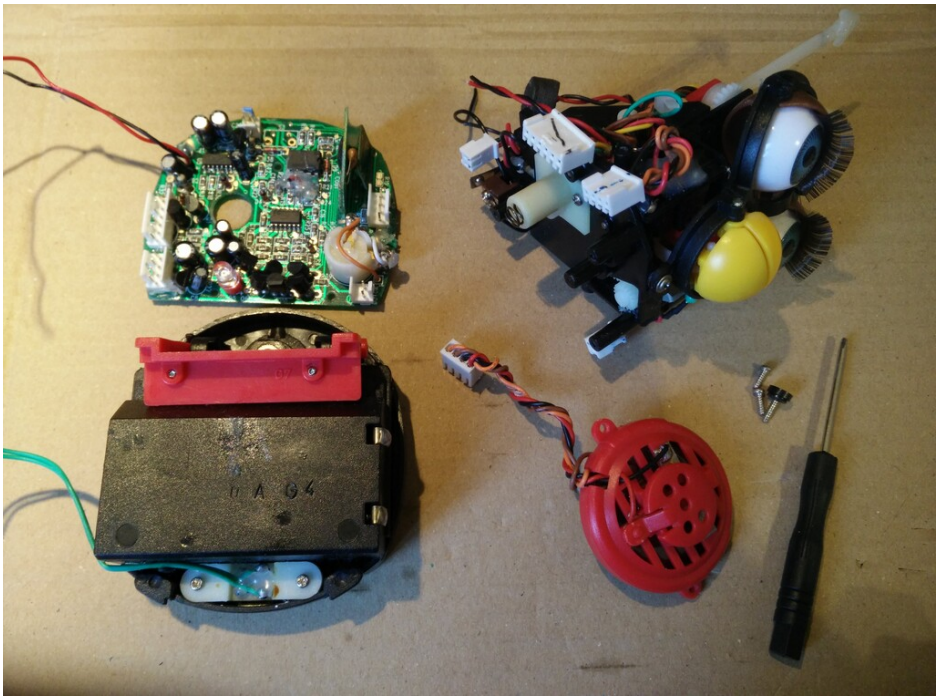


Figure 1.1.: Example microcontroller in embedded application (disassembled Furby electronic toy) [1]

1.4. Microcontrollers vs. Microprocessors

The term "microcontroller" may sound familiar to you, due to its similarity to the term "microprocessor". A microprocessor is the "brain" of a computer, such as the AMD or Intel processors found in a laptop or desktop computer. Microprocessors are often quite powerful, especially when contrasted with the computational power of most microcontrollers, but they themselves are

not an entire computer.

To understand what constitutes a computer, let's take a look at the five basic components of a Von-Neuman computer, which is the architecture of most modern computers:

In Figure 1.2, we can identify four major components:

- Memory
- Processor
  - Arithmetic Logic Unit
  - Control Unit
- Input
- Output

The memory is responsible for holding the computer's information, such as the instructions comprising the programs and any data currently in use. The arithmetic logic unit (ALU) is capable of performing computations on the data, such as addition, subtraction, etc. The control unit reads instructions and directs the ALU to perform the appropriate action. Together, the control unit and the ALU form the processor. The input collects data from the outside, such as through a keyboard or mouse, and the output can display information, such as a monitor, or perhaps even manipulate something, like printing a page on a printer.

The ALU and control unit are often bundled together and called the "processor". These two components are what comprise the microprocessor as well! As we can see here, the microprocessor is only two out of five components necessary for making an entire computer. The microcontroller actually contains all five of these components, but often in a more limited capacity than your laptop.

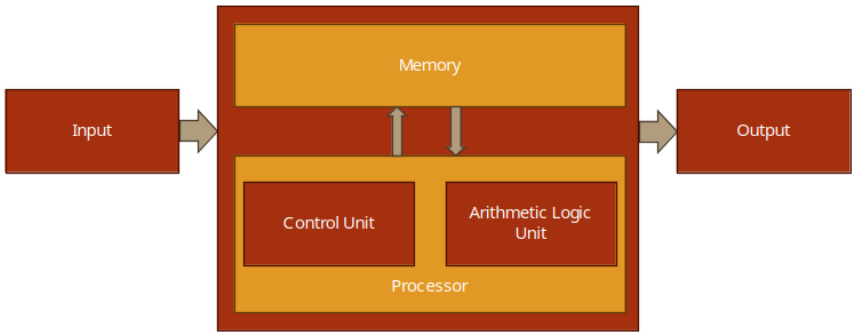


Figure 1.2.: Basic Structure of Von Neumann Computer

1.5. Why Embed a Computer?

Embedding a computer into another system can bring many advantages. Since computers can be programmed to create a variety of behaviors, they can replace the need for engineers to design a custom circuit to perform specific tasks. Say an engineer is tasked with creating a circuit that will blink a warning light on a car's dashboard with a one second interval. This would usually require a custom circuit with a specific timer and clock frequency, but rather than design this circuit, the computer can be integrated and programmed to blink this light. Not only would it be capable of handling this task, but many others at the same time. This is a simple example, but microcontrollers can also imitate the behavior of much more complex circuits as well.

Embedding computer to avoid having engineers create custom circuits cuts down on a product's non-recurring engineering (NRE) cost. This is the cost of paying an engineer to create a new custom design. The engineer will still have to program the microcontroller, but this is generally easier than building and testing a custom-designed circuit.

Microcontrollers also help reduce the cost of a design because they are a generic component. The same microcontroller can be integrated into many different systems and be programmed for very different tasks. Microcontrollers will be manufactured on a scale that makes them extremely affordable, often costing less than 25 cent per chip. Even though the microcontroller's design may be far more complex than a custom circuit for a specific task, its mass manufacturing still makes it the more cost-effective option.

Microcontrollers can also bring more advantages than just expense. In the same space any other circuit would occupy, a microcontroller can implement additional capabilities. This could be additional features beyond the base functionality of the device, such as safety features – both for the operator and the electronics themselves. This is because we are implementing

features through software rather than hardware, which doesn't require more physical space!

1.6. An Example of Embedding Computers

As an example, let's examine how embedded systems have affected automobiles. A typical modern car has dozens of microcontrollers embedded within it. Let's see why.

Consider the windows of a typical car door. Often, they are controlled by a motor and a button is used to control lowering and raising them. The motor that controls these windows could be driven by a simple switch, but including a microcontroller allows us to create additional features. Using only a switch we could run into trouble when the windows are fully closed, but the motor continues to run as the window presses against the frame. This could cause the motor to stall, pulling more current than it should, which could in turn damage the circuit. With a microcontroller, we can read the button press through a program, then in turn drive the motor. Moreover, we could read some feedback from the motor in terms of an encoder – which could indicate when the motor has moved an appropriate distance to fully close the window – or we could include a current sensor – which could be used to programmatically shut off the motor controls when the stall current is detected.

In addition to electronic safety features, we can also add child locks on the windows in the back passenger seats of the car. By networking the microcontroller with other microcontrollers in the vehicle, the driver may engage some safety lock feature which would digitally communicate to the other microcontrollers to disable the ability to roll the windows down.

1.7. Limitations of Embedded Systems

While we have observed many benefits that can come from embedded systems, such as cost reductions and automation, there are also many limitations that must be considered when designing for them. It was mentioned earlier that embedded systems are not as powerful as the microprocessors found in your laptop computer, and therefore their computing power and other resources are limited. This must be kept in mind when selecting the embedded system appropriate for the application.

If limited resources are such a concern, why not use a microcontroller with more memory and processing power than what's required for the application? There are two reasons for this: cost and power consumption. Cost is the more obvious of the two – more resources, which equals more hardware, which equals higher cost. This cost may only increase the microcontroller's cost by a few cents, but when the product that is utilizing the microcontroller is being manufactured on the scale of thousands to millions of units, these pennies begin to add up. Power consumptions is another major concern. More memory means more hardware components that require power, and faster processors have a higher clock frequency, which means more transistors switching, which means higher power consumption. Embedded systems often find themselves at home in battery-operated mobile devices, therefore keeping the power consumption low is of upmost importance to ensure battery life is elongated as much as possible. Have you ever purchased one device, such as a smartphone or laptop, over another because one had a longer battery life?

Embedded systems are also sometimes limited by their input and output capabilities. Most embedded systems come with some features that allow them to produce digital signals, read analog voltages, and communicate with other devices using serial protocols. These features can vary from microcontroller to microcontroller, so one must be selected that contains the necessary interfacing hardware, cost, and computational resources. It

is the embedded systems engineer's responsibility to select the correct microcontroller based on the project's demands for these criteria.

1.8. Developing for Embedded Systems

When developing for an embedded system, the design must first go through a prototyping phase to verify functionality. This prototyping phase usually involves breadboarding the circuit that will integrate the selected microcontroller, and the microcontroller will often be interfaced through a **development board**.

Development boards are prototyping boards that contain a microcontroller and onboard tools and components that aid in development and testing. These components often include the following:

- Voltage regulation
- Programmer circuit
- Debugger
- Breakout header pins
- Push buttons and LEDs

These components often appear on development boards because they are commonly needed by programmers to power the board, upload code, and test inputs and outputs.

Let's look at an example development board, the MSP-EXP430G5ET. This board has a lot going on, but what is most important is the chip at the center of the board, the MSP430G2553. This is the microcontroller, and all the other components here are to help with programming and testing of this chip.

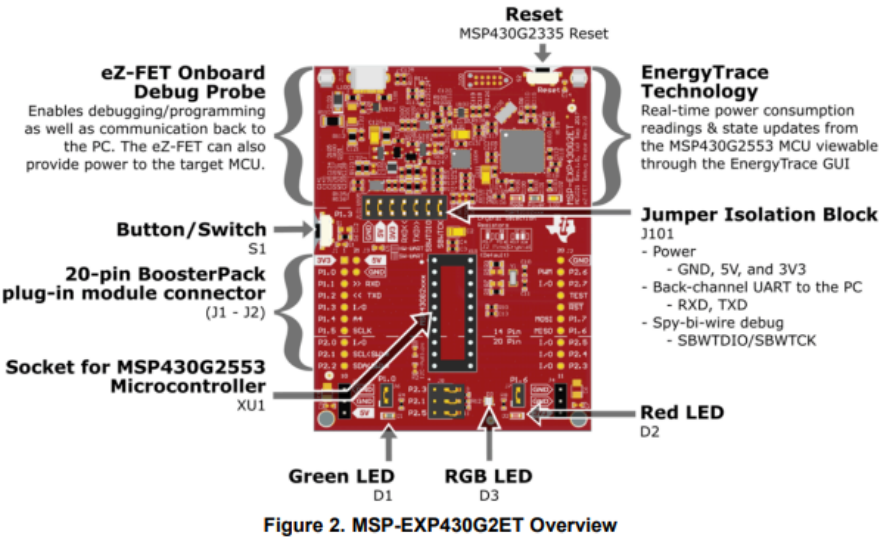


Figure 1.3.: An overview of the MSP-EXP430G2ET Launchpad development board

The top half of the board contains the "eZ-FET" debugger circuitry. A debugger allows us to meticulously examine our microcontroller's code behavior by setting breakpoints and stepping through the code line by line, while simultaneously being able to check memory contents. If this doesn't make much sense right now, don't worry, we will be elaborating on this again later in the book. This portion of the board also provides power regulation, which provides power to the microcontroller at a specific voltage. This part of the board also provides a virtual serial port connection from the microcontroller to your laptop or desktop computer, allowing communication between the two.

On the lower half of the board, beyond the jumper isolation block, we have the socket for the microcontroller, along with some extra connected components.

Arguably, the most important addition here is the inclusion of "breakout" pins. Breakout pins are pins that are large and spaced out, designed to be easily connected to jumper wires for interfacing with a prototyping circuit. The pins on a typical microcontroller package are much too small to attempt to plug or solder wires directly into, so the development board contains traces (wires embedded into the board) that connect these tiny package pins to the breakout pins.

In addition to the breakout pins, development board very commonly include components such as buttons and LEDs, as they are very often used in prototyping scenarios for easy getting user input via a button press or indicating the status of something through illuminating an LED.

Separating the top "half" from the bottom, we see a "jumper isolation block". These jumpers bridge electrical connections between the debugger circuit and the microcontroller. These bridged connections allow power and communications to reach the lower part. However, if we were planning on someday building a custom circuit using the MSP430G2355, we wouldn't be including a debugger/programmer, since we wouldn't expect the person purchasing the embedded product, the "end user", to reprogram the device. These jumpers allow us to disconnect the microcontroller and test the chip

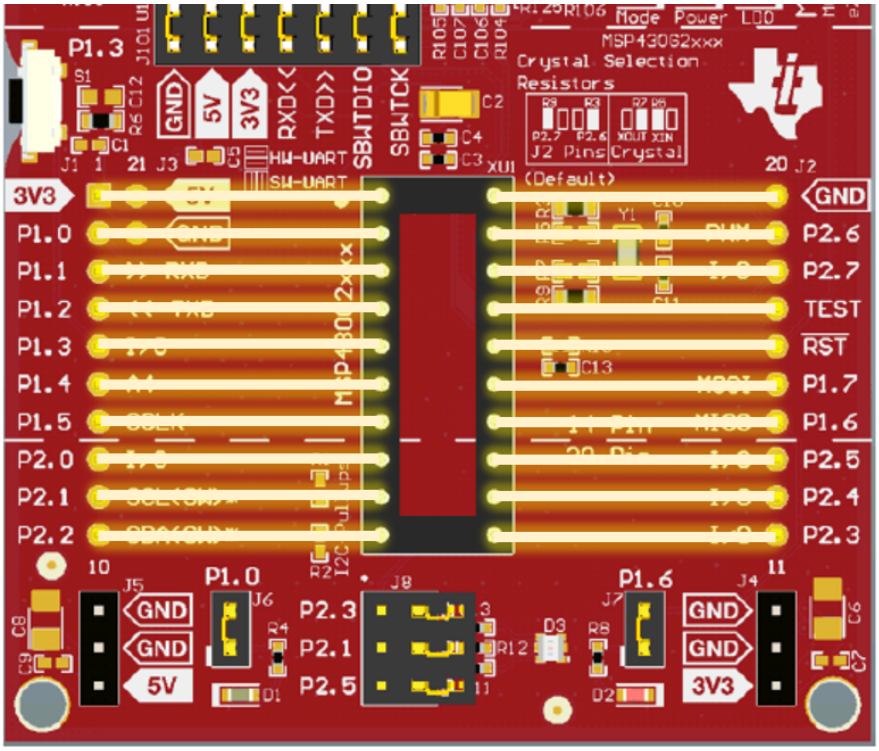


Figure 1.4.: How the legs of the microcontroller connect to the header pins of the development board.

without the support of that circuit, emulating what it might behave like before we create a custom board.

1.9. Recap

An embedded system is an application-specific computer system which is built into a larger system or device. Using a computer system enables improved performance, more functions and features, lower cost, and greater dependability. With embedded computer systems, manufacturers can add sophisticated control and monitoring to devices and other systems, while leveraging the low-cost microcontrollers running custom software.

1.10. References

1. P.-L. Ageneau, "Chaplier Fou," Aug. 25, 2023. [Online]. Available: https://chapelierfou.org/blog/a-usb-furby.html

2. C Programming

2.1. Learning Objectives

This chapter will serve as a C programming refresher, or a to-the-point introduction for those who have programmed in other languages but are using C for the first time. In this chapter we will provide the motivation for using C, the basic syntax, how C works with computer memory, and how to use the Code Composer Studio IDE to develop C programs for MSP430 microcontrollers.

In this chapter we will learn:

- The differences between low- and high-level languages
- Assembly language and machine code
- C Programming basics
- C and memory management

2.2. Programming embedded systems

Like any computer, embedded systems must be programmed before they can do anything. Before we get into how we program them, let's step back and define just what a program is. Because we use computers every day, I'm sure we all have an intuitive idea what a program is. What modern operating systems have come to refer to as an "App" may come to

mind, or maybe you think of a video game, or a text editor like Microsoft Word. While these are all programs, a program by definition is much more simplistic than any of these. A program is a set of instructions stored in memory to be executed by the processor.

This definition begs the question, what is an instruction? An instruction is some simple operation that a computer can do. This can be an arithmetic operation, like add two numbers together; something logistical like compare two numbers; or even skip some other instructions based on a condition. Each of these operations is defined by the processor's **instruction set architecture** (ISA). The ISA specifies a list of all operations the processor is capable of and how those instructions are formatted.

Let's take a look at an example of one of these instructions.

```
add.w #10, R5
```

This instruction adds 10 to the value stored in a memory location R5 and saves the result back to that location. R5 refers to a memory location inside a small memory component called the register file. This instruction could be used to compute a simple mathematical expression, such as

```
x = x + 10
```

Where x would be stored in the register file memory location, R5. We will see that when writing programs, we will often refer to the computer's memory components, such as the aforementioned register file. While memory can have many intricacies, for now let's just understand that memory in general has many locations for holding data, all indexed by a number that we call the **address**. In this example, R5 represents the address of a memory location (location 5) in the register file.

These instructions are presented in a way that we, as the programmers, can understand them. Hence why instructions, such as "add", are presented in English. Naturally, they are translated into **machine code**, or the binary

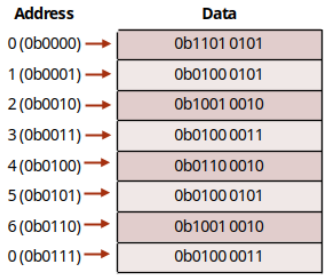


Figure 2.1.: An example of computer memory addressing

1's and 0's that a computer understands by the use of a program called a **compiler**. The language that we just saw is what we call **assembly language**, which directly corresponds to the machine code produced by the compiler.

In our previous example, we were able to add a number (10) to an existing number (the data stored in R5). Let's consider another example, but this time let's specify a mathematical operation we want our computer to perform:

```
x = x + 10 + 15 + 20 + 25
```

Now we want to sum up a variety of numbers. How would we write a program that does this? Assuming that x corresponds to register file location R5, we could write:

```
add.w #10, R5
add.w #15, R5
add.w #20, R5
add.w #25, R5
```

And this will perform this operation. This series of instructions, or program, starts from the top instruction and performs each operation sequentially. Computers perform one instruction after the other, and we can see by the time we reach the last line, we would have computed the result of the equation described earlier. One observation that can be made even this early, is as the application's complexity increases, so do the number of instructions. You can imagine that writing in assembly language would be very laborious for programmers creating complex applications such as Microsoft Windows. To assist with this problem, other programming languages have been created, such as C.

To implement the equation described earlier in C, we would simply type:

```
x = x + 10 + 15 + 20 + 25;
```

Which looks nearly identical to the equation itself! While the computer in the end would have to perform the same operations we saw in assembly, describing this in C is much simpler on the programmer's part. The compiler will then be responsible for interpreting this line of C code into the associated assembly instructions, and from there into the computer's native machine code. Because of the added step of going to assembly and then to machine code, we classify C as a "**high-level**" programming language. High-level refers to how distanced the lines of code are from the machine code instructions that would be required to execute them. In contrast, assembly language would be considered a "**low-level**" language.

2.3. Why Program in C?

This question might seem obvious at first, simply because the complexity of C seems lesser compared to our earlier example, which is largely still true. The one drawback to using C and other high-level languages is the efficiency. We've now taken full control of writing the individual instructions that comprise our program and handed it over to the compiler. A compiler may not be as skilled at interpreting the high level instructions into low level instructions, resulting in a less efficient and larger program.

This is the trade-off that exists with any high-level language: the further from the individual instruction the language is, the less efficient it tends to be. However, this trade off is nearly worth it every single time, as it would be too challenging to write modern programs directly in assembly. Not only would getting these programs written would be challenging, but the number of bugs (accidental, unintentional program behaviors) as a result of the complexity would be great.

2.3.1. Portent Portables

Writing programs in C does bring with it some other advantages, most notably: portability. We've mentioned assembly language and ISA, but we did not mention that each type of processor has its own specific ISA, and therefore a different set of instructions that they are capable of executing. The assembly language understood by the processor in your laptop computer is likely very different from the one in your smartphone. Therefore, programs written in assembly for one machine cannot be moved an executed on a different machine that has a different ISA. The C language however must be interpreted into assembly by the compiler before being interpreted to machine code. Therefore, if a compiler exists for each target machine, the C code can be rebuilt for each machine. This makes the C language more generic, and therefore more portable.

2.3.2. Why Not Program in C++?

If you're already familiar with programming, you may be familiar with the C++ language. C++ is a language derived from C, adding in many new features used by programmers to organize code. These features include a concept called a "class", which allows programmers to group related functions and data together, making it what we refer to as an **object-oriented** language.

While these features are attractive from the programmer's point of view, they also cause the language to be even further distanced from the instruction level than C. Because of this, C++ programs don't compile as efficiently as most C programs do. This typically is too much of a tradeoff, as microcontrollers still have limited amounts of memory, and cannot store very large programs. C++ is graining traction in the world of embedded systems development, especially as many modern microcontrollers have more memory than ever before, but still C rules supreme. There are also historical reasons for this as well, as many older projects active in both

hobbyist and industry spaces, are still ongoing and originally written in C.

2.4. Introduction to C

In this section we'll get familiar with the basic syntax of the C language. A language's **syntax** refers to the structure and keywords that exist within that language. Keywords or reserved words are words that are recognized by the compiler that have some explicit meaning or function. Let's begin by analyzing an example of a simple C program:

```c
#include <stdio.h>
int main(){
    // print hello world to terminal
    printf("Hello World!");
    return 0;
}
```

This is a simple program that prints the message, "Hello World!", to the standard output, which if ran on a traditional laptop or desktop computer, would be some terminal application.

<show hello world terminal screenshot>

The first thing to point out about any C application is that the program begins with the **main** function. We'll define functions in C more formally later on, but for right now, know that lines of code that follow main and are contained within the curly braces are the instructions that come next.

In this example, we have only two lines of C code. One is another function call, "printf()", and the other is "return 0". An individual line of C code is easy to identify, because each line ends with a semicolon. This tells the compiler when a single line of a C program is finished and ready to be

interpreted into the resulting assembly instructions. The “printf” function will write to the output whatever text is placed within the parenthesis that follow it. The return keyword ends the current execution of instructions within the function, therefore terminating our program and passing a 0 back to the operating system that called it. This 0 is typically a code that indicates that everything executed okay.

At the very top of the code we see an additional line, which reads: ”#include <stdio.h>”. This is an example of a preprocessor macro. The **preprocessor** is a part of the compilation process that runs before the compiler does. It processes any lines that begin with a “#” symbol and in this case is used to retrieve additional code to add to our program. This additional code is referred to as a **library**, and it gives us access to the “printf” function. Libraries are a handy way to provide other programmers prebuilt functionality, so you do not have to re-write everything from scratch.

We also skipped over the line that began with “//”. The double backslashes indicate a comment. A comment is a note left to add context or a message. These are usually left to remind the programmer what a complex series of instructions might be doing, or to explain something to another programmer who also is developing in this code base but didn’t write a certain portion of the code. Comments may also encapsulate multiple lines using the “/*” and “*/” syntax:

```
/*
Comments here
Comment here
And here. All on different lines!
*/
```

Comments are ignored by the compiler and do not add to the overall code size.

2.5. Creating Variables

If we wish to write our own programs, we need to be able to store data. Like many other programming languages, C uses variables to achieve this. Variables look just like a variable in a mathematical expression, and function in a similar way. A variable represents a location in the computer's memory that has been reserved to hold data.

In C, variables must be given a type. The type determines what kind of data will be stored in that variable. Let's look at an example of how one might declare a variable:

```
int main(){
    int x;
    return 0;
}
```

In this example, we've declared a variable called "x" and it has been designated to hold an integer by preceding our variable name with the keyword "int". There are many different data types in C, but we'll examine some of the basic types here:

- int – holds only integer numbers
- float – holds decimal point numbers
- char – used to hold values that correspond to characters in the ASCII table
- bool – used to hold true or false values. Short for "Boolean"

These variables can all be used in different scenarios depending on the need of the data type.

2.5.1. Operators

Now that we're able to reserve memory locations to hold our data in the form of a variable, we need operators to be able to actually do things with the data. There are many operators in C, but we'll start with the most simple, the assignment operator, "=", which allows us to write data to a variable. Let's modify our earlier example to assign a value of 5 to our variable x we declared:

```
int main(){
    int x = 5;
    return 0;
}
```

Now after reserving the space for variable x, the value 5 will be written to that location. There are many more operators available in C, so let's have a look at all the different types available.

<Add initialized vs uninitialized junk here>

2.5.1.1. Arithmetic Operators

Arithmetic operators are used to perform arithmetic operations, such as addition, subtractions, etc. These operations require variables or values to perform the operation with, and a destination variable to write the result. Let's see an example of the addition operator:

```
int main(){
    int x = 5;
    int r = x + 10;
    return 0;
}
```

We've expanded our earlier example by adding an additional variable, r, which hold the result of our addition operation, which adds 10 to the value stored in x, 5. After this operation, r will hold the value, 15. There are many arithmetic operators in C, here are some of the most common:

- + – performs addition
- - – performs subtraction
- * – performs multiplication
- / – performs division

2.5.1.2. Boolean Operators

In addition to arithmetic operators, we also have a set of Boolean operators. Boolean operators are operations that do not have a numerical result, but rather a true/false result. Boolean operators are used to compare two numbers for properties such as equality, greater than, etc.

- x == y – used to compare two values for equality
- x != y – used to compare two values for inequality
- x > y – used to evaluate greater than
- x < y – used to evaluate less than

<side note: are adding variables (x,y) more or less confusing?>

2.6. Branching and Loops

Boolean operators are often used in conjunction with branching and loops within a C program. Branching refers to performing different actions based on a condition, and looping allows us to re-run lines of code over and over. These structures give us the ability to make decisions within a program.

2.6.1. If Conditions

"If" conditions are the essential branching method in C. An "If" condition will run a specified group of instructions based on a Boolean condition. Let's take a look at an example:

```
int main(){
    int x = 5;
    int r = 0;
    if( x > 2){
        r = x + 10;
    }
    return 0;
}
```

Here we see the keyword, "if", followed by parenthesis. Within these parenthesis, a Boolean expression is described. If the Boolean expression resolves as true, the lines of code contained within the following parenthesis will be executed. Otherwise, it will be skipped and execution will continue.

If conditions can also be expanded upon by adding additional conditions and an "else" condition that resolves when any of the preceding conditions are not true. An example of this is demonstrated in the code below:

```
int main(){
    int x = 5;
    int r = 0;
    if( x > 2){
        r = x + 10;
    } else if(x > 5) {
        r = x + 15;
    } else {
        x = r + 5;
    }
```

```
    return 0;
}
```

2.6.2. While Loops

In addition to conditional branches, programmers often have the need to repeat certain lines of code. The "while" loop serves this function by repeating grouped lines of code so long as its Boolean condition resolves as true:

```
int x = 0;
while ( x < 5){
    x = x + 1;
}
```

In this while loop, the body of the loop (the lines of code encapsulated by the curly braces) will execute five times, terminating once x reaches 5, as the condition will no longer evaluate as true. The condition of a while loop is evaluated prior to executing the body, so if x had already been greater than 5, the body of the loop would not execute even once.

If there is a need to execute a body of code once, THEN check the condition, a do-while loop may be used:

```
int x = 5;
do {
    x = x + 1;
} while ( x < 5);
```

Here, the body of the loop will be ran BEFORE checking the condition.

2.6.3. For Loops

While all of our looping needs can be satisfied by the while loop, another type of loop exists as a shortcut for common looping conditions: the for loop. It is very common that programmers might want a loop to execute for a certain number of iterations. To achieve this, a while loop can be constructed and a counting variable might be used, similar to how the variable x behaved in our while loop example earlier. A for loop incorporates a counting variable into its syntax, allowing easy control over how many times a loop iterates.

```
int i;
for(i = 0; i < n; i++){
    // body of the loop goes here
}
```

The for loop header is composed of three parts: initialization, condition, and increment/decrement. We can see here that in the parenthesis that follow the keyword "for", the variable i is initialized to 0. This is followed by the condition, i < n, which like in the while loop, tells the body when to stop looping. The last part is the increment/decrement, where the variable is incremented by one (++ is the c shortcut for adding 1 to a variable). Each part is separated by a semicolon.

2.7. Functions

Functions are the main source of organization in the C language. Functions allow us to group lines of code and invoke them at various points in our program. Let's consider and example and discuss the syntax:

```
float circleArea(float radius){
    float area;
    area = radius * radius* 3.14;
    return area;
}
```

This is a function that computes the area of a circle given a radius value. The beginning of the function has three parts: the return value, the name, and the parameters.

The return value is what the result of the function is; for a circle that computes the area of a circle, we expect a decimal point value as the result, so our return value is simply indicated by a "float".

The function name is "circleArea". This part is not a keyword, and can be named whatever we want. We've chosen a name that is indicative of the function's purpose.

The parameters are the values that are passed to the function. Functions can have one, many, or no values passed to them. The parameters are contained within the parenthesis and contain a data type and a variable name. The values passed to a function are copied from their original locations, and these variables will only exist between the curly braces that contain the body of the function.

Now that we've discussed the format of a function, we can revisit our main function we saw in the "Hello World" example:

```
int main(){
    // print hello world to terminal
    printf("Hello World!");
    return 0;
}
```

We can see how main follows the same structure as our other function example. We'll note that main takes 0 parameters and returns an integer. The main function is called by the operating system if on a desktop or laptop computer, and the integer it returns encodes an error value. While functions can be named anything, main is an exception, as the compiled program will start at the function called main.

2.7.1. Scope

As mentioned before, variables declared within a function are not accessible outside of that function, and they only exist in memory so long as that function is still be executed. This limited availability of a variable is what we refer to as the variable's "scope".

Let's consider an example using the area function earlier:

```
float circleArea(float radius){
    float area;
    area = radius * radius* 3.14;
    return area;
}
int main(){
    float r = 5;
    float area = circleArea(r);
    …
```

Here, the variables "r" and "area" within "main" are only accessible within the main function's curly braces. The function "circleArea" is called within main, so main's variables do not get removed from memory yet, but are not accessible within the "circleArea" function. We can observe that circleArea has two variables within its scope, "area" and "radius". The "radius" variable is populated with a copy of the data contained in "r" when the function is called. We also notice that our variable "area" shares a name

with the "area" variable from main. Variables must have unique names so long as they are in the same scope, but here, since these "area" variables are only available within their respective functions, this isn't a problem and they are both easily differentiated from each other due to their scope. This kind of scope is what we call a **local scope**, as theirs scopes are local to the functions where they were declared.

We can also bypass the local scope of a variable by declaring a variable outside of the functions.

```
float area = 0;
void circleArea(float radius){
    area = radius * radius* 3.14;
}
int main(){
    float r = 5;
    area = circleArea(r);
    …
```

Here, "area" has been moved and declared outside of both functions, giving it a **global scope**. Global variables are scoped to all functions that exist within the same C source code file. By making "area" global, we no longer have to create multiple local variables and return the area from the function.

This may seem like a convenient shortcut, but when projects become large, this scales poorly and overuse of global variables can result in wasted memory and very difficult to trace bugs.

2.7.2. Function Prototypes

In the examples we just saw, all of our functions were defined above the main function, and this was no coincident. In C, source files are compiled from the top-down, which means if a function is called before the compiler

has processed that the function exists ("circleArea", for example), it won't be able to perform the compilation, and will throw and error. As files grow larger, programmers often wish to place main() closer to the top of the file, given the importance of main and understanding how a program behaves. However, we need some way to do this without breaking compilation.

Function prototypes exist to remedy this issue. A function prototype is a line of code that alerts the compiler of the existence of a function, what parameters it takes, and what it returns; but excludes the implementation. Let's use our example from earlier, but instead we'll place main() above circleArea()'s implementation:

```
#include <stdio.h>
float circleArea(float radius);
int main(){
    float r = 5;
    float area = circleArea(r);
    printf("Area: %f", area);
    return 0;
}
float circleArea(float radius){
    float area;
    area = radius * radius* 3.14;
    return area;
}
```

Now circleArea() has a prototype that resides at the top of the file, and the implementation of the function exists below main. This prototype will be encountered by the compiler before the call to it in main(). When a function call is compiled, the compiler doesn't need to know what the function does, just what needs to be passed to and returned from the function. Since the compiler knows what "circleArea()" is, it can finish the compilation, just so long as it eventually finds an implementation that

pairs with the function prototype. If it doesn't, then the code will fail to compile.

2.8. Arrays and Structs

We've discussed the basic data types of the C language, but often we need more than a single value to represent different concepts or a collection of values.

2.8.1. Arrays

Arrays are used to maintain a contiguous sequence of data in memory.

Let's look at an example of an array:

```
int a[5] = {4, 6, 2, 8, 0};
```

Here, the array is declared following the data type it holds like any other variable, but the defining line of syntax is the angle brackets that follow the name. Within the angle brackets the number of elements contained within the array is defined. Here, the compiler makes space in memory for five integers one after another. Following those brackets, the array has been initialized with some values to fill the five elements specified. These are contained within the curly braces and separated by commas. In this case, the "5" within the angle brackets could be removed, as the initialized list of elements is implicit of the size of the array.

Each element in "a" is accessed by an "index", which begins at 0. For example:

```
int r = a[0] + 10;
```

This code accesses the first element of the array which exists at index 0. The last element of the array exists at index 4. An array in C is a very simple structure, as the array itself only maintains the position in memory where the array begins, and just counts the requested index worth of space away from that starting position. Arrays have no way of knowing how large they are, as they are just given the requested amount of space when they are declared. Because of this, programmers must be careful, because the following code will compile with valid syntax for our current example:

```
a[9001] = 74;
```

This is clearly outside of the number of elements the compiler created for us, but to the computer it just wants to access the memory location that is 9001 integers away from the start of the array. This can result in the computer attempting to read from an invalid memory address.

2.8.2. Structs

Structs are a way to group related variables in C. For instance, we might want to maintain an x, y point in 2-D space. I could declare two separate float variables for x and y, but then I need to keep track of them together throughout the rest of the code, and it might not be immediately clear to other programmers that they are a pair. To resolve this issue, a struct can be used:

```
struct point {
    float x;
    float y;
};
```

This defines a structure called point, and this can be used as if it where a new datatype. For instance, I can declare an instance of my struct and assign values to x and y as follows:

```
struct point p;
p.x = 2;
p.y = 3;
```

We can see here that I declared the struct using "struct point" followed by a name for the variable, "p". I then accessed and assigned the members of that struct, x and y, using the "." Syntax. This operator allows me to access the variable contained within the specified structure. The variables within a struct can be used like any other variable after being accessed through the "." operator.

2.9. Pointers

Pointers are one of the most confusing and unique aspects of both C and C++ programming. A pointer is a variable that holds the memory address of another variable, thus causing them to "point" to another variable. While confusing in nature, this allows specific control and access of memory locations, which we will see is very important for microcontroller programming. Many languages try to hide this aspect of programming from the programmer, as mismanagement of pointers can lead to some very serious bugs.

Let's see an example of a pointer:

```
int* p;
```

From this, we can see that pointers are declared like any other variable. What makes the declaration different is the addition of the asterisk following the data type. This is what defined the variable as a pointer, by indicating what type of data the pointer will hold the address. In this case, variable "p" will hold the address of another integer variable.

So to actually use the pointer, we need another variable to point to.

```
int* p;
int x = 5;
p = &x;
```

Now we've added an integer variable x, and used a new operator, "&", which extracts the address from the variable it precedes. This takes the address of x and stores it in p. Now, I can use "p" to manipulate "x" through the memory address it holds.

```
*p = 10;
```

This line of code assigns the value 10 to the variable "x". This is done by use of the asterisk that precedes "p". This is called "**dereferencing**" the pointer variable, which means that we follow the pointer back to the memory address it holds and use the variable that exists there. This allows us to both read and write that's variables data.

<Create visual reference for what's happening in memory here>

2.9.1. Pointers and Functions

So far we've introduced pointer syntax, but how is it useful? In C, one of the most valuable usages of a pointer is to allow other functions to utilize a variable that is out of scope. Instead of passing a variable to a function, which results in a copy being made, we can pass a pointer to variable, which allows a function to make permanent changes to a variable outside of its scope. For example, we can write:

```
void swap(int* x, int * y){
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

This function will permanently switch the values of the integers passed to this function as pointers.

```
int main(){
    int a = 1;
    int b = 2;
    swap(&a, &b);
```

In this example, we called swap by using "&" to get the addresses of "a" and "b" to pass to the function. The function dereferences those pointers to make changes to variables "a" and "b", which aren't normally accessible by "swap". After the swap function is called, the value of "a" is 2 and the value of "b" is 1.

2.9.2. Pointers and Arrays

Arrays have a unique relationship with pointers as well. Earlier, we mentioned that arrays only retain the location in memory where the array begins. Because of this, we can use the name of the array as a pointer. For example:

```
int a[] = {1, 2, 3, 4}
int* p = a;
```

Here, I can use "a" to assign integer pointer, "p", as the name of the array refers to the address where the array begins. Because of this relationship, we can then pass pointers to arrays to other functions to modify the original location of an array, just as we did with other variables. The important thing to observe here is the absence of the & operator. Since the name of the array already refers to the address, we do not have to & prior to extract the address.

2.9.3. Strings

When writing programs, especially any that display information to a user, we are interested in storing entire words and sentences. Earlier we introduced the char data type, which was used to store individual characters.

```
char c = 'A';
```

However, we need to be able to store a group of characters to form words or sentences. This is where the concept of a string comes in. A **string** refers to an array of characters ending with a null character.

```
char s[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

This array is a string that holds the word "Hello". Note that the last element contains '\0', which represents the null character. A null character is used to indicate the end of a string, and is essential when attempting to determine where a string ends, as arrays do not know how long they are. By including a null character at the end, we can create loops that iterate through a string and stop when they encounter '\0' as that indicates the end of the string has been reached.

2.10. Variable Sizes in Embedded Systems

One question that might come to mind is why differentiate between data types? If you think about it, you should be able to do everything with a "float". Decimal point numbers can represent integers, we could use them to hold corresponding character values as well, and even use them to represent true and false by designating a certain value to represent "true". The answer comes down to speed and efficiency. Binary can naturally represent integer values, but it requires a special format to represent decimal numbers.

Floating point numbers are also difficult to process and require special hardware. Floats also take up more memory than chars or bools, and depending on the machine, "int"s as well. Therefore, we define other data types for faster processing and better use of the limited memory available on our microprocessor.

<Add example here of two programs producing variables of different sizes>

To this end, C defines even more varieties of data types that give use even more control over the amount of memory reserved for each data type.

- short – an integer that uses even less memory space than int, but holds smaller numbers
- long – an integer that uses more memory space than int, but holds larger numbers
- double – a decimal point number that uses more memory than float, but holds larger numbers

2.10.1. Modifiers and Qualifiers

We also might want to be able to further control the data types and how they're used, and to aid with this, C allows us to specify modifiers and qualifiers for our data types.

2.10.1.1. Modifiers

Some common modifiers are:

- `long`
- `short`
- `unsigned`

- `signed`

These modifiers can be used in conjunction with most basic data types. For instance, I can declare:

```
long int x;
```

Which makes an integer that on most desktop machines holds 64 bits rather than 32 bits. I can also use the "short" modifier to make a 16-bit integer. Other modifiers can determine whether a variable is treated as having negative values or not:

```
unsigned int x;
```

This prevents "x" from being treated as though it has a negative value. From a numerical perspective, this allows a greater range of positive values, as we no longer have to dedicate the sign bit to representing negatives. This modifier is used a lot in microcontroller programming, as we are often modifying data that exists in memory locations where each bit's value is important, and the numerical value of the data isn't of interest. This will be contextualized in the following chapter.

2.10.1.2. Qualifiers

C also supports type qualifiers. Qualifiers specify how the variable can be used within a program. Some common qualifiers include:

- const
- static
- volatile

The const qualifier specifies the value of a variable is constant. This means after the variable has been assigned a value, it cannot be changed. This is useful for defining actual mathematical constants in our program, such as pi:

```
const float PI = 3.14;
```

While you could hard-code 3.14 everywhere in memory where you wanted to use pi, this enhances the readability of the code. It can also be useful for values that serve as thresholds that may need to be adjusted throughout a program's development.

The static keyword allows a variable to hold its value even after going out of scope. This allows functions to essentially have their own variable that retains its memory between function calls.

```
int increment(){
    static int x = 0;
    x = x + 1;
    return x;
}
```

In this example, the first call to the function will return 1, then the following call will return 2, as the initialization of the static variable, "x", only occurs on the first call, and it retains its data between subsequent calls.

The volatile qualifier is especially relevant for embedded systems as we will see in coming chapters. The volatile keyword forces the compiler to explicitly build all instructions that the variable is involved in. Sometimes the compiler will attempt to analyze and optimize C code as it translates it into assembly, and because of this process, it may exclude some operations that it evaluates as unnecessary. The volatile keyword prevents this from happening, and forces all of its operations to be built.

2.11. Variables and Memory

When programming microcontrollers in C, it is especially important for the programmer to be aware of how their code is affecting the use of memory, especially since this tends to be a limited resource. Depending on where a variable is declared within the lifetime of a program, it may be stored in a different location and exist for either a limited amount of time or for the duration of the program.

2.11.1. Memory Map

Memory in a computer is generally divided up into different regions, each with a specific purpose. diagram below shows the most common memory regions. The diagram below shows these regions. The order and address ranges of these regions may differ between microcontroller architectures, or even include additional memory regions, but across all architectures the ones presented here will be present.

These regions can be separated into two categories: **Read Only Memory (ROM)** and **Random Access Memory (RAM)**. Let's take a look at the purpose of each of these regions and their respective category.

2.11.1.1. Text

The region labeled text is where the instructions from the compiled program are stored. Once a program has been compiled and loaded, these instructions cannot be changed, making this region ROM.

2.11.1.2. Data and BSS

The data and BSS regions hold global, static, and constant variables. If a variable is uninitialized or initialized to 0, it is stored in the BSS

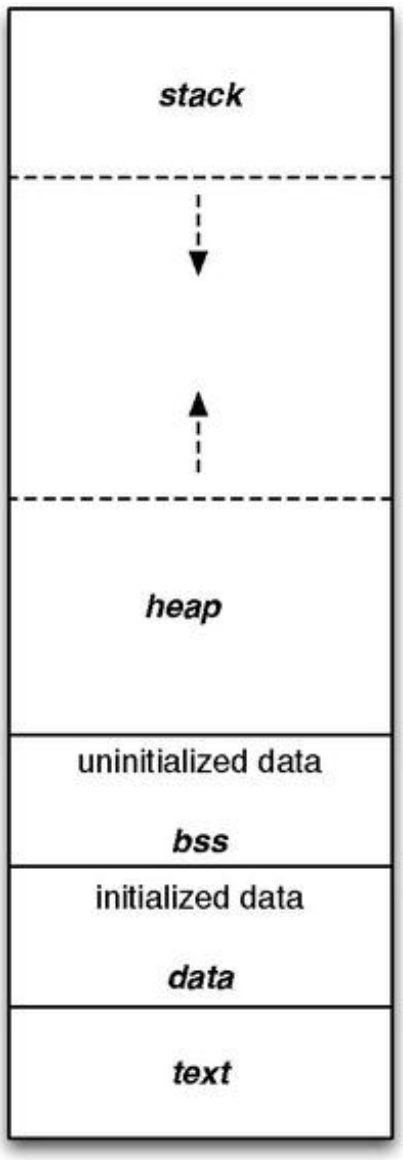


Figure 2.2.: Generic Microcontroller Memory Map

regions. Otherwise, it gets stored in the data region. These regions can be manipulated at runtime, making it a form of RAM.

2.11.1.3. Stack

The stack is where local variables are stored, and it behaves in a first-in, last-out fashion, like a stack of items in real life. Local variables are automatically placed on the stack when they are created, giving them the name, "**automatic**" variables. In most architectures, the stack grows from larger address towards lower addresses. The nature of the stack means that it must exist in RAM.

2.11.1.4. Heap

The heap is where dynamically allocated variables are stored. These are variables created at runtime by other instructions and are not stored in the stack. The programmer determines the lifespan of these variables by having to be manually responsible for their creation and deletion. The heap grows in the opposite direction of the stack, to best utilize the range of memory allocated for both of these structures. Like the stack, this region is also considered RAM.

2.11.2. Advanced Considerations: Memory Behavior

2.11.2.1. Instructions and the Text Region

The text region contains the individual instructions that comprise the compiled program. These instructions are the binary representation of the MSP430 assembly language. Figure 2.4 shows how a compiled C line of code could be represented by a set of assembly instructions. Here, a single line of multiple addition operations would result in multiple assembly addition instructions.

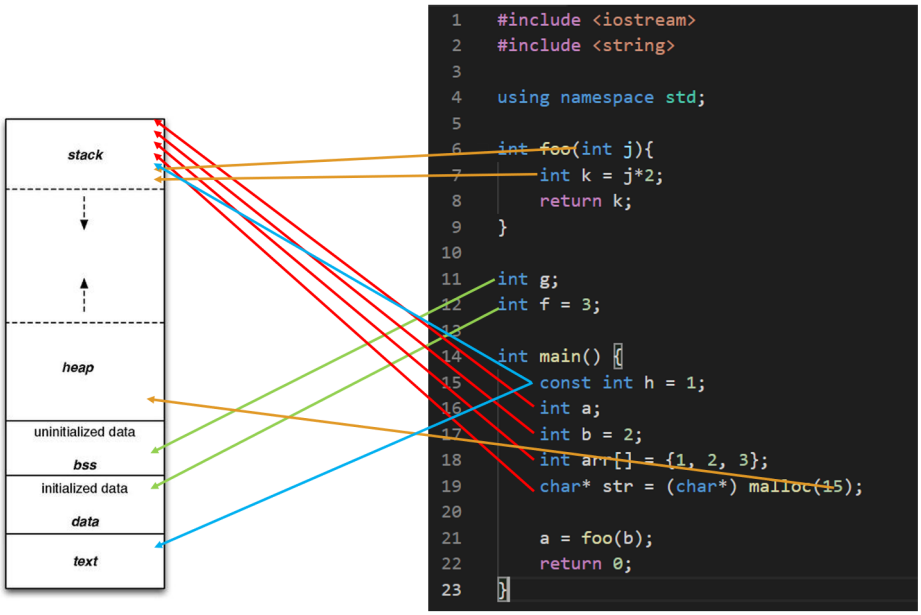


Figure 2.3.: Example C code and the associated memory location of each variable. Note that some compilers may optimize const variables during compilation and place them within the text region.

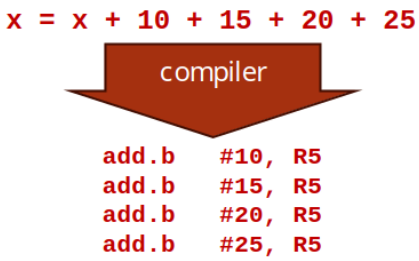


Figure 2.4.: Example section of the text region containing compiled instructions.

These instructions are then placed in the text region and read out and executed by the processor in a linear sequence. The processor has a special register called the "Program Counter" that tracks the memory location of the current instruction being read. As the program is executed, the program counter will increment to the memory location of the next instruction.

For example, to execute the example code in Figure 2.5, the program counter would start at address 0xC012 to read out the instruction "ADD.B #10, R5". Since the current instruction requires 4 bytes, the next instruction is 4 bytes away from the current instruction address, 4 will be added to the program counter's value. The program counter would then read out the instruction "ADD.B #15, R5". The process will then continue. The program counter will simply increment to the next instruction unless a branching condition, such as an if statement or a function call, would cause the program counter to jump to another location in code.

2.11.2.2. Stack Behavior

All variables that are declared inside of a function (with some exceptions, such as const and static variables), are stored within the stack. The stack functions in a way that is intuitive to its naming: variables are piled up on top of each other in memory – growing from high addresses to lower addresses. For a simple C program, we might see stack contents similar to the diagram in Figure 2.6.

Note that in Figure 2.6 the compiler has created space on the stack to hold all of the local variables within main. Since the code is paused before the call to the "foo" function, the contents of variable "z" are still uninitialized; only space on the stack has been allocated for "z" to exist – the contents of "z" are unknown at this time, as they are whatever bit values happened to be in that memory location prior to allocation of space for "z".

We can also observe the memory contents for each variable in this figure. Note that "x" is allocated 2 memory locations, as integers require 16 bits

Instruction	Memory	Address
ADD.B #10, R5	0x35	0xC012
	0x50	0xC013
	0x0A	0xC014
	0x00	0xC015
ADD.B #15, R5	0x35	0xC016
	0x50	0xC017
	0x0A	0xC018
	0x00	0xC019
ADD.B #20, R5	0x35	0xC01A
	0x50	0xC01B
	0x14	0xC01C
	0x00	0xC01D
ADD.B #25, R5	0x35	0xC01E
	0x50	0xC01F
	0x19	0xC020
	0x00	0xC021

Figure 2.5.: Example section of the text region containing compiled instructions.

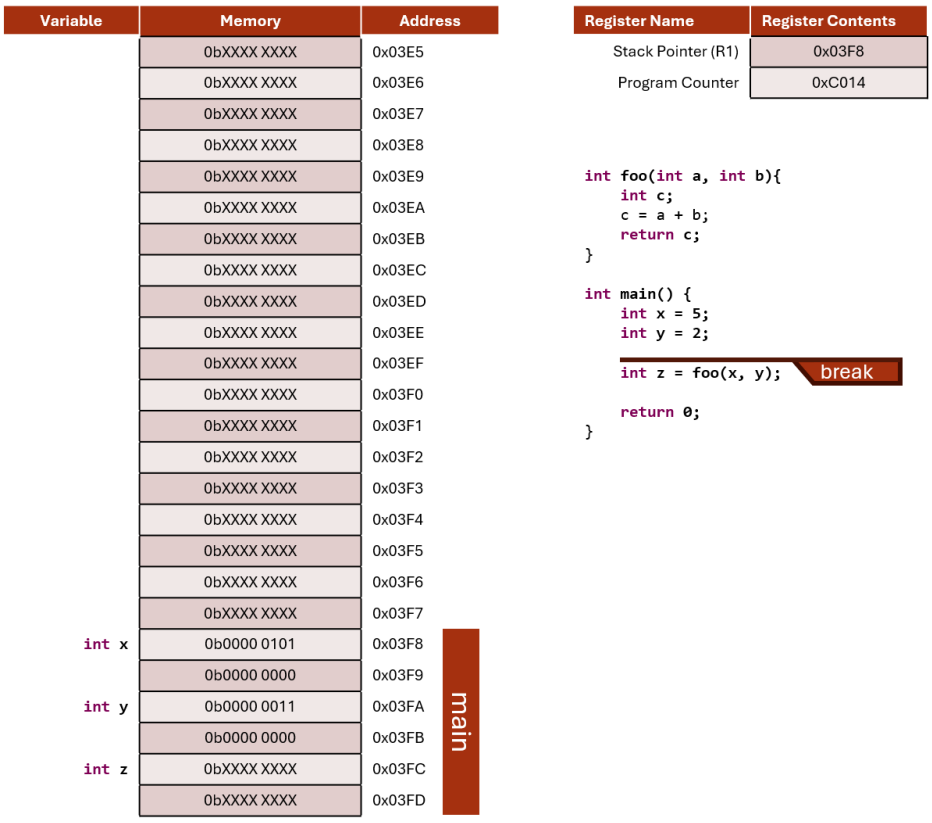


Figure 2.6.: Example C code and the condition of the stack up to the indicated break point. “X” indicates bit values that have been uninitialized.

in the msp-gcc compiler used by code composer studio, and memory is addressed in 8-bit segments. If we used 16 bits to represent 5 in binary, it would look like "0000 0000 0000 0101". But if we look at the memory location used to hold "x", we see the least significant byte, "0000 0101" comes before "0000 0000". This is because the MSP architecture is "little endian", which refers to the order that data is broken up into when stored in memory. "Little endian" means the least significant byte goes in the lowest memory address. This contrasts with "big endian", which would store the bytes in reverse. The order bytes are stored in memory is architecture-specific, but ultimately has no consequence on the device performance and is an arbitrary decision.

Each function in C is given its own "stack frame" when called. We can see that in Figure 2.6 where a contiguous section in memory is carved out for all of "main's" variables. If we let the program break at a point in "foo", we'll see another stack frame created for the "foo" function, as illustrated in Figure 2.7.

In this figure we can see the "foo" stack frame stacked on top of the "main" stack frame. Note that the stack frame of "foo" contains all of the local variables (including input parameters) along with a "return address". The return address is used to indicate where in the instruction memory to return to once the "foo" function concludes. We can note between Figures 2.6 and 2.7, the return address in Figure 2.7 is 4 bytes greater than the value of the Program Counter in 2.7, which makes sense as the next instruction is 4 bytes away from the instruction that called "foo", and, once "foo" is complete, we should call the next instruction after "main" would have called "foo".

2.11.3. Dynamic Memory Allocation in C

Utilizing the heap in a C program can be a very tricky thing, and if handled improperly, can lead to memory leaks, which are very dangerous to an embedded system. Dynamic memory allocation refers to utilization of the

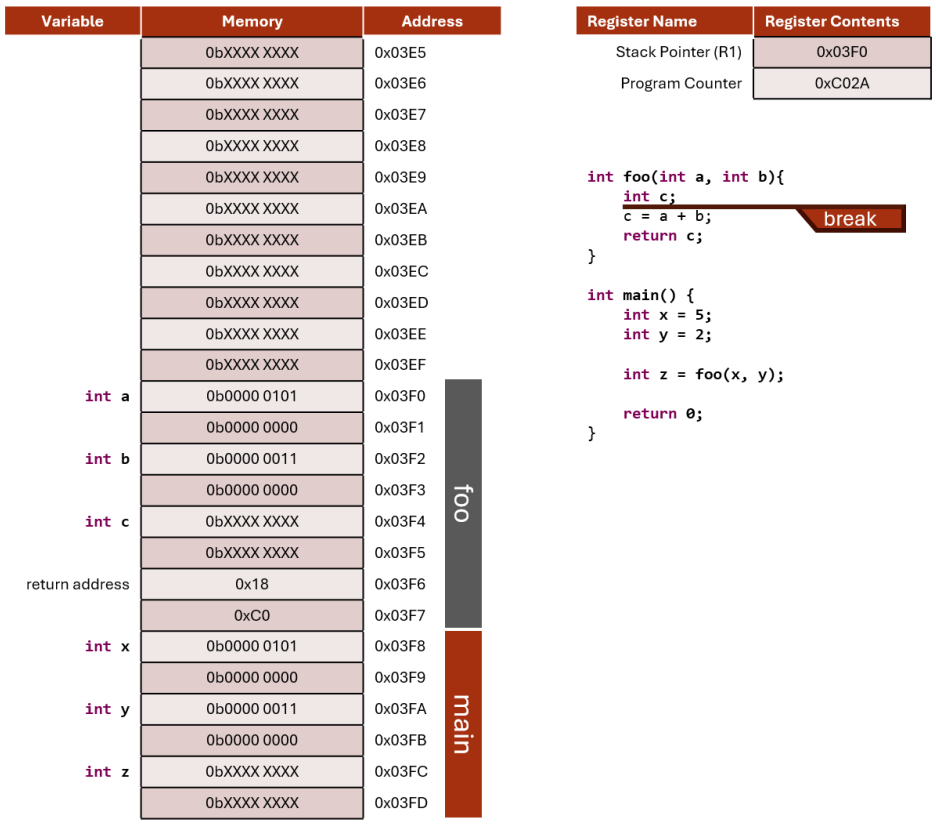


Figure 2.7.: Example C code and the condition of the stack up to the indicated break point. “X” indicates bit values that have been uninitialized.

heap to store data which will not be subject to the behavior of the stack. The stack is constantly growing and shrinking, and functions are called and completed. Sometimes we have a need to maintain data for a limited period of time but might need to exist for a time longer than the call of a function. This is where the heap comes in.

In the heap, we can specify that space for a particular data type, array, or structure may be allocated. The space is then made, and we are returned the address to that space. That space can then be used like any other variable; however, it is then our responsibility to deallocate that space when we are finished using it. If we do not deallocate it, then that memory will be locked out for the rest of the program's life span. If we continue using the heap and never freeing up the memory when we are finished, this leads to a **memory leak**. If this process continues over time, the heap will grow so large that it consumes too much of the memory range it shares with the stack and will begin overwriting it, resulting in all sorts of undesirable behaviors.

2.11.3.1. Malloc

To allocate space in the heap, we use the function "malloc", which comes from the stdlib library. Malloc is short for memory allocation. This function will reserve a specified number of bytes in the heap and return a pointer to that space. We can then manipulate the data reserved by malloc by dereferencing the pointer it returns. Let's look at an example:

```
char* p = malloc(1);
```

This call to malloc reserves a single byte, which is the size of a character on all architectures. Malloc returns a void pointer (a pointer that doesn't point to any specific data type) and assign it to a character pointer, as a character pointer will point to a single byte. The type of pointer we assign the space to indicates how that space will be used as well.

It might be a bit strange to allocate space for a single character, so let's use malloc to allocate space for an entire string containing 6 characters (including the null character):

```
char* s = malloc(6);
```

We can then populate this string like we would any other array. Recall that the name of an array can be used interchangeably with the name of an array:

```
s[0] = 'H';
s[1] = 'e';
s[2] = 'l';
s[3] = 'l';
s[4] = 'o';
s[5] = '\0';
```

This has been very straightforward so far when it comes to malloc, as we've just indicated how many bytes we need to allocate and get the pointer to that space. However, this has been simple because each char only requires one byte. For instance, a float would require 4 bytes worth of space, so if we were allocating space for an array of floats, we would have to allocate 4 * n, where n would be the number of elements in the float array we wanted to create.

We could manually specify the number of bytes required, but this can lead to miscalculations and a lack of code portability as the data sizes may change between microcontroller C compilers. In order to write the most dependable malloc function call, we will utilize the sizeof function. Sizeof returns the number of bytes for the data type passed to it. Therefore, to use malloc to allocate a float array of size n we would write:

```
float* a = malloc(n * sizeof(float));
```

2.11.3.2. Free

After we are done using the space we created with malloc, it is then our responsibility to free that memory back up so that it may be allocated at a later time for different purposes. To do this, we simply call the function "free" and pass it the pointer to the allocated space. To free up our float array allocated previously, we would write:

```
free(a);
```

Now all data allocated in the heap at the address stored in "a" would be available to be reallocated. Failure to do so would result in potential memory leaks.

2.12. C Build Process

Now we've discussed the basic syntax of the C language, let's look into how we convert that syntax into an executable that can be ran on a computer. We've already introduced the tool responsible for this conversion, the compiler. The compiler reads the C source files and ultimately converts it into an executable file which contains the 1's and 0's of the architecture's instructions. However, the Compiler is actually composed of many different applications, each with a specific responsibility. We've already introduced some of these, such as the preprocessor, but here we'll see where they all exist within the compilation process.

The four main steps of the compilation process are as follows:

1. Preprocessor

2. Compiler
3. Assembler
4. Linker

Let's take a look at each stage's responsibilities in detail.

2.12.1. Preprocessor

The preprocessor runs before any of the other steps and is responsible for all lines of code that are preceded with the "#" symbol. These lines of code are called "**macros**" and are eliminated and replaced by C syntax in some way before the preprocessor passes the processed source code to the compiler. Preprocessor macros can be used to add constant values to locations in the C source code by using commands such as "#define" or used to add entire header files through "#include".

Let's consider an example of the preprocessor's effects on a C source file. Let's say I have the following C source files:

main.c

```c
#include "circle.h"
int main(){
    float r = 3.5;
    float area = circleArea(r);
    return 0;
}
```

circle.h

```c
#define PI 3.14
float circleArea(float r);
```

circle.c

```
#include "circle.h"
float circleArea(float r){
    return PI*r*r;
}
```

Here we have a file called “main.c”, which contains my main function (the file containing the main function is usually named “main” by convention). I have another c source file called “circle.c”, which contains a function for computing the area of a circle. I also have a shared header file, “circle.h”, which contains the function prototype for the area function and a definition for the value of PI. When compiling, each “.c” file gets compiled separately, so the preprocessor will be ran for both “main.c” and “circle.c”. Let’s see what these files look like after the preprocessor has run:

main.c

```
float circleArea(float r);
int main(){
    float r = 3.5;
    float area = circleArea(r);
    return 0;
}
```

circle.c

```
float circleArea(float r);
float circleArea(float r){
    return 3.14*r*r;
}
```

What we observe here is that the function prototype for the area has been appended to the top of each file, and anywhere the label “PI” was found,

it was replaced by its defined value, 3.14. In this example, that was only a single location in the circleArea() function. Now that all the preprocessor macros have been resolved, the code is ready to be sent to the compiler.

2.12.2. The Compiler

The compiler is responsible for interpreting the C source code into its assembly language equivalent. We haven't gone into the details of assembly, and we won't in this book, but let's take a look at what the compiler will do to each of our files. Note that the ".c" extension will be replaced with ".s" for each respective assembly file, as ".s" is the typical assembly file extension:

main.s

```
main:
SUB.W #8, R1
MOV.W #0, 4(R1)
MOV.W #16480, 6(R1)
MOV.W 4(R1), R12
MOV.W 6(R1), R13
CALL #circleArea
MOV.W R12, @R1
MOV.W R13, 2(R1)
MOV.B #0, R12
ADD.W #8, R1
RET
```

circle.s

```
circleArea:
PUSHM.W #7, R10
SUB.W #4, R1
```

```
MOV.W R12, @R1
MOV.W R13, 2(R1)
MOV.W @R1, R12
MOV.W 2(R1), R13
CALL #__mspabi_cvtfd
MOV.W R12, R4
MOV.W R13, R5
MOV.W R14, R6
MOV.W R15, R7
MOV.W #-31457, R12
MOV.W #20971, R13
MOV.W #7864, R14
MOV.W #16393, R15
MOV.W R4, R8
MOV.W R5, R9
MOV.W R6, R10
MOV.W R7, R11
CALL #__mspabi_mpyd
MOV.W R12, R8
MOV.W R13, R9
MOV.W R14, R10
MOV.W R15, R11
MOV.W R8, R12
MOV.W R12, R4
MOV.W R9, R12
MOV.W R12, R5
MOV.W R10, R12
MOV.W R12, R6
MOV.W R11, R12
MOV.W R12, R7
MOV.W @R1, R12
MOV.W 2(R1), R13
CALL #__mspabi_cvtfd
```

```
MOV.W R12, R8
MOV.W R13, R9
MOV.W R14, R10
MOV.W R15, R11
MOV.W R8, R12
MOV.W R9, R13
MOV.W R10, R14
MOV.W R11, R15
MOV.W R4, R8
MOV.W R5, R9
MOV.W R6, R10
MOV.W R7, R11
CALL #__mspabi_mpyd
MOV.W R12, R4
MOV.W R13, R5
MOV.W R14, R6
MOV.W R15, R7
MOV.W R4, R12
MOV.W R12, R8
MOV.W R5, R12
MOV.W R12, R9
MOV.W R6, R12
MOV.W R12, R10
MOV.W R7, R12
MOV.W R12, R11
MOV.W R8, R12
MOV.W R9, R13
MOV.W R10, R14
MOV.W R11, R15
CALL #__mspabi_cvtdf
MOV.W R12, R10
MOV.W R13, R11
MOV.W R10, R13
```

```
MOV.W R11, R14
MOV.W R13, R12
MOV.W R14, R13
ADD.W #4, R1
POPM.W #7, r10
RET
```

Despite circleArea() being a small function in C, the resulting assembly is quite large. This is due to the fact that it uses many floating-point operations, which are notoriously difficult to pull off on a small microcontroller. Therefore, programmers prefer integer operations, when possible, as they are considerably smaller and easier for the processor to handle from a performance perspective.

2.12.3. The Assembler

The assembler takes the assembly files created by the compiler and converts them into object files. Object files are part machine code, part symbols. The symbols represent things that aren't available in the source and must be resolved within other files, such as the implementation of the "circleArea()" function call in main. Main understands that "circleArea()" is a function with certain parameters because of the prototype that was included in the "circle.h" header file. Main will need to generate a symbol for that function so that it can be resolved in the next step: the linker. The location where the symbol will belong can be seen in the assembly code above for main; the line containing "CALL #circleArea" will have a symbol to be linked for "#cirlceArea".

2.12.4. The Linker

The linker's job is to take all the object files, resolve the symbols, and create the executable. This is the stage where all unimplemented functions

across all the object files will be tied to their appropriate definitions. Once the linker is completed, the executable file is produced, containing the machine code to be ran by the processor.

2.12.5. The Loader

A fourth step exists for embedded system applications, which is the loader. The loader is a tool responsible for placing the executable into the microcontroller's memory. Small-scale microcontrollers often simply store their program in the text region of memory, and this region must be completely re-written when updating the code.

2.12.6. Cross Compilers

We've explored the process of compilation, but there is one more important detail to acknowledge when it comes to building C programs for embedded systems, and that is the processor that is building the program and what processor the program is to be ran on. In typical software development, when writing applications to be ran on a desktop or laptop computer, the type of machine the code is being built on is also the same machine the code will be intended to be ran. Therefore, most compilers are made to build for the machine doing the building, but that is not the case for embedded systems.

When writing programs for microcontrollers, such as the MSP430G2553, we are writing and building the code on a desktop or laptop, which is most likely an x86 architecture, and running the code on an MSP430 architecture. Therefore, we need a special compiler known as a cross-compiler. A cross compiler will build code on one machine to be ran on a machine of a different architecture. The machine doing the building is known as the **host** machine, and the machine the program is built for is known as the **target** machine.

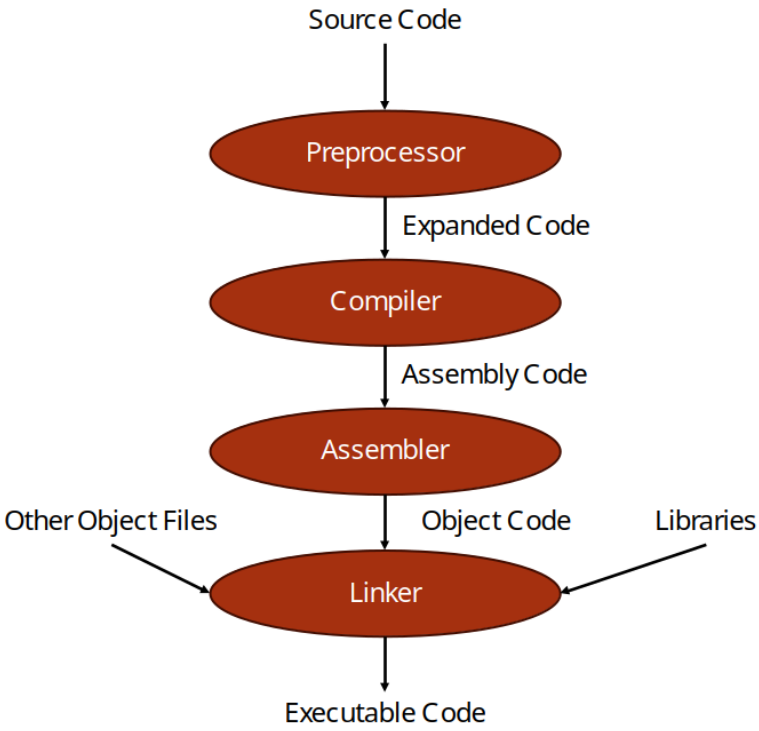


Figure 2.8.: The compilation process

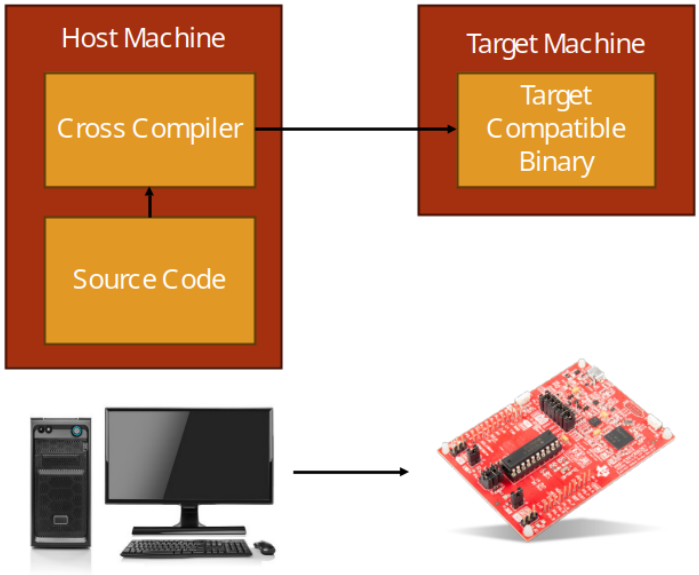


Figure 2.9.: The Cross-Compilation Process

2.13. Integrated Development Environments

To aid with the complexity of running cross compilers, editing code, and the many other needs required for creating programs for embedded systems, programmers often use an **integrated development environment** (IDE). IDEs are applications which usually include a variety of tools to help with software development. For embedded systems, this typically includes the following:

- Text Editor and Syntax Highlighter
- Build Tools
- Debugging Tools

Even more features may come in other IDEs, but for embedded systems these are the most common.

2.13.1. Code Composer Studio

When developing for MSP430 microcontrollers, or any TI microcontroller, Code Composer Studio (CCS) is the IDE provided by TI to support their products. CCS contains the all the tools listed in the previous section, plus a few more, such as a resource explorer, where example programs can be imported into your CCS environment to quickly get started developing code.

2.13.1.1. Code Composer UI Overview

Let's take a look at the user interface (UI) of the CCS environment. We'll start by highlighting the large section in the center of the window, the text editor. The locations of these windows shown here are the defaults, but they can be moved around per the user's preferences. The text editor

provides syntax highlighting and a space to edit the code. It also can support the addition of breakpoints, which when in debugging mode will allow the processor to stop execution at the line of code marked with a breakpoint and allow the user to inspect things such as memory content and variable values.

On the left side you can see several directories with different items inside. This is the project directory and it displays the contents of the workspace directory on your computer. The workspace directory is a location you specify where CCS will store your projects. A project is a collection of source files (.c, .h) and build scripts that tell the MSP430 cross compiler what microcontroller the project is building for and how to build it. By organizing everything into a project directory, a lot of the intricacies and complications of cross compilation is managed by CCS rather than the user having to remember all the details. Each directory listed here contains a different CCS project, and can be for a variety of TI microcontrollers.

On the top, we have the toolbar menu, which gives the user access to all of CCS's features and tools. The toolbar has everything organized under the labels: file, edit, view, project, run, tools, scripts, window, and help. There are a lot of features packed under these labels, but for convenience, several of the more common ones are broken out underneath with several icons to indicate their purpose. We'll highlight the most basic ones here:

- Hammer Icon – builds the active project
- Bug Icon – builds the active project, uploads it to the target, and begins a debugging session
- Play Button – continues the current debug session
- Pause Button – pauses the current debug session
- Stop Button – stops the current debug session
- Step Into – steps into the next function or line of code in the debug session

- Step Over – steps over the next instruction in the debug session

The resource explorer is another very useful feature of CCS. When the program is first launched for the first time, you will be greeted with a welcome screen where the text editor window is. From this window, and from under the "view" tab, you can access the resource explorer. The resource explorer will allow you to search for example code to get started with development for the MSP430 and other TI microcontrollers.

To find a C example for the MSP430FR2553, we will open the resource explorer and navigate to MSP430 Microcontrollers->Embedded Software->MSP430Ware->Development Tools->MSP-EXP430GET->Peripheral Examples->Register Level->MSP430G2553. There are many to choose from here, each titled with something related to a particular hardware peripheral. To get started with a simple example, we can import "msp430g2xx3_1.c" into our workspace by hovering over the name, clicking the three dots that appear, and selecting "Import to CCS IDE", as shown in Figure 2.11. We will then be prompted to select a microcontroller, as this example can work for two different devices. Select "MSP430G2553". We will then see a new project has been added to our workspace.

The imported project can be expanded by clicking the directory in the project explorer. Inside, we will find several files that all maintain the build configuration for this CCS project, along with a single ".c" source file. If we double click it, it will open, and we will be greeted with what we see in Figure 2.10. This example code will simply flash the LED on the MSP-EXP430G2ET development board. If the board is connected to the host computer through USB, we can click the bug button to build, upload, and run the code. When the code is successfully uploaded, it will enter debug mode and the code will automatically be paused at the beginning of main. You will need to press the play button to begin execution.

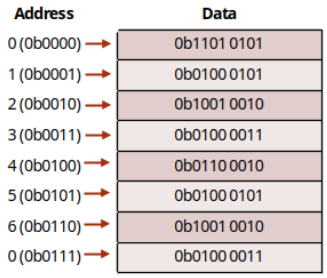


Figure 2.10.: The Code Composer Studio IDE

Figure 2.11.: A screenshot of a computer Description automatically generated

....

2.14. Recap

In this chapter we have learned why we program embedded systems in C, the basic syntax of C, and how C variables related to memory in terms of bytes occupied and location. We also have been introduced to the generic computer memory map and what each region is used for and how it operates.

Figure 2.12.: Where to find MSP430G2553 Demo Code in the Resource Explorerv

Part II.

Digital I/O and Program Structure

3. General Purpose Input/Output (GPIO)

3.1. Learning Objectives

In this chapter we will learn:

- What peripherals are
- What General Purpose Input/Output is
- How to control peripherals by writing to registers
- Bit masking
- Functionalizing code

3.2. Interfacing with the Outside World

When we introduced the basic components of a computer in Chapter 1, we mentioned they have five primary components: Memory, ALU, Control, Input, and Output. For any computer to have any use, it must have some way to interface with a user, which is the purpose of the input and output components. On your desktop or laptop computer, you may think of these as physical components, such as your keyboard, mouse, monitor, speakers, etc. These are of course necessary for you to interact with your computer.

For an embedded system, being able to interface with another system is of upmost importance and may require less-obvious needs. Embedded systems may need to interface with another circuit by reading a digital voltage and interpreting it into a 1 or 0. They may need to read an analog voltage and convert it into a digital value for processing, such as reading the signal from a microphone. They may need to interface with another microcontroller and communicate using a specific protocol. Given all of these possible input and output needs an embedded system might have, how are these interfaces created? The answer comes in the form of a **peripheral**. A peripheral is a separate hardware component included in the microcontroller's architecture to perform some task, typically providing one of these input or output interfaces.

3.3. General Purpose Input Output

Let's focus on the most fundamental peripheral found in nearly every single microcontroller, the **General-Purpose Input Output** (GPIO). The GPIO peripheral allows the microcontroller to read or set digital voltages on certain pins of the chip. This allows the microcontroller to create some of the most basic forms of inputs and outputs, such as illuminating an LED or reading a button press.

First, let's recall what is meant my "digital" voltage. A digital voltage is a voltage that represents either a 1 or a 0. How these 1's and 0's are represented may vary depending on the microcontroller, but for the most part, a 0 is represented by 0 volts, and a 1 is represented by the operating voltage of the microcontroller, which tends to be either 5 or 3.3 volts. When speaking ambiguously about these voltages, we will often use the terms **high** and **low** voltages to refer to whatever the voltage is that represents a 1 and a 0 in the system, respectively. The microcontroller will interpret any voltage that is near those values as such, but voltages that fall in between may be up to the microcontroller's interpretation and should avoided when using GPIO.

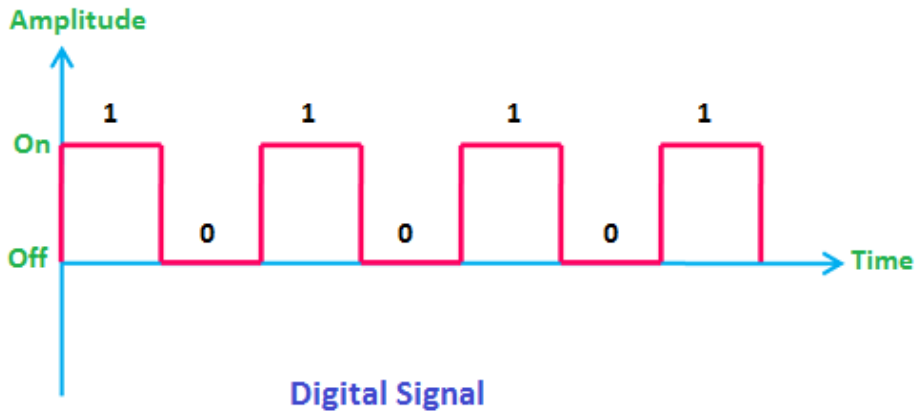


Figure 3.1.: Example of a Digital Signal

<Note: Modify image to show “gray” areas between digital values. Change “On” and “Off” to voltage values>

3.3.1. Controlling an LED

As mentioned before, these digital signals can be used to apply either high or low voltages to the external pins of the microcontroller. We can connect these pins to external circuits to do things such as illuminate an LED or read a button press. Let’s consider an example of illuminating an LED using the GPIO pin of a microcontroller. Let’s say we have generic microcontroller with a GPIO peripheral that controls a pin labeled “P1”. Attached to P1 we have a small resistor in series with an LED, which is then connected to ground, as shown in Figure 3.2.

When the GPIO provides a high voltage on P1, a current is generated which flows from P1, through the resistor, then the LED, then to ground. When current flows through the LED, it emits light. If the GPIO writes a low voltage to P1, which low is typically always 0 for GPIO, we now have

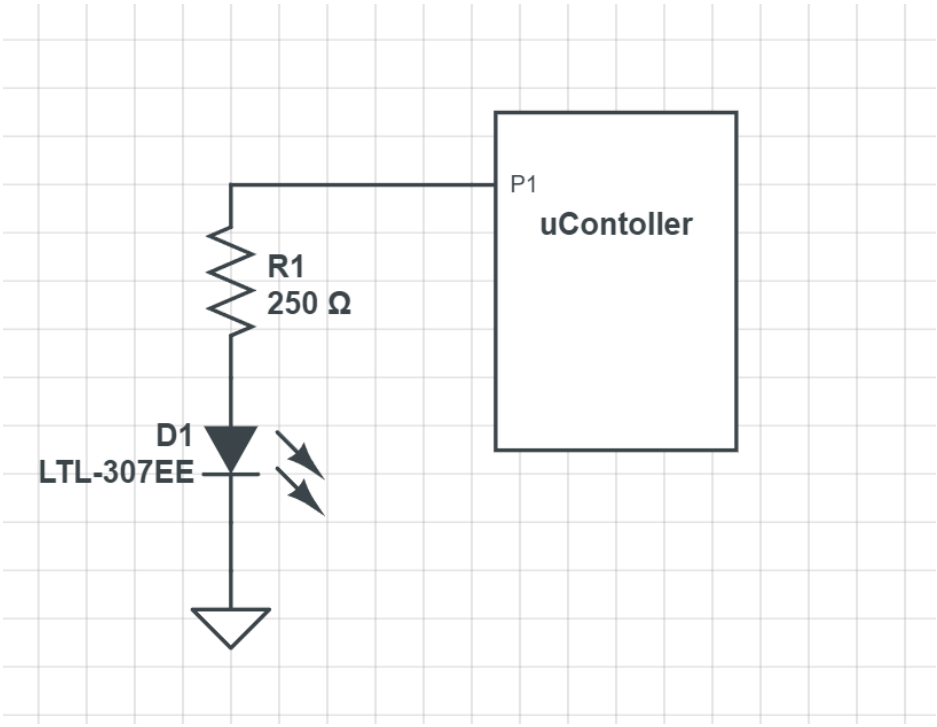


Figure 3.2.: Example GPIO LED application

no electrical potential across the LED, as the voltage on ground is also 0. For current to flow, a path from a higher voltage to a lower voltage must be provided, and here we have none, so the LED turns off.

3.3.2. Reading a Button Press

Let's consider another common GPIO application, reading a button press. To read a button press, we need to have a switch that can control when a high or low voltage is presented to the GPIO input pin. We'll use the same generic microcontroller as in the previous example, but this time P1 will be used to read the voltage rather than produce a voltage. We'll connect P1 to a resistor that is wired to a high voltage source, and also to a switch which will connect the pin to ground, which can be see in Figure 3.3.

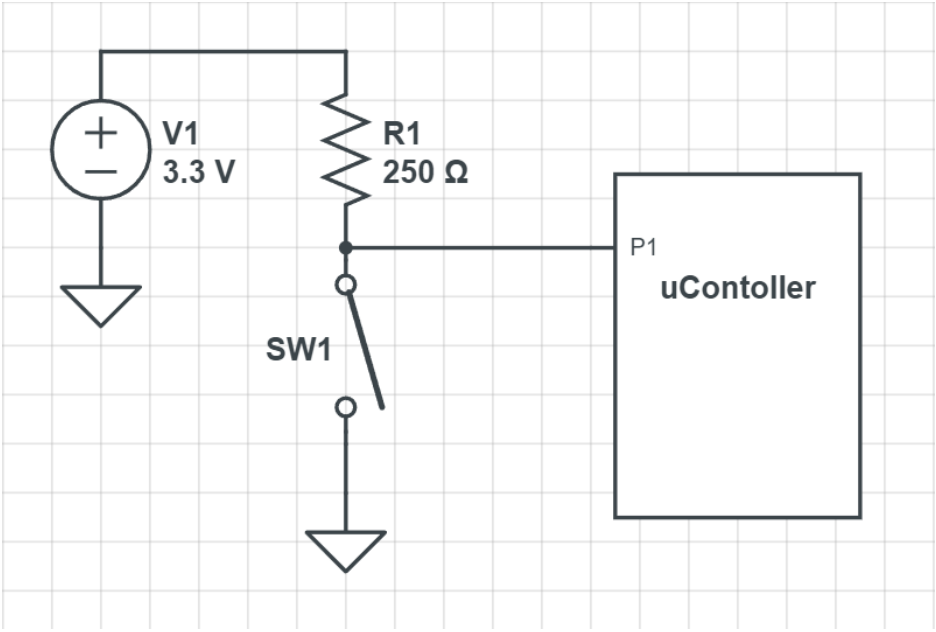


Figure 3.3.: Example GPIO LED application

When the switch is closed, P1 is directly connected to ground, and will read 0 volts; all the voltage is dissipated across the resistor in this scenario as well, and none of the current flows into the microcontroller through P1. When the switch is open, there is no longer a direct path to ground, and the current is forced to flow into P1 and through the microcontroller. The input for GPIO pins tends to be of an extremely high resistance, and therefore most of the voltage is dissipated through the microcontroller. This means that only a very small (negligible) amount is lost across the resistor and P1 reads a high voltage.

We've now covered the two most fundamental input/output scenarios for an embedded system. This is good in theory, but how do we achieve this on the MSP430FR2355? Let's learn how to control the GPIO on this microcontroller.

3.3.3. Peripherals and the Memory Map

From the perspective of an individual instruction, there are only two things a line of code can control: the operation (add, subtract, etc.) and the memory location it writes the result. So, in order to control these peripherals using program instructions, they need to appear like memory locations so the instructions can write to them. Each peripheral contains many **control registers**, which are just like any other memory unit, except the data written to these registers will dictate how that peripheral behaves. For GPIO, that might be indicating whether a pin is an input or output, whether we write a high or a low to a pin, or if we read a high or low on a pin.

3.3.4. The MSP430FR2355 Memory Map

Microcontroller programming is heavily centered around reading and writing to peripheral control registers. Therefore, the memory map is very important to the programmer developing code for the microcontroller.

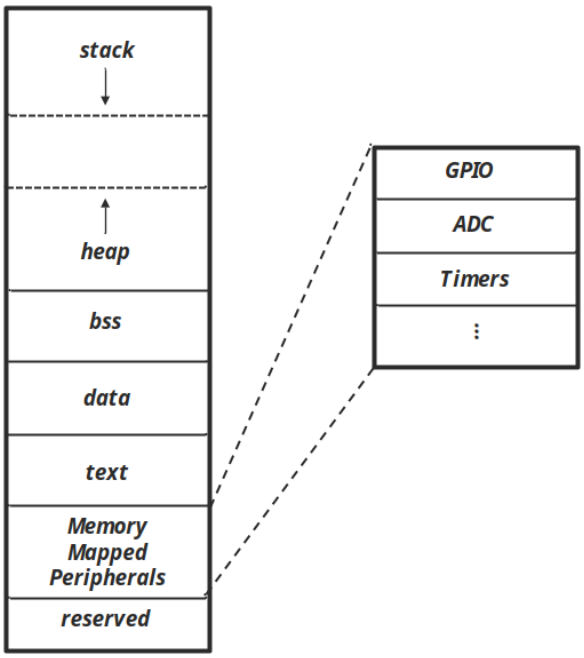


Figure 3.4.: Generic Example of Memory Mapped Peripherals

Each microcontroller has a different memory map, so to know where our control registers reside in memory, we will need to consult the datasheet or user's manual to find out. A **datasheet** is an accompanying document that describes many of the physical characteristics of the microcontroller, such as physical dimensions of the packaging, pinout descriptions, electrical characteristics, etc. Sometimes the datasheet may contain information about the memory map, but for TI products, the user's manual contains this information. The **user's guide** has detailed descriptions of the microcontroller's architecture, memory map, and peripherals; essentially everything the programmer needs to know to use the microcontroller. We'll learn that much of embedded systems involves reading user's guides and datasheets to understand how to integrate various existing components into our designs.

The MSP430G2553's memory map can be seen in Figure 3.6. In this figure, we see the memory map's regions divided up into various address ranges. Some of the regions we've encountered before aren't present here, such as data, BSS, and stack, and are all simply listed as RAM. Since these regions are all regions that can be written to at runtime, their boundaries are defined by software and not the hardware itself. The compiler is responsible for setting those boundaries and creating instructions that behave within them. This diagram shows boundaries that have some important physical hardware connections, such as the peripherals. We can see in this diagram what memory locations belong to peripherals.

Table 8. Memory Organization

		MSP430G2153	MSP430G2253 MSP430G2213	MSP430G2353 MSP430G2313	MSP430G2453 MSP430G2413	MSP430G2553 MSP430G2513
Memory	Size	1kB	2kB	4kB	8kB	16kB
Main: interrupt vector	Flash	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0
Main: code memory	Flash	0xFFFF to 0xFC00	0xFFFF to 0xF800	0xFFFF to 0xF000	0xFFFF to 0xE000	0xFFFF to 0xC000
Information memory	Size	256 Byte	256 Byte	256 Byte	256 Byte	256 Byte
	Flash	010FFh to 01000h	010FFh to 01000h	010FFh to 01000h	010FFh to 01000h	010FFh to 01000h
RAM	Size	256 Byte	256 Byte	256 Byte	512 Byte	512 Byte
		0x02FF to 0x0200	0x02FF to 0x0200	0x02FF to 0x0200	0x03FF to 0x0200	0x03FF to 0x0200
Peripherals	16-bit	01FFh to 0100h	01FFh to 0100h	01FFh to 0100h	01FFh to 0100h	01FFh to 0100h
	8-bit	0FFh to 010h	0FFh to 010h	0FFh to 010h	0FFh to 010h	0FFh to 010h
	8-bit SFR	0Fh to 00h	0Fh to 00h	0Fh to 00h	0Fh to 00h	0Fh to 00h

Table 8. Memory Organization

		MSP430G2153	MSP430G2253 MSP430G2213	MSP430G2353 MSP430G2313	MSP430G2453 MSP430G2413	MSP430G2553 MSP430G2513
Memory	Size	1kB	2kB	4kB	8kB	16kB
Main: interrupt vector	Flash	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0
Main: code memory	Flash	0xFFFF to 0xFC00	0xFFFF to 0xF800	0xFFFF to 0xF000	0xFFFF to 0xE000	0xFFFF to 0xC000
Information memory	Size	256 Byte	256 Byte	256 Byte	256 Byte	256 Byte
	Flash	010FFh to 01000h	010FFh to 01000h	010FFh to 01000h	010FFh to 01000h	010FFh to 01000h
RAM	Size	256 Byte	256 Byte	256 Byte	512 Byte	512 Byte
		0x02FF to 0x0200	0x02FF to 0x0200	0x02FF to 0x0200	0x03FF to 0x0200	0x03FF to 0x0200
Peripherals	16-bit	01FFh to 0100h	01FFh to 0100h	01FFh to 0100h	01FFh to 0100h	01FFh to 0100h
	8-bit	0FFh to 010h	0FFh to 010h	0FFh to 010h	0FFh to 010h	0FFh to 010h
	8-bit SFR	0Fh to 00h	0Fh to 00h	0Fh to 00h	0Fh to 00h	0Fh to 00h

3.3.5. The MSP430G2553 GPIO Peripheral

The MSP430G2553 has a fairly standard GPIO peripheral. The TI documentation refers to it as simply "Digital I/O", but this is still a general-purpose input/output peripheral. The MSP430G2553 has a 40-pin chip, and many of these pins can be controlled by the GPIO peripheral. However, the GPIO peripheral itself can only manage up to 16 pins per module, so the architecture includes three individual GPIO peripherals, allowing control over nearly every pin, excluding the pins for power and ground. Three GPIO units can potentially control up to 48 pins, but since the package only has 40, some of these are unused.

Figure 3.7 shows the block diagram of the MSP430G2553 architecture. This diagram contains a lot of items we aren't familiar with yet, but it does contain some that we can identify, based on our understanding of the five basic components of a computer. To the far left, we can see a block labeled "16-MHz CPU including 16 registers". This is the processor (ALU and control) component of the computer. That means everything else

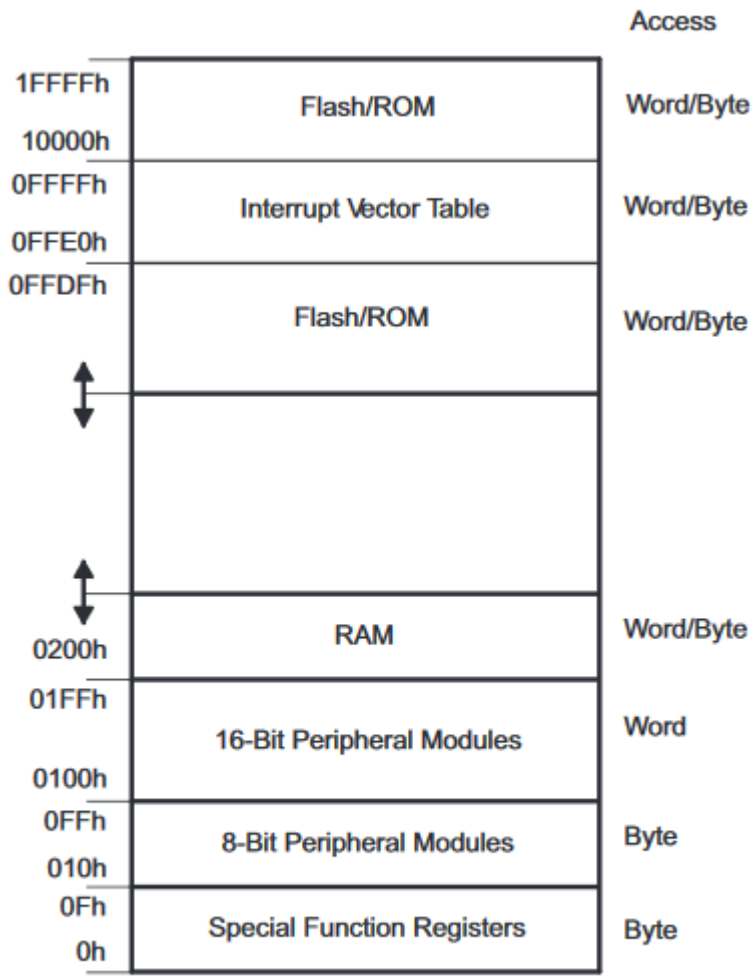


Figure 1-2. Memory Map

Figure 3.5.: MSP430G2553 memory map

must be classified as input/output or memory. We mentioned earlier, that to control the input/output, or peripherals as we now know them, they must appear as memory addresses. Figure 3.6 indicated this by illustrating control registers as locations within a contiguous block of memory. This is how the hardware might "appear" to the instructions; but in reality, these memory addresses are spread out across multiple hardware components, such as our GPIO modules. They appear as memory addresses due to the fact they are wired into the memory bus. The **memory bus** is what connects all components to the memory map, often accompanied by a memory bus controller. When the processor reads or writes to a location, that location is accessed through this bus, whether it's actual RAM or a peripheral control register. We can identify our three GPIO peripherals in Figure 3.7 as the blocks in the top right of the diagram labeled "I/O Ports".

We see that each GPIO peripheral has a different nomenclature describing the module. Each module is referred to as "Port" followed by a number. At the top right of the diagram in Figure 3.7, we see a GPIO module called "Port P3". Each port controls 8 GPIO pins each.

3.3.6. Which pins have GPIO?

From here you might be wondering, which pins do each of these ports control? And that is a question that has to be answered by the datasheet. Within that document, it will identify which pins on the package are connected to which ports on the GPIO module. Some microcontrollers might come in different package sizes as well, so it's important to consult the documentation for this. Figure 3.8 shows an excerpt from the datasheet indicating the pinout of the chip. On this diagram, we will notice each pin has many different acronyms associated with it. This is because apart from the GPIO, several of the pins might share a connection with some other peripherals. We can identify the pin with a GPIO connection based on the ones labeled with a port / pin combination. We mention earlier that each

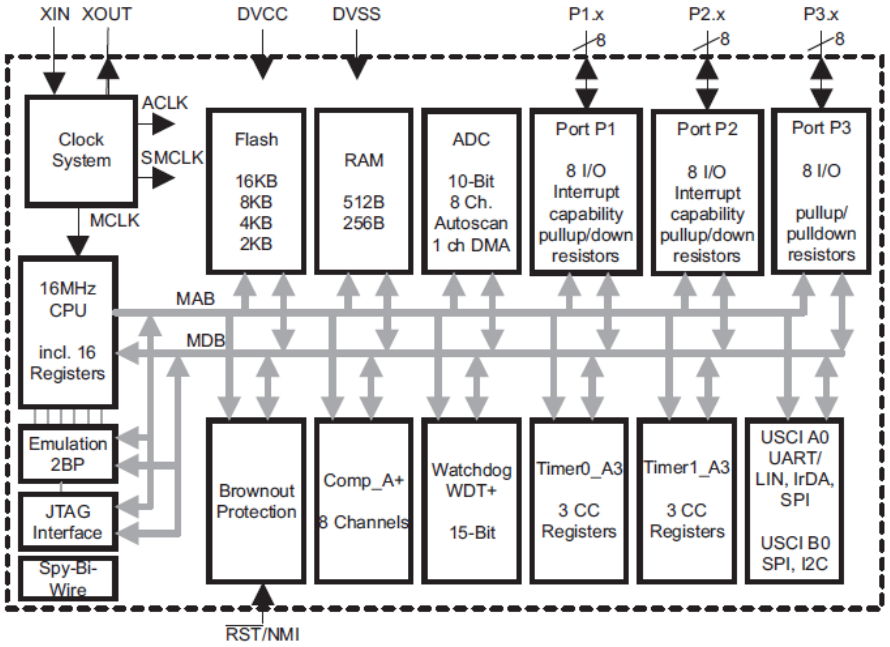


Figure 3.6.: MSP430G2553 Functional Block Diagram

GPIO module is separated by port, and each port can control up to 8 pins each. Therefore, the pin with the label, P1.3, has Port 1 Pin 3 attached to it. And similarly, the pin labeled P2.0 has Port 2 Pin 0 attached to it.

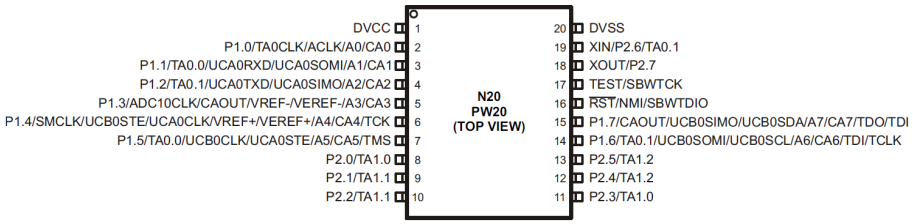


Figure 3.7.: MSP430G2553 Pinout Diagram

Since we're using a development board, the pins on the chip have been connected to the male header pins on the sides of the board. To make identification of the port and pin each one is attached to easy, the development board has included some images that use the same abbreviations to indicate which peripherals are available on which pins. This diagram is shown in Figure 3.9.

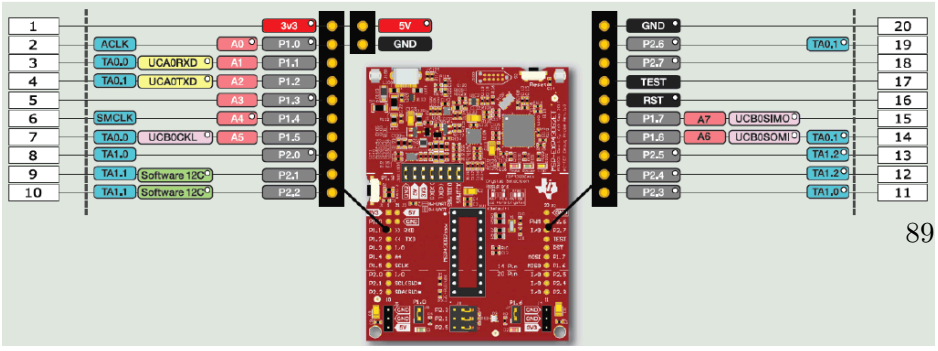


Figure 3.8.: MSP-EXP430G2ET Pinout Diagram

3.4. Controlling the GPIO

We have discussed the why and where of the GPIO peripherals; now let's discuss the how. In order to control the GPIO hardware we have to write to its control registers. The control registers are memory units in the hardware itself, where each bit in the register alters the behavior of the peripheral. There are many control registers associated with the GPIO peripheral, but to start let's focus on the most essential.

3.4.1. Direction Register

Before doing anything with the GPIO pins, we first have to establish one thing: is the pin an input or an output? Are we reading or writing to the pins? The direction register sets this property for each pin of the microcontroller that it controls. The register itself is 8 bits wide, and if you recall, each GPIO module controls 8 pins. Therefore, each bit in the direction register will determine the direction (input or output) of the pin. For the MSP430G2553, writing a 0 to a bit sets the corresponding pin as an input, and writing a 1 sets it as an output. We will see this relationship of an individual bit within a register corresponding to a pin on the microcontroller used many times when discussing GPIO. Figure 3.10 illustrates how the direction register of P1 is tied to each pin that it controls. For the sake of illustration, the pins appear in sequence, but as it can be seen in Figure 3.8, these pins can be wired to various legs all around the chip and not in any particular order.

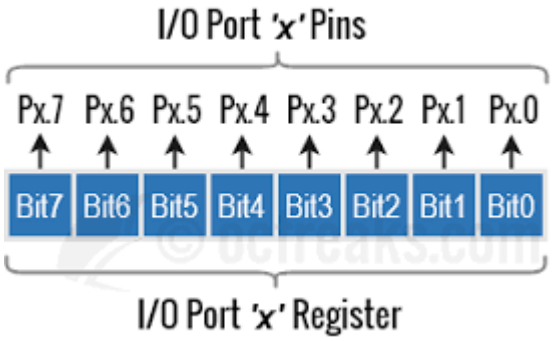


Figure 3.9.: Timeline Description automatically generated

Figure 3.10: Port bit and pin relationship

3.4.2. Output Register

The output register controls whether a high or low voltage is written to a pin. Like the direction register, the output register is an 8-bit register with each bit wired to control a single, corresponding pin. Writing a 1 or a 0 to an individual bit in the register will cause a high or low voltage, respectively, to be produced on its corresponding pin. In order for these voltages to be produced, the corresponding pin must have been configured as an output pin in the direction register.

3.4.3. Input Register

The input register is another 8-bit register that contains the digital value of the corresponding voltage on a pin. With the input register, we can read digital voltages produced from other circuits. Each bit will hold a 1 or a 0 if a digital high or low is read on the corresponding pin. Unlike

the direction and output register, this register is read only, so it cannot be written to by the software.

3.4.4. Pullup or Pulldown Resistor Enable Register

In Figure 3.3 we described how to use GPIO to read a button press. This example used a switch to connect a pin directly to ground when pressed, but when disconnected, a "pull-up" resistor connected the pin the voltage source. The use of pull-up resistors and their inverse, a pull-down resistor (which connects to ground rather than the voltage source), are common enough in embedded systems that the GPIO module includes them right in the peripheral hardware so we don't have to wire our own externally!

The Pullup/Pulldown Enable Register is 8-bit wide and will enable a pull-up or pull-down resistor based on whether the corresponding bit contains a 1 or 0. If the bit contains a 1, the resistor is enabled. If it contains a 0, it is disabled. The next question is how do we select a pull-up or pull-down? In this case, we let the output register do double duty for us.

Since pull-ups or pull-downs are only relevant when reading a voltage, such as in the button example earlier, we really only care to enable them when the direction register has configured the pin of interest as an input. If the pin is an input, then the bits in the output register that correspond to that pin have no purpose. Therefore, we can give those pins a purpose by allowing them to select a pull-up or pull-down resistor when the corresponding pin's pullup/pulldown register bit has been enabled.

3.4.5. The GPIO Hardware

Let's take some time and observe how the control registers create the hardware functionality we just described.

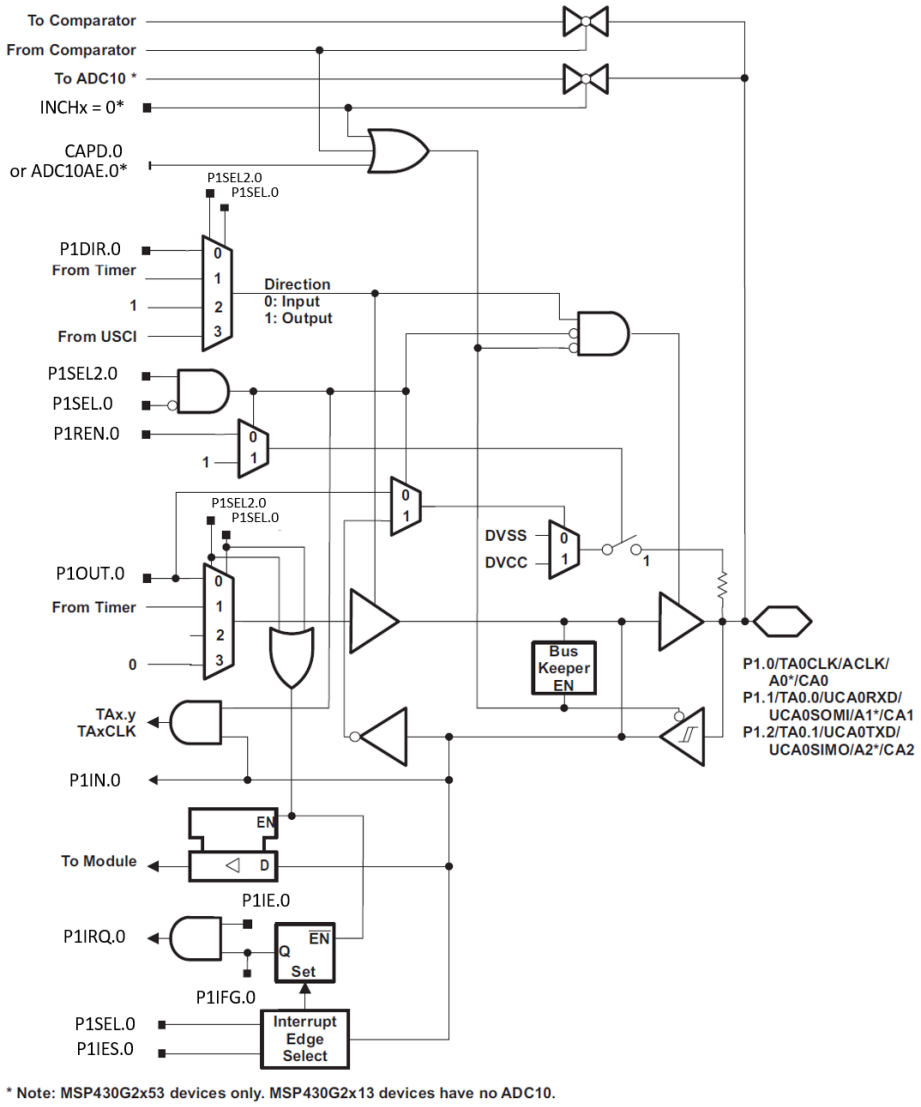


Figure 3.10.: GPIO Pin control schematic for Port 1 Pin 0

3.4.6. Example: Setting an LED

Let's revisit the example we discussed earlier where we used the GPIO to turn on an LED connected to one of the pins. For this example, we'll say there is an LED connected to port 1 pin 3. If we want to write a high voltage to that pin, we must first set the pin's direction as an output. We know that we'll need to write a 1 to the appropriate position within the direction register, but how do we achieve this in a C program? We know that registers are mapped into memory, so we need to know the address where it resides. We can find this address by consulting the documentation.

The table in Figure 3.12 shows the address ranges for the P1, P2, P3 peripherals. This table gives us each register's name and associated address. We can see that P1's direction register is mapped to address 0x022.

Now that we know P1's directional register address, 0x022, how do we write to that memory location? In the C programming language, we learned that pointers are variables that hold the address to another variable. Therefore, we can make a pointer variable, assign it the address of the register, then dereference the pointer to write to the desired register. This code may look like:

```
volatile unsigned char* p1dir = (unsigned char*) 0x022;
```

Here we used an unsigned character pointer to point to our register. The reason an unsigned char was used is because the char data type is 8 bits, and so is our direction register. We also declared it unsigned, because the value we write to a control register such as this has no numerical significance, and therefore the two's complement interpretation is meaningless for that number. We also had to cast the address to the type of pointer we are assigning. Another aspect to note is the inclusion of the **volatile** keyword. The volatile keyword prevents the compiler from optimizing out any instructions containing the variable. Sometimes, register assignments may look confusing to the compiler, or as if they do not do anything at all

Table 8-2. Digital I/O Registers

Port	Address	Acronym	Register Name	Type	Reset	Section
P1	020h	P1IN	Input	Read only	Unchanged	Section 8.3.1
	021h	P1OUT	Output	Read/write	Unchanged	Section 8.3.2
	022h	P1DIR	Direction	Read/write	00h with PUC	Section 8.3.3
	023h	P1IFG	Interrupt Flag	Read/write	00h with PUC	Section 8.3.4
	024h	P1IES	Interrupt Edge Select	Read/write	Unchanged	Section 8.3.5
	025h	P1IE	Interrupt Enable	Read/write	00h with PUC	Section 8.3.6
	026h	P1SEL	Port Select	Read/write	00h with PUC	Section 8.3.7
	041h	P1SEL2	Port Select 2	Read/write	00h with PUC	Section 8.3.8
	027h	P1REN	Resistor Enable	Read/write	00h with PUC	Section 8.3.9
P2	028h	P2IN	Input	Read only	Unchanged	Section 8.3.1
	029h	P2OUT	Output	Read/write	Unchanged	Section 8.3.2
	02Ah	P2DIR	Direction	Read/write	00h with PUC	Section 8.3.3
	02Bh	P2IFG	Interrupt Flag	Read/write	00h with PUC	Section 8.3.4
	02Ch	P2IES	Interrupt Edge Select	Read/write	Unchanged	Section 8.3.5
	02Dh	P2IE	Interrupt Enable	Read/write	00h with PUC	Section 8.3.6
	02Eh	P2SEL	Port Select	Read/write	C0h with PUC(1)	Section 8.3.7
	042h	P2SEL2	Port Select 2	Read/write	00h with PUC	Section 8.3.8
	02Fh	P2REN	Resistor Enable	Read/write	00h with PUC	Section 8.3.9
P3	018h	P3IN	Input	Read only	Unchanged	Section 8.3.1
	019h	P3OUT	Output	Read/write	Unchanged	Section 8.3.2
	01Ah	P3DIR	Direction	Read/write	00h with PUC	Section 8.3.3
	01Bh	P3SEL	Port Select	Read/write	00h with PUC	Section 8.3.7
	043h	P3SEL2	Port Select 2	Read/write	00h with PUC	Section 8.3.8
	010h	P3REN	Resistor Enable	Read/write	00h with PUC	Section 8.3.9

Figure 3.11.: GPIO Port 1 - 3 address table

from an algorithmic perspective, and therefore get "optimized" out of the program when compiled. Because of this, all pointers to registers that we create will include the volatile keyword.

Now that we have access to Port 1's direction register through the pointer, we can write Pin 3 as an output. Each bit in the register corresponds to the pin with the same enumeration, so we need to write a 1 in the first bit position (we begin counting at 0, so pin 0 is the first bit from the left side).

```
*p1dir = 0x01; // writes 00000001 to p1 direction register
```

We're only concerned about pin 0, so we'll just leave all the other pins as 0's, which will set them all as inputs for now. The only thing left to do is write to the output register to create a high voltage. Just as before, we'll begin by creating a pointer to the output register's address. The address according to Figure 3.12 is 0x021.

```
volatile unsigned char* p1out = (unsigned char*) 0x021;
```

Now we just need to write a 1 to the bit 0 position to create a high voltage on pin 0.

```
*p1out = 0x01; // writes 00000001 to p1 output register
```

And that's it! From here on we can write either a 1 or a 0 to the first bit on the output register to turn the LED on and off, respectively. Since all the other pins were configured as inputs, whether a 1 or a 0 was written to the other bit positions doesn't really matter. We'll see that a large component of embedded systems is reading and writing to these register memory addresses, which is known as **register level programming**.

From a hardware perspective, let's consider what the effects of these control register values have. There are many registers associated with the GPIO,

but since we've only set two registers for this application, we'll assume the others are all set to 0's. Figure 3.13 shows the hardware for Pin0 in Port 1 with the configurations specified in the previous lines of code. The red lines represent digital high values. From here we can see that The direction register enables the buffer which allows the high value in the bit 0 position of port 1 to be forwarded to the output. We'll also note that there are two signals, P1SEL0 and P1SEL1, which control the two multiplexors. When they are set as 0, as they are in this example, the values of the P1DIR and P1OUT register are forwarded to the circuit. The GPIO might have other modules connected to it as well, which is why the multiplexors appear in this circuit. We're only using the basic GPIO functionality here, so we leave those as 0 and effectively ignore any other peripheral hardware connected to this circuit.

3.4.7. Example: Reading a Button Press

For the next example, we'll revisit the example earlier where we read a button press. This time, we'll consider that we have a button configured as it was in Figure 3.3, with the button attached to Port 1 Pin 3. Just as we did in the LED example, we'll have to begin by specifying the pin's direction. We'll write a 0 to the bit 3 position of the direction register to achieve this.

```
volatile unsigned char* p1dir = (unsigned char*) 0x022;
*p1dir = 0x00; // writes 00000000 to p1 direction register
```

This will configure all 8 pins as inputs on port 1, but most importantly, pin 0 gets configured as such. For the specified circuit, everything is configured now, and all we have to do is read the input register to see if the button is pressed. Our button is connected to ground, so that means if we see a 0 in the bit 0 position, the button is pressed, and a 1 indicates otherwise. Just as with any other register, we'll need to make a pointer to interact with the associated memory location. The input register has an offset of

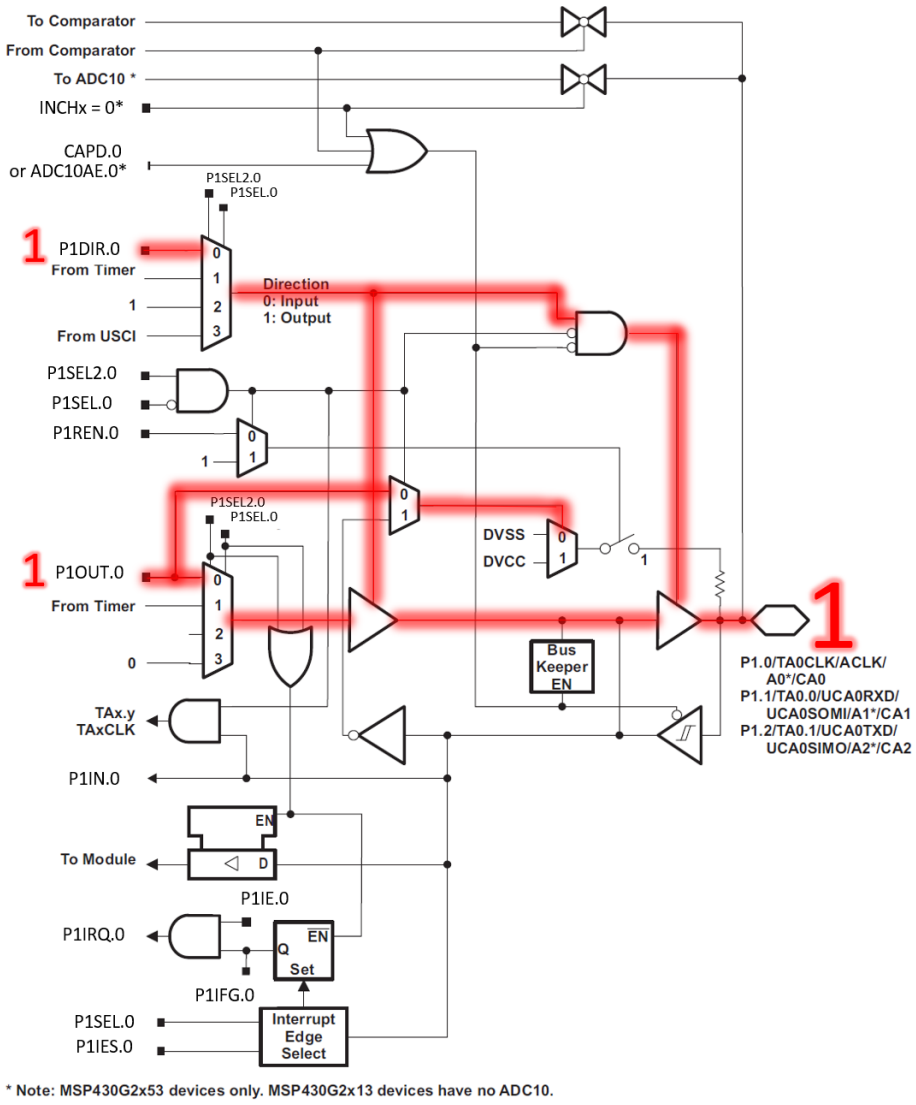


Figure 3.12.: Writing a high voltage to Port 1 Pin 0. Wires highlighted in red carry a digital high voltage (1), while unhighlighted carry a digital low (0). Note that some wires and components connect the output pin to other peripherals and may carry high or low voltages. For simplicity's sake, these wires have been ignored.

0 from the base address, so it will reside at the same address as the base address.

```
volatile unsigned char* p1input = (unsigned char*) 0x020;
volatile unsigned char p1data = *p1input; // reads p1input and stores the result
```

We can now evaluate the "p1data" variable to determine if the button is pressed.

The circuit in Figure 3.3 includes a pull-up resistor, but we discussed that the GPIO peripheral has the capability of configuring an internal pull-up resistor. We can configure the internal pull-up and then exclude the external pull-up resistor, reducing the circuit complexity and cost by removing components. To enable the internal pull up, we just need to set the enable bit in the pull-up/pull-down resistor enable register and set the associated bit in the output register to select which type of resistor we are enabling. The code to achieve this would appear as follows:

```
volatile unsigned char* p1dir = (unsigned char*) 0x022;
*p1dir = 0x00; // writes 00000000 to p1 direction register
volatile unsigned char* p1ren = (unsigned char*) 0x027;
*p1ren = 0x01; // writes 00000001 to p1 pull-up enable register
volatile unsigned char* p1out = (unsigned char*) 0x021;
*p1out = 0x01; // writes 00000001 to p1 output register
```

Now we can read the button press without the external pull-up. Note that the register address for the pull-up/pull-down resistor enable register is 0x06 from the base address.

Figure 3.14 shows the condition of the Port 1 Pin 0 hardware given the configuration above and when reading a high voltage. In this diagram we will be ignore the JTAG lines, as they are beyond the scope of what we're trying to illustrate here. The important things to notice are how the P1REN.0 line enables the switch, which is then connected to either power

(DVCC) or ground (DVSS) depending on P1OUT.0, which determines whether a resistor is a pull-up or pull-down.

3.4.8. Example Reading a Button Press and Setting an LED

Before moving on, let's consider one final example where we will combine the two previous examples. We'll say that we have an LED attached to Port 1 Pin 3 and a button on Port 1 Pin 0, which connects to ground when pressed. This circuit is illustrated in Figure 3.14. We want to write an application that will turn the LED on when the button is pressed. Let's consider what we'll need to achieve this.

Let's begin by creating pointers to the registers we want to configure:

```
volatile unsigned char* p1in = (unsigned char*) 0x020;
volatile unsigned char* p1out = (unsigned char*) 0x021;
volatile unsigned char* p1dir = (unsigned char*) 0x022;
volatile unsigned char* p1ren = (unsigned char*) 0x027;
```

Now we can write to each of those pointers. We want Port 1 Pin 0 as an output, and Port 1 Pin 3 as an input. This means we want a 0 in the fourth bit, and a 1 in the first bit, and we'll write 0's to all the others, because for this application, we don't care what they are because they aren't being used.

```
*p1dir = 0x01; // writes 00000001 to p1 direction register
```

The directions are set, let's also ensure that the LED starts set off, and we'll also enable the pullup resistor for pin 0.

```
*p1ren = 0x08; // writes 00001000 to p1 resistor enable register
*p1out = 0x08; // writes 00001000 to p1 output register
```

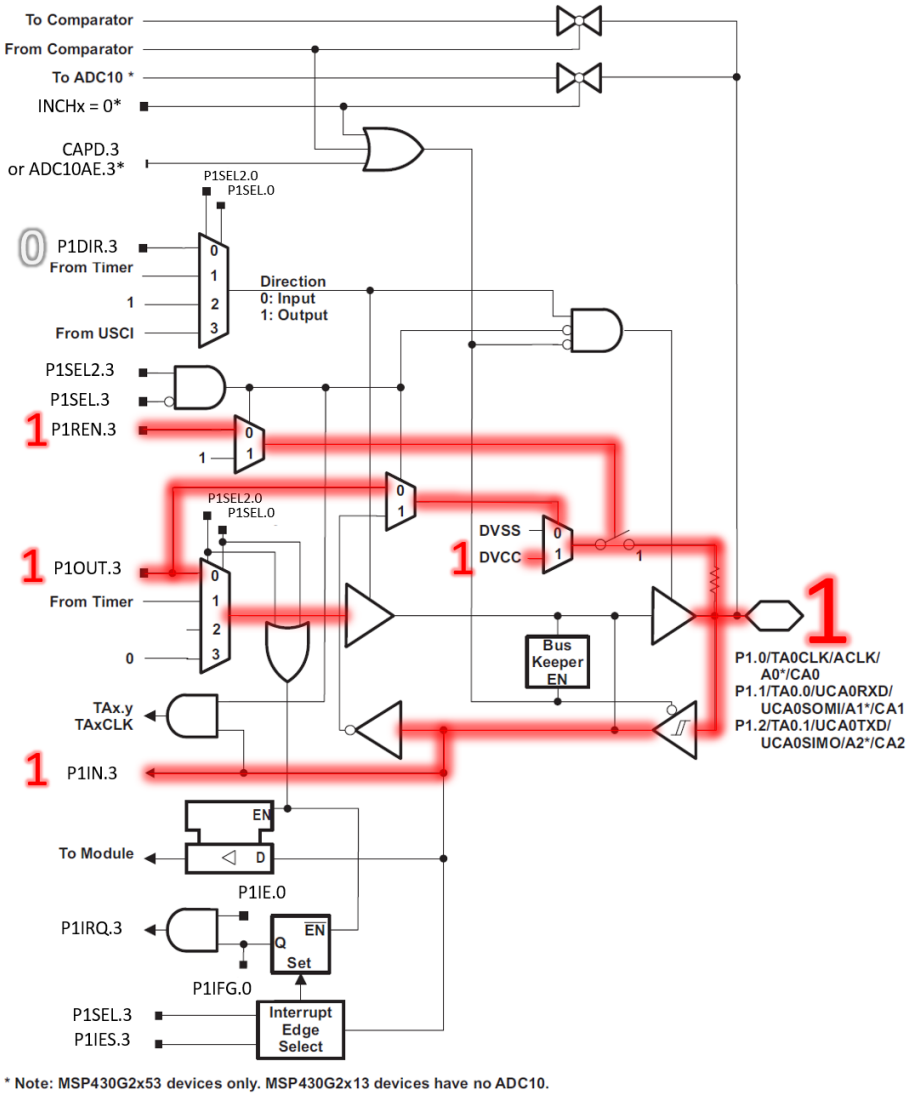


Figure 3.13.: Reading a high voltage from Port 1 Pin 3

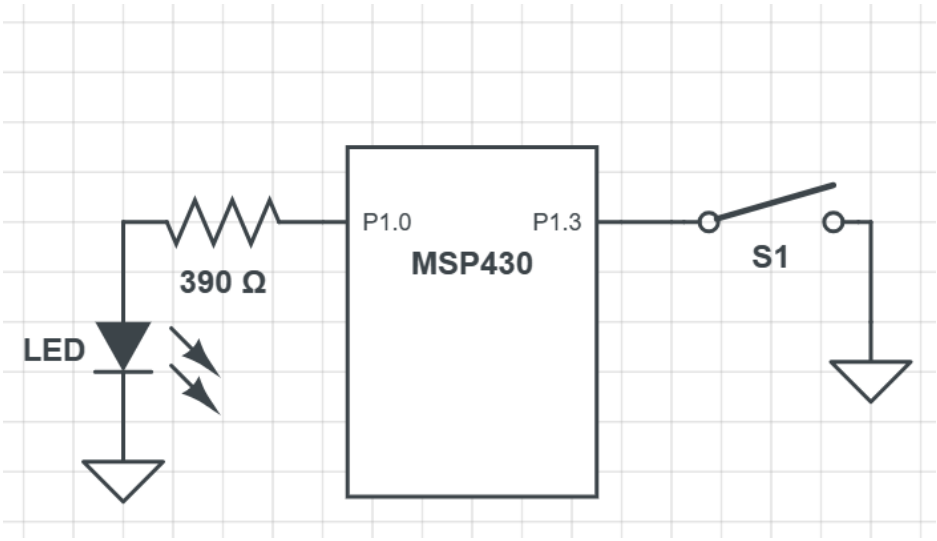


Figure 3.14.: Example circuit schematic

Remember that p1out sets the output voltage for pins configured as output but selects the pull-up or pull-down configuration for inputs that have their pull enable bit set. What's important here is that port 1 bit 0 has a 0 (to turn off the LED) and bit 0 has a 1 (to configure a pull-up resistor).

Lastly, we need to read the input register and see if port 1 bit 3 has a 1 or 0 to determine if the button is pressed, then set the appropriate LED condition.

```
if((*p1in & 0x08) == 0x00){
    *p1out = 0x09; // 00001001 (LED on, pull-up enabled)
} else {
    *p1out = 0x08; // 00001000 (LED off, pull-up enabled)
}
```

Here, the if condition dereferences the input register and checks if bit 3 is

0 (Using a process called "bit-masking". We'll discuss this later). If they are equal, then that means the button is pressed, and we want to turn the LED on. On the following line we do that by writing to the output register. We want to set a 1 in the bit 3 position, but we want to make sure we keep a 1 in the bit 0 position as well, otherwise we'd lose our pull-up resistor. In the else condition we catch if the button is not pressed (as the input register would be holding 0x01, when not pressed), and we write again to the output register 0x08, which will set a 0 to the bit 0 but keep a 1 on bit 3 for the pull-up.

Putting the code all together, we would get the following:

```
int main(){
    // Stop the [watchdog]{.underline} timer
    volatile unsigned int* wdtctl = (unsigned int*) 0x0120;
    *wdtctl = 0x5A80;
    volatile unsigned char* p1in = (unsigned char*) 0x020;
    volatile unsigned char* p1out = (unsigned char*) 0x021;
    volatile unsigned char* p1dir = (unsigned char*) 0x022;
    volatile unsigned char* p1ren = (unsigned char*) 0x027;
    *p1dir = 0x01; // writes 00000001 to p1 direction register
    *p1ren = 0x08; // writes 00001000 to p1 resistor enable register
    *p1out = 0x08; // writes 00001000 to p1 output register
    while(1){
        if((*p1in & 0x08) == 0x00){
            *p1out = 0x09; // 00001001 (LED on, pull-up enabled)
        } else {
            *p1out = 0x08; // 00001000 (LED off, pull-up enabled)
        }
    }
}
```

There are a few additional observations to be made regarding this program. First is the inclusion of the "while(1)" loop. This is an infinite loop and is

found in nearly all embedded systems C programs. While infinite loops are generally considered a negative thing, in embedded systems we want our functionality to loop until the device is no longer powered, which the infinite while loop provides. A "1" is placed in the while's condition, which evaluates as true.

At the top of the code, we have some additional lines included. These are to disable the watchdog timer and unlock the GPIO. On power up, the MSP430 will enable a watchdog timer (which resets your program if not handled). This is a safety feature which we will elaborate on later in the book.

3.4.9. Going Further Beyond

We've covered the basics of GPIO in this chapter, but there are still many more control registers associated with GPIO. However, we don't have all the context and motivation to understand them yet, so we'll introduce them as they become relevant in later chapters. For now, it's important that the fundamentals of reading and writing to the GPIO are well understood, as this content will serve as our foundation for interacting with other peripherals throughout this book.

3.5. Register Macros

Using pointers, we are able to write to any of the register macros that we need to and have total control over the hardware. However, it can be quite tedious to have to create the pointer and dereference it every time we want to write to a peripheral. We can condense this code by creating preprocessor macros to make writing to memory locations easy.

Preprocessor macros are the lines of code that begin with the "#" sign. These lines get replaced by actual C syntax after the preprocessor runs, according to what the macro contains. To make writing to registers easy,

we'll use the #define directive to create dereferenced pointers that have names associated with the register locations they represent.

Let's start with the port 1 direction register:

```
#define P1DIR *((volatile unsigned char*) 0x022)
```

The define preprocessor directive will replace the text that follows the keyword (P1DIR in this case) with the code or value that follows. Now we can just use the text, "P1DIR", anytime we want to write to that register. This macro will replace any instance of "P1DIR" in our code with the declared, cast, and dereferenced pointer, allowing us the easily read and write to that memory location.

For example, we can adapt the code example earlier into the following:

```
#define P1IN *((volatile unsigned char*) 0x020)
#define P1OUT *((volatile unsigned char*) 0x021)
#define P1DIR *((volatile unsigned char*) 0x022)
#define P1REN *((volatile unsigned char*) 0x027)
#define WDTCTL *((volatile unsigned char*) 0x0120)
int main(){
    // Stop the watchdog timer
    WDTCTL = 0x6980;
    P1DIR = 0x08; // writes 00001000 to p1 direction register
    P1REN = 0x01; // writes 00000001 to p1 resistor enable register
    P1OUT = 0x01; // writes 00000001 to p1 output register
    while(1){
        if(P1IN == 0x00){
            P1OUT = 0x09; // 00001001 (LED on, pull-up enabled)
        } else {
            P1OUT = 0x01; // 00001000 (LED off, pull-up enabled)
        }
    }
}
```

Now, all the locations where we once created a pointer to dereference it have been replaced with the defined macro at the top of the code. This not only makes the code more concise in the long run, but also easier to read.

Manufacturers of microcontrollers understand this quite well, and to make their products more attractive to the programmers who might be designing embedded systems that use them, header files are often created that contain macros for all of the different registers and their addresses. Texas Instruments is no different, and they have included a header file that provides just that. If we are using CCS, we can update our code to include this header file and not have to define each of these macros ourselves:

```
#include <msp430.h>
int main(){
    // Stop the watchdog timer
    WDTCTL = 0x6980;
    // Disable the GPIO power-on default high-impedance mode
    PM5CTL0 = 0xFFFE;
    P1DIR = 0x08; // writes 00001000 to p1 direction register
    P1REN = 0x01; // writes 00000001 to p1 resistor enable register
    P1OUT = 0x01; // writes 00000001 to p1 output register
    while(1){
        if(P1IN == 0x00){
            P1OUT = 0x09; // 00001001 (LED on, pull-up enabled)
        } else {
            P1OUT = 0x01; // 00001000 (LED off, pull-up enabled)
        }
    }
}
```

By adding the preprocessor macro "#include <msp430.h>" at the top of our code, CCS will add the appropriate header file that contains the register macros for our target MSP430 microcontroller. Now we don't have

to define the macros ourselves and get straight into writing code to make our applications work.

3.6. Bit Masking

When we are manipulating individual bits on a control register, one of our biggest concerns is being able to read and write to individual bits. As we saw earlier, if I want to observe is my button is pressed or not, I need to read a single bit. However, since memory is divided into 8-bit locations, this is not possible, as I have to read at minimum 8 bits at a time. If we want to isolate a single bit, we'll need to use something called a **bit mask** to single it out. Bit masks are binary values constructed in such a way that when logical ANDed or ORed with another binary value, they will zero out all other bits apart from the one of interest. It "masks off" the bits we aren't interested in, just like you would apply masking tape when painting a wall to prevent painting on undesired surfaces.

3.6.1. Bit Masking for Inputs

Let's consider the example where we want to read the button press on bit 3 of Port 1. To obtain the status of the button, I'm interested in whether bit 3 of the register is a 0 (button is pressed) or a 1 (button is not pressed). However, there is no way to simply read a single bit of an input port register – I can only read the entire register, which is 8 bits long and contains the status of all 8 pins on the input port. So how do I isolate a single bit from a register? The answer is bit-masking, where we will construct a specific binary pattern that can be applied to the register's contents via bit-wise logical operators to perform operations on individual bits.

3.6.2. Understanding the Problem

Before describing the process of bit masking, let's take a moment to understand the problem here. We want to know whether bit 3 is a 1. If we just read all eight bits of the register, couldn't we simply do the following?:

```
if(P1IN == 0x08)
```

Here we are accessing the entire register, P1IN, and checking if the contents equal to having a 1 in the bit 3 position. However, this only works if there is no other high voltages on all the other bits. Let's consider the case where we have a second button on bit 2. If that pin were to also generate a 1, P1IN's contents would contain 0b00001100 or 0x0C in hex. Therefore, if we wanted to check bit 3 and ignore bit 2's inputs, we could not compare the register's contents to 0x08.

3.6.3. Creating a Bit Mask

If I want to observe only bit 3 and check if it is a "1" or a "0", I can create an 8-bit mask with the following value: 0b0000 1000 or 0x08 in hex. I can then perform a bitwise AND operation with the input register and this value:

```
if((P1IN & 0x08) == 0x00)
```

The bitwise AND operation (&) will perform a logical AND with each bit in the two values as seen in Figure 3.16.

Now, if the button is not pressed, the result of the bitwise AND will result in 0b0000 1000 regardless of the contents of the other bits, as the bitwise AND with the mask will have set them to 0. With this mask we can compare the result to 0x00 or 0x01 to check for either state of the button.

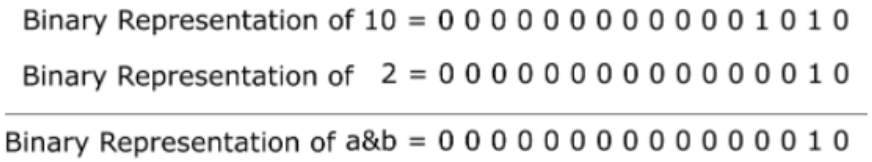


Figure 3.15.: C Bitwise AND operation

3.6.4. Bit Masking for Setting Bits

We are also often interested in manipulating an individual bit as well. We can utilize bit masks for this purpose too. When we want to set an individual bit in a register we can use the bitwise OR operator (|) to achieve this. Figure 3.17 illustrates bitwise OR operator.

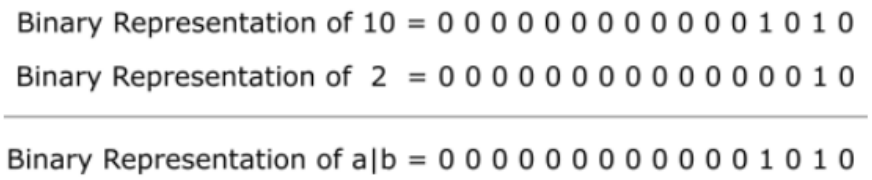


Figure 3.16.: C Bitwise OR operation

Let's say we want to set pin 0 in port 1 as an output, therefore requiring a 1 to be in the fourth bit position. If I've already set other pins as inputs and outputs within the register and do not want to accidentally overwrite those values, I can use a logical OR with the current register contents and a bit mask with a 1 in the bit 0 position.

```
PDIR = PDIR | 0x01; // PDIR | 00000001
```

By using the bitwise OR, only the bits with 1's in the positions of the mask will be set. This works great for setting bits, but the OR operator

can only write 1's, it cannot clear them. Therefore, if we want to clear a bit, we need to create a mask that places 1's in the positions we want to leave undisturbed and 0's in the positions we want to clear. We'll then use the bitwise AND operator to clear bits. For instance, if I wanted to set bit 0 as an input, I would write:

```
PDIR = PDIR & 0xFE; // PDIR & 11111110
```

3.6.5. AND and OR Assignment Operators

Often when bit masking, we are applying a mask to a register then assigning the result of that operation back to the register we just applied it to. Because of the frequency of these operations, the C language provides an operator shortcut just for this. We can reduce the following operation

```
PDIR = PDIR | 0x01;
```

To

```
PDIR |= 0x01;
```

Which are equivalent. The "|=" operator performs a logical OR with the contents on the left side with the right and assigns the result back to the left. Likewise, a similar operator exists for AND:

```
PDIR &= 0xFE;
```

Which performs a logical AND with the contents of PDIR and assigns it back to PDIR.

3.6.6. Creating Bit Masks

We've seen how to create bit masks for isolating a single bit for reading and writing, but we've created these masks by hand and hard coded them as hexadecimal constants within our code. While this is effective, determining which bits are being masked quickly can be challenging, even for experienced programmers. To improve our code's readability, there are several optimizations that you will observe embedded systems programmers utilize. The first is the shift operator.

3.6.6.1. The Shift Operator Method

The shift operator in C allows a programmer to specify a value, then apply a bitwise shift on that value. For instance, let's observe the following operation:

```
int x = 5 << 2;
```

This line of code creates a 5 (0b0000 0101) then applies a left bitwise shift («) by two binary digits. The resulting value stored in x will be 20 (0b0001 0100).

This syntax can be used to create a bit mask easily. We can declare a 1 then shift it to the position of the bit we wish to mask. If I want a mask that sets in P1OUT bit 3 using the shift operator, then I would write:

```
P1OUT |= 1 << 3;
```

This operator creates a 1 (0b0000 0001) then performs a left shift three times, placing the 1 in the bit 3 position: (0000 1000). This operator can be used to target bit 0 as well:

```
P1OUT |= 1 << 0;
```

Which creates a 1 and shifts it to the left 0 times. While this line of code ultimately does nothing, it communicates that we are creating a mask to target bit 0. Most compilers will observe that this shift does nothing and will optimize the operation out of the code, leaving a hardcoded value like we were writing earlier. Creating readable code is almost just as important as creating functional code and being able to identify which bits are being masked at a glance is extremely valuable.

We can also create masks using this method that can be used to clear a bit as well. The shift operator only shifts in 0's, so what we need to do is place the 1 in the position of the bit we wish to clear, then use the bitwise inversion operator (~) to change all the 0's to 1's and vice versa. If I wanted to clear bit 3 on P1OUT, I would write:

```
P1OUT &= ~(1 << 0);
```

By wrapping the mask created with a shift in parenthesis, the shift will happen first, creating the value (0000 1000), then the ~ operator will turn the value into its bitwise inverse (0b1111 0111). This method allows us to create masks for clearing bits that are still easy to read.

3.6.6.2. Bit Mask Macros

We can optimize for readability a step further using macros combined with the shift operator. Macros can be used to create labels for each mask that clearly communicates which bits we are targeting. For example, I could write:

```
#define BIT0 (1 << 0)
#define BIT1 (1 << 1)
#define BIT2 (1 << 2)
#define BIT3 (1 << 3)
```

And in the body of the code use it to set bit 3:

```
P1OUT |= BIT3;
```

Now the programmer doesn't even have to interpret the shift operator to understand which bit is being set. We can also use the ~ operator with these macros as well to create masks for clearing bits:

```
P1OUT &= ~BIT3;
```

These macros are also useful for creating masks that set multiple bits. We can use the | operator to combine multiple masks then apply them to the desired register. If I wanted to set both bit 0 and bit 3, I could write:

```
P1OUT |= BIT3 | BIT0;
```

The | operator would be applied to both masks creating the value, 0000 1001, then applying that mask to P1OUT. Again, this might seem like a wasted operation, given we could simply hardcode the mask and save the microprocessor the trouble of creating the mask at runtime, but any decent compiler will optimize this operation out of the resulting code at compile-time.

3.6.6.3. MSP430 Header File Macros

Due to how common the desire for defined bit mask macros is, nearly every microcontroller manufacturer will provide a header file that includes these macros. In the TI msp430 library, these macros are already defined. Figure 3.18 shows a clip from the msp430 library which includes these macros. We'll notice here that the shift operator was not used, and the macros were hard-coded, but since the macro has been defined, this doesn't affect the readability of the mask in any way. We'll also observe that these macros go up to 16 bits. Despite all the registers we've discussed up to this point are 8-bits wide, some registers are 16-bit. This can be observed at the beginning of the main function in the LED set example for the MSP-EXP430G2ET board, where we disabled the watchdog timer using its 16-bit register (hence, why an int* was used rather than a char*).

```
73 /************************************************************
74 * STANDARD BITS
75 ************************************************************/
76
77 #define BIT0                (0x0001)
78 #define BIT1                (0x0002)
79 #define BIT2                (0x0004)
80 #define BIT3                (0x0008)
81 #define BIT4                (0x0010)
82 #define BIT5                (0x0020)
83 #define BIT6                (0x0040)
84 #define BIT7                (0x0080)
85 #define BIT8                (0x0100)
86 #define BIT9                (0x0200)
87 #define BITA                (0x0400)
88 #define BITB                (0x0800)
89 #define BITC                (0x1000)
90 #define BITD                (0x2000)
91 #define BITE                (0x4000)
92 #define BITF                (0x8000)
```

Figure 3.17.: Bit mask macros from the MSP430 library

3.7. Example Application: LED Button Press

Let's consider an example application. In this example we will use the on-board components of the MSP-EXP430G2ET to create an application that will illuminate the green LED when the right button is pressed. To achieve this, we'll need to use the GPIO to read the state of the button and set the LED's state accordingly. Before writing any code, we first need to know the hardware connections. If I want to read and write digital signals to the button and LED, I need to know which GPIO peripherals and pins they are attached to. To know this, I can either consult the schematic found in the MSP-EXP430G2ET user's guide.

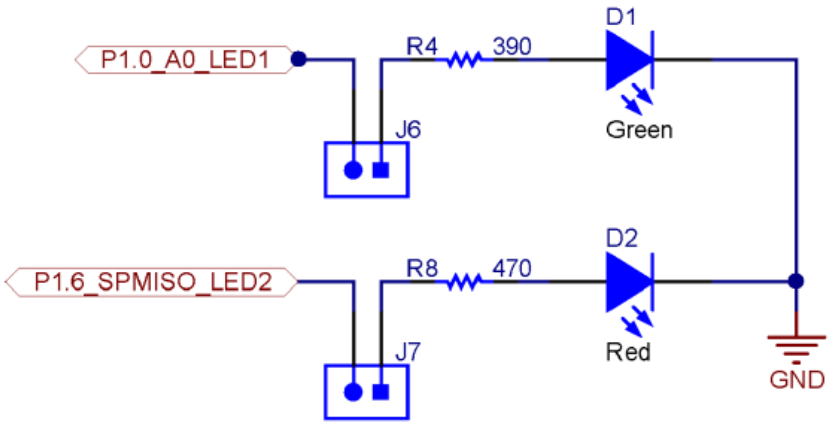


Figure 3.18.: LED connections from the schematic of the MSP-EXP430G2ET

Figure 3.19 depicts the LED connections taken from the development board schematic. We can see here that the green LED is connected to port 1 pin 0 by recognizing the GPIO abbreviation. A current limiting resistor and jumper are also connected to this pin. Figure 3.20 shows clippings of the schematic with the button connections. SW1 is the button found on the

left side of the development board, and we can see that it is connected to port 1 pin 3. Also, from this schematic we can tell that the buttons are connected directly to ground when pressed, which means we'll need an internal pull-up resistor connected in order to read a digital high voltage when the button is not pressed.

Now that the connections are known, we can begin writing the code. We'll use the MSP430 library to access the previously defined register macros and bit definitions.

```
#include <msp430.h>
```

At the start of main we'll disable the watchdog timer, clear the GPIO lock, and set the initial control register connections. We need P1.0 configured as an output, and we need P2.3 configured as an input with an internal pull-up resistor.

```
int main(void)
{
    // Stop the watchdog timer
    WDTCTL = 0x5A80;
    // Button configs
    // Set P1.3 as input (all others as input due to being 0)
    P1DIR &= ~BIT3;
    // Enable P2.3 pull-up resistor
    P1REN |= BIT3;
    // Set P2.3 as high to configure pull-up
    P1OUT |= BIT3;
    // LED configs
    // Set P1.0 as output (all others as input due to being 0)
    P1DIR |= BIT0;
```

Next, we'll add the infinite while loop. Within this while loop, we will use a bit mask to isolate P2.3 and check if the value is set to 0. If P2OUT is

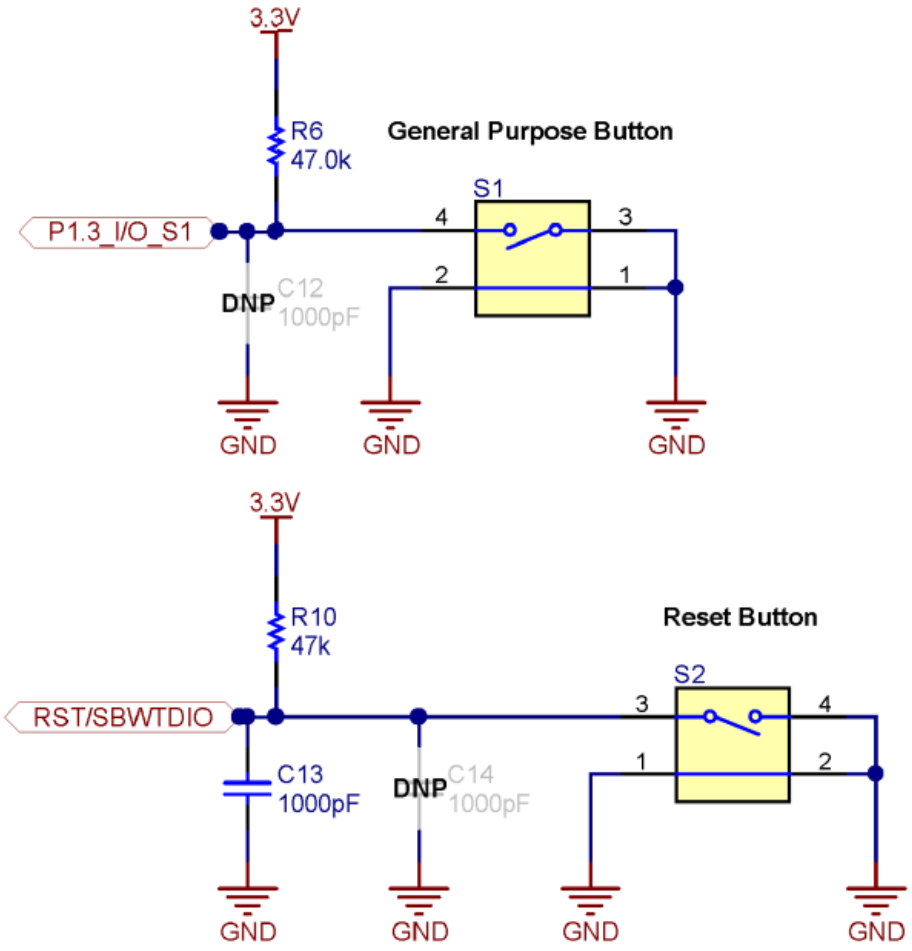


Figure 3.19.: Button connections from the schematic of the MSP-EXP430G2ET

all 0's after the mask, that means bit 3 was set to 0 by pressing the button, which we will then write a 1 to P1.0 to turn the LED on. Otherwise, we read a 0x08, indicating the button is not pressed, and the pull-up resistor is providing a digital high, and we will write a 0 to P1.0 to turn the LED off.

```
while(1)
{
    if((P1IN & BIT3) == 0x00){
        // Button pressed, turn LED on
        P1OUT = P1OUT | 0x01;
    } else {
        // Button not pressed, turn LED off
        P1OUT = P1OUT & 0xFE;
    }
}
}
```

This code will run until power is disconnected to the microcontroller, constantly checking the button and setting the LED accordingly.

3.8. Recap

In this chapter we've learned:

- How peripherals are used to interface a microcontroller with another circuit
- How to control peripherals by writing to control registers
- How to read and write digital signals using GPIO
- Using pointers and macros to access control registers

- Bit masking to isolate individual bits in a register
- How to create readable bit masking code

4. Embedded Programming

4.1. Learning Objectives

In this chapter we will learn:

- Functionalizing Code
- Hardware Abstraction Layers
- Software State Machines
- System Clocks
- Interrupts

4.2. Developing code for embedded Systems

We now have the knowledge to write C code, control the peripheral hardware, and utilize the existing libraries and tools for developing code for embedded systems. However, due to an embedded system's relationship with the system that it is integrated within, we often have concerns that do not manifest when writing code for a desktop computer. These concerns are often related to the **real-time** nature of an embedded system.

4.3. Functionalizing Code

The most basic aspect of writing clean, bug-free C code is proper functionalization. **Functionalizing** code refers to using functions to group related code segments. Since C isn't an object-oriented language, we do not have classes or other complex structures to abstract our code, so we must very carefully organize our code into functions.

Most embedded systems programmers will create initialization routines for their programs. Since the peripherals often need to be configured on start-up, and often only once (such as setting a GPIO pin direction), it makes sense to group everything into an initialization routine. That way at the start of the main function, we aren't bombarded with hundreds of lines of peripheral initialization code. This makes the code much easier to read. Using our example from the end of Chapter 3, let's look at what an initialization function might look like:

```
void init(){
    // Stop the watchdog timer
    WDTCTL = 0x5A80;
    // Button configs
    // Set P1.3 as input (all others as input due to being 0)
    P1DIR &= ~BIT3;
    // Enable P2.3 pull-up resistor
    P1REN |= BIT3;
    // Set P2.3 as high to configure pull-up
    P1OUT |= BIT3;
    // LED configs
    // Set P1.0 as output (all others as input due to being 0)
    P1DIR |= BIT0;
}
```

Another goal of functionalizing your code is to create layers of abstraction. Abstraction will hide the details of the underlying hardware so that your

function names relate to the application. For instance, if we want to change the state of the green LED on the board, we might create a function such as:

```
void setGreenLed(bool enable){
    if(enable){
        P1OUT |= BIT0;
    } else {
        P1OUT &= ~BIT0;
    }
}
```

Where the function's name indicates its purpose, and the details regarding the pin connection of the LED are buried away behind the function. Similarly, we can create a function interface for a push button:

```
bool isButtonPressed(void){
    if((P1IN & BIT3) == 0x00){
        return true;
    } else {
        return false;
    }
}
```

Note that the "bool" datatype is not a native datatype in the C language, and the <stdbool.h> library must be included for this function to compile.

4.3.1. Advanced Considerations: Inline Functions

It can be difficult to remember which devices are connected to a specific port and pin or understand what a bit masking operation is doing at a

glance. Functionalizing code that performs common behaviors, such as setting an LED, greatly increases the code's readability by assigning the function a name that describes its behavior. Placing code into functions also has the benefit of decreasing our overall compiled code size. Since rather than adding additional instructions in the code at every location where we would like to turn an LED on or off, we reuse the functionalized code over and over.

However, there is a drawback associated with this. When a function is called, it creates a "frame" on the stack. A function's stack frame contains the functions' local variables, parameters, and return address. This process takes more time and more memory than if we would have just written the register assignments at the line of code where we wanted to control the LED. This creates a trade-off between slowing down the run time performance or decreasing the code size. This trade-off, however, does not consider the readability of the source code! If I want to functionalize for the sake of readability, but I desire to increase my code size for performance (which would require that I code the register assignments at each line of code where I might change an LED), what could I do?

Fortunately, the C compiler has a solution for this: inline functions. By declaring a function as "inline", when the code is compiled, everywhere the inline function is called, it will be replaced by the instructions that make up that function, preventing a new stack frame from being created and increasing performance at the expense of compiled code size.

For example, I can make the "setGreenLed" function inline by simply adding the "inline" keyword at the beginning of the function definition before the return type:

```
inline void setGreenLed(bool enable){
    if(enable){
        P1OUT |= BIT0;
    } else {
        P1OUT &= ~BIT0;
```

```
    }
}
```

However, it can be important to note that the inline keyword does not get considered by the linker, so sometimes you may run into trouble compiling inline functions. To resolve this, we can include the static keyword as well. Static functions are only visible to the current file (or "translational unit", which includes header files). So for functions that provide a simple low-level translation, such as "setGreenLed" above, we would use a static inline function.

```
Static inline void setGreenLed(bool enable){
    if(enable){
        P1OUT |= BIT0;
    } else {
        P1OUT &= ~BIT0;
    }
}
```

4.4. Hardware Abstraction Layers

Embedded programmers will often take this concept a step further and create what is known as a **hardware abstraction layer** (HAL). A HAL is typically a header file that provides define macros that describe the GPIO connections of a certain component, such as port and pin numbers. For instance, we could create a header file that contains the following define macros for representing the red LED connections:

```
#define LED_PIN 0
#define LED_DIR_REG P1DIR
#define LED_OUT_REG P1OUT
```

Then, we would use the port and pin numbers here in the functions that perform manipulation of the LED:

```
void setLed(bool enable){
    if(enable){
        LED_OUT_REG |= 1 << LED_PIN;
    } else {
        LED_OUT_REG &= ~(1 << LED_PIN);
    }
}
```

What this provides us with is flexibility and modularity. Currently, this code will set the green LED, but let's say some design change came through that required us to use the red LED rather than the green. Now, all I have to change is the port and pin number in the hardware abstraction layer, and all of my code has adapted to that change!

```
#define LED_PIN 6
#define LED_DIR_REG P6DIR
#define LED_OUT_REG P6OUT
```

HALs can also provide modularity when switching between different microcontrollers as well. If some requirement in the project requires that we switch to a different type of MSP430 with more memory, or some peripheral that was available on the MSP43G2553, then all we have to do is change the HAL to the appropriate new hardware connections and our code instantly adapts.

4.5. Header Files and Source Files

Programs in C are largely organized into two types of files: source files (.c) and header files (.h). So far, all of the code we've written has been placed

in a ".c" file (usually "main.c"). However, as projects grow in size, it's not practical to keep all of our code in a single ".c" file.

That's where header files come in. Header files can be included in the build process and allow us to place our code into other files. We've already been using header files, such as "msp430.h" and "stdint.h", in our projects by "#include"ing them at the top of our ".c" file. Header files can store preprocessor directives (#define, #include, etc.) in the header file to declutter our main ".c" file, but they can also be used to reference functions in separate ".c" files.

When we "#include" a header file in our code, the compiler will essentially "copy and paste" the contents of that header file at the top of the file that included it. For example, let's say I have a header file called "leds.h" with the following contents:

```
#define GREEN_LED_PIN 0
#define GREEN_LED_DIR_REG P1DIR
#define GREEN_LED_OUT_REG P1OUT
#define RED_LED_PIN 6
#define RED_LED_DIR_REG P1DIR
#define RED_LED_OUT_REG P1OUT
```

And I have a "main.c" file with the following contents:

```
#include "leds.h"
void setGreenLedOn(){
    GREEN_LED_OUT_REG |= 1 << GREEN_LED_PIN;
}
void setRedLedOn(){
    RED_LED_OUT_REG |= 1 << RED_LED_PIN;
}
void allLedsOff(){
    GREEN_LED_OUT_REG |= ~(1 << GREEN_LED_PIN);
```

```
    RED_LED_OUT_REG |= ~(1 << RED_LED_PIN);
}
void init(){
    // Stop the watchdog timer
    WDTCTL = 0x5A80;
    GREEN_LED_DIR_REG |= 1 << GREEN_LED_PIN;
    RED_LED_DIR_REG |= 1 << RED_LED_PIN;
}
int main(){
    volatile unsigned long int i;
    init();
    while(1){
        setRedLedOn();
        for(i = 0; i < 1000000; i++); // delay
        setGreenLed();
        for(i = 0; i < 1000000; i++); // delay
        allLedsOff();
        for(i = 0; i < 1000000; i++); // delay
    }
}
```

When we compile this “main.c” file, the preprocessor runs first and resolves all lines of code that begin with a “#” sign. First, the “leds.h” header file will be included. After the preprocessor resolves this line, our code would look like the following:

```
#define GREEN_LED_PIN 0
#define GREEN_LED_DIR_REG P1DIR
#define GREEN_LED_OUT_REG P1OUT
#define RED_LED_PIN 6
#define RED_LED_DIR_REG P1DIR
#define RED_LED_OUT_REG P1OUT
void setGreenLedOn(){
```

```
    GREEN_LED_OUT_REG |= 1 << GREEN_LED_PIN;
}
void setRedLedOn(){
    RED_LED_OUT_REG |= 1 << RED_LED_PIN;
}
void allLedsOff(){
    GREEN_LED_OUT_REG |= ~(1 << GREEN_LED_PIN);
    RED_LED_OUT_REG |= ~(1 << RED_LED_PIN);
}
void init(){
    // Stop the watchdog timer
    WDTCTL = 0x5A80;
    GREEN_LED_DIR_REG |= 1 << GREEN_LED_PIN;
    RED_LED_DIR_REG |= 1 << RED_LED_PIN;
}
int main(){
    volatile unsigned long int i;
    init();
    while(1){
        setRedLedOn();
        for(i = 0; i < 1000000; i++); // delay
        setGreenLed();
        for(i = 0; i < 1000000; i++); // delay
        allLedsOff();
        for(i = 0; i < 1000000; i++); // delay
    }
}
```

Notice that the “#include “leds.h” ” has been replaced by the contents of “leds.h”. Next, the preprocessor would resolve all of the #define lines that have been included.

```
void setGreenLedOn(){
    P1OUT |= 1 << 0;
}
void setRedLedOn(){
    P1OUT |= 6;
}
void allLedsOff(){
    P1OUT |= ~(1 << 0);
    P1OUT |= ~(1 << 6);
}
void init(){
    // Stop the watchdog timer
    WDTCTL = 0x5A80;
    P1DIR |= 1 << 0;
    P1DIR |= 1 << 1;
}
int main(){
    volatile unsigned long int i;
    init();
    while(1){
        setRedLedOn();
        for(i = 0; i < 1000000; i++); // delay
        setGreenLed();
        for(i = 0; i < 1000000; i++); // delay
        allLedsOff();
        for(i = 0; i < 1000000; i++); // delay
    }
}
```

We can see that all references to “GREEN_LED_PIN”, “GREEN_LED_OUT_REG”, etc., have been replaced by the value that they were macro’d to (ex., all instance of “RED_LED_PIN” have been replaced by “6”, since the line was originally “#define RED_LED_PIN 6”). Now, this still isn’t a

complete example, as "P1OUT" still needs to be replaced by the macros found in <msp430.h>.

4.6. Software State Machines

Another common method of abstraction in embedded systems is the usage of state machines. A state machine is a program or machine that organizes its functionality into different "states". The machine will then transition between the states as it receives inputs from sensors or users. Almost any problem can be organized into a state machine, which helps abstract complex problems by dividing them up into finite states which are easier to define.

Let's consider a simple example application where a microcontroller turns an LED on or off based on a button press. If we wanted to organize this application into a state machine, we would want to consider the behaviors of this application and how we can organize them into states, which are the system's behaviors. In this simple application, we can easily identify two behaviors: turning the LED on and turning the LED off. We would organize these two behaviors into separate states, then we would need to describe how the system moves between states. The button press is to turn the LED on or off, so we would draw arrows indicating the transitions between states with a label indicating what caused that transition. This example state machine can be seen in the diagram of Figure 4.1.

4.6.1. Anatomy of a State Machine

There are several important components that need to be present in every state diagram, such as the one in Figure 4.1:

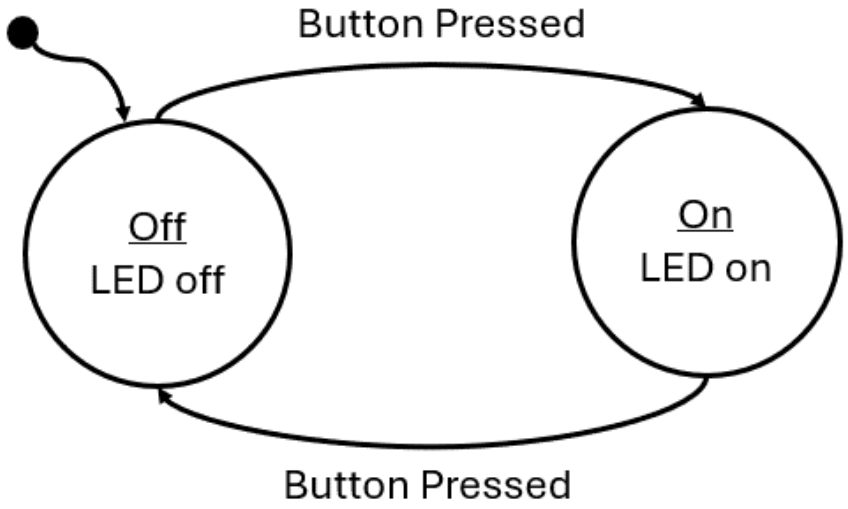


Figure 4.1.: State Machine Diagram for turning an LED on and off

- States – A circle or box containing the state label and the behavior (outputs and actions) the state. Sometimes if the behavior of the state is too complex, it will be omitted and described elsewhere.
- Transitions – An arrow indicating when to transition from one state to another. These arrows will also include the condition the prompts the transition.
- Initial State – A dot with an arrow is drawn on the diagram to indicate which state is the initial state.

4.6.2. Implementing a Software State Machine

Once diagrammed, software state machines are easy to implement. Ultimately, the behavior of each state simply needs to be grouped together with some branching conditions to separate them based on a variable which represents the current state. In C, this can be done with a series of "if-else" statements, but more commonly the switch statement is used. Switch statements are easier to read for many different states. For example, a state machine can simply be represented with the following:

```
while(1){
    int state = 0;
    switch(state){
        case 0:
        // some behavior for state 0
        if(/* transition condition */){
            state = 1;
        }
        break;
        case 1:
        // some behavior for state 1
        if(/* transition condition */){
            state = 2;
```

```
            else if(/* transition condition*/) {
                state = 0;
            }
            break;
            case 2:
            // some behavior for state 1
            if(/* transition condition */){
                state = 2;
                else if(/* transition condition*/) {
                    state = 0;
                }
                break;
                default:
                // catch potential errors here
                break;
            }
        }
```

Where the variable, state, contains a numeric value that represents the current state. Each iteration of the while loop will run the current state, whose behavior is selected by the switch condition. At the end of each state, some condition will be tested to determine if the state will transition to a new state or run the same state again.

In this template code example, the behavior of each state is represented with just a comment, as the state's behavior is dependent on whatever the application is supposed to do. These behaviors can be quite long, so in order to keep the state machine readable, it is common to functionalize the state's behavior. When functionalized, the behavior is neatly organized and the next state the machine should transition to can be returned by that function:

```
while(1){
    int state = 0;
    switch(state){
        case 0:
        state = runState0(/* pass state inputs */);
        break;
        case 1:
        state = runState1(/* pass state inputs */);
        break;
        case 2:
        state = runState2(/* pass state inputs */);
        break;
        default:
        // catch potential errors here
        break;
    }
}
```

With the functionalized structure, no matter how complex the state's behaviors become, this template will not change as all the complexity is hidden behind the functions.

4.6.3. Example: Traffic Light

As a simple example, we'll implement a traffic light using the MSP430G2553 using a state machine to model the behavior. To begin, we'll present the circuit diagram for the traffic light. We'll use three LEDs to represent each color on the traffic light! How each of these LEDs are interfaced to GPIO pins of the microcontroller is shown in Figure 4.2.

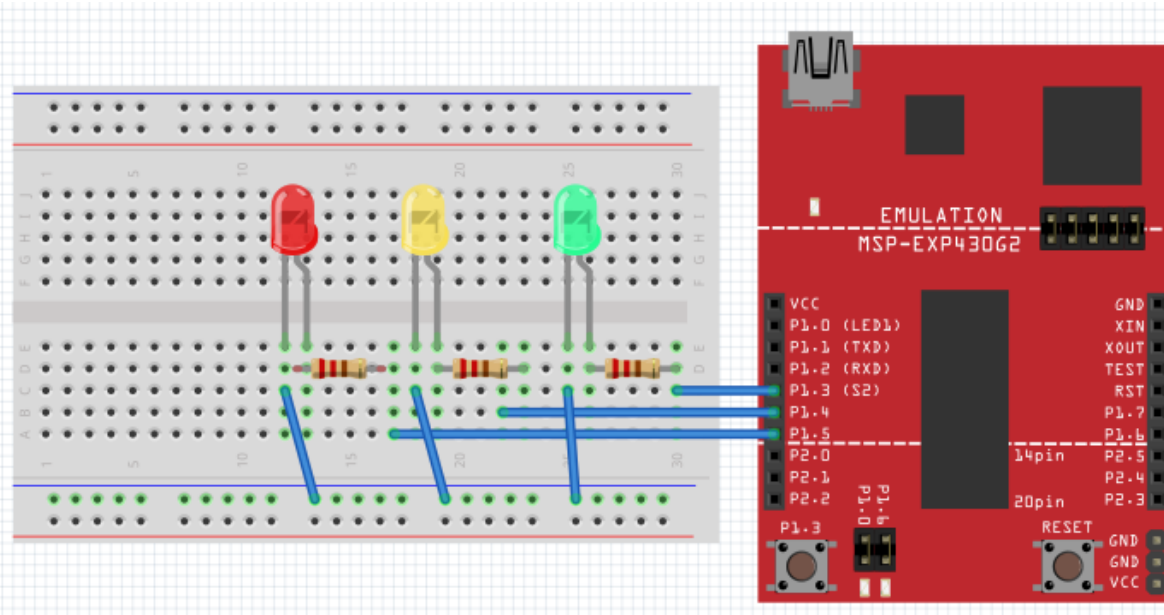


Figure 4.2.: A circuit board with wires and wires Description automatically generated

Figure 4.2: Traffic light schematic diagram

In Figure 4.2, we can observe that we have connected the green, yellow, and red LEDs to P1.3, P1.4, and P1.5, respectively. To create the traffic light behavior, we will organize the behavior into induvial states that control the output (the LED that should be illuminated) based on inputs (the amount of time that should each color should be illuminated for) and transition conditions (when the lights should change colors).

4.6.3.1. Creating Interface Functions

We will create functions for initializing and manipulating the LEDs, which will be set by the current state:

```
static inline void setGreenLed(bool enable){
    if(enable){
        P1OUT |= BIT3;
    } else {
        P1OUT &= ~BIT3;
    }
}
static inline void setYellowLed(bool enable){
    if(enable){
        P1OUT |= BIT4;
    } else {
        P1OUT &= ~BIT4;
    }
}
static inline void setRedLed(bool enable){
    if(enable){
        P1OUT |= BIT5;
    } else {
        P1OUT &= ~BIT5;
    }
}
void init(){
    // Stop the watchdog timer
    WDTCTL = WDTPW | WDTHOLD;
    // LED configs
    P1DIR |= BIT3 | BIT4 | BIT5;
}
```

4.6.3.2. Capturing Behavior with a State Diagram

Before creating a state machine, we first need to model its behavior using a state diagram. Now that we have the functions needed for manipulating our traffic light, we need to create a state diagram to describe the behavior of the traffic light. This diagram can be seen in Figure 4.3.

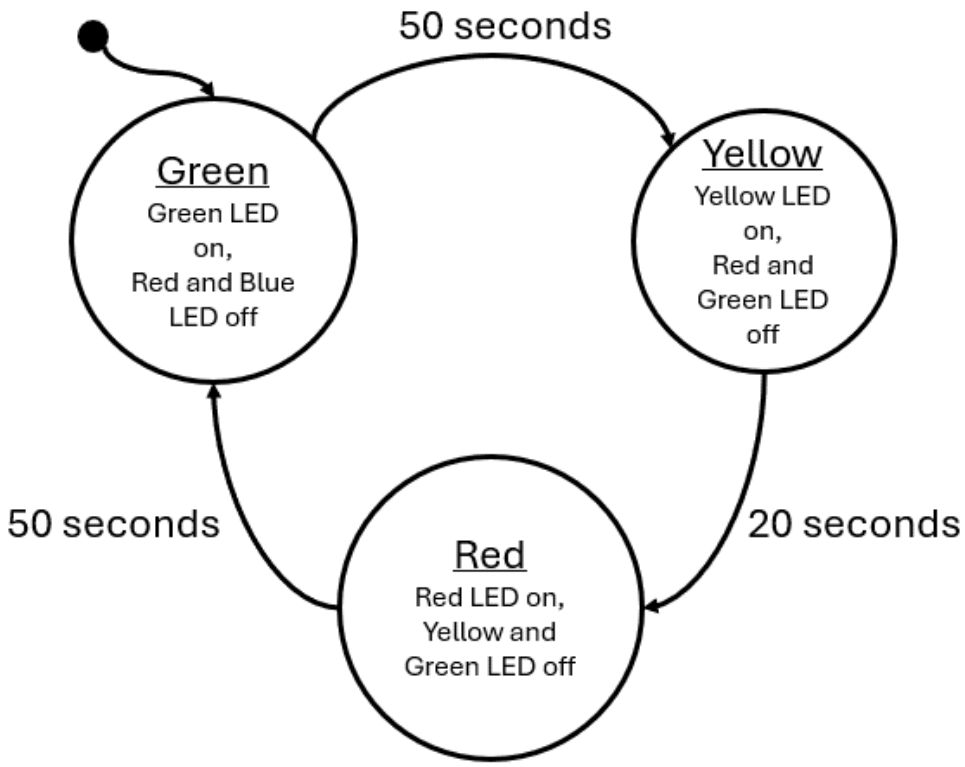


Figure 4.3.: A diagram of a light source Description automatically generated

Figure 4.3: Traffic light state diagram

The outputs of this state are simply the illuminated LED and the input to the state machine is time in seconds. Every 50 seconds, the green state will transition to the yellow state, after 30 seconds the yellow will transition to red, and after 50 seconds red will transition back to green. We can see that the input to this system is real-world seconds, we need some way to create a function that will give us a delay in seconds.

4.6.3.3. Building Delay Functions

To build a delay function to stall the processor, we can use the MSP430's built-in ___delay_cycles() function. This is a function that will insert "no-ops" (no operation) instructions into the processor. As its name implies, a no-op does nothing but wastes time as it moves through the processor – perfect for creating delays. What we must understand now is: how much time is wasted with a single no-op? The MSP430 completes one instruction each clock cycle. The clock is an oscillating square-wave signal which provides synchronization for digital circuits. If one instruction is completed each clock cycle, then each no-op will take the same amount of time as the period of the clock signal. In the MSP430G2553 documentation, we can learn that by default, the processor clock speed is 1 MHz, so 1,000,000 cycles per second. Therefore, each instruction takes 0.000001 seconds to execute.

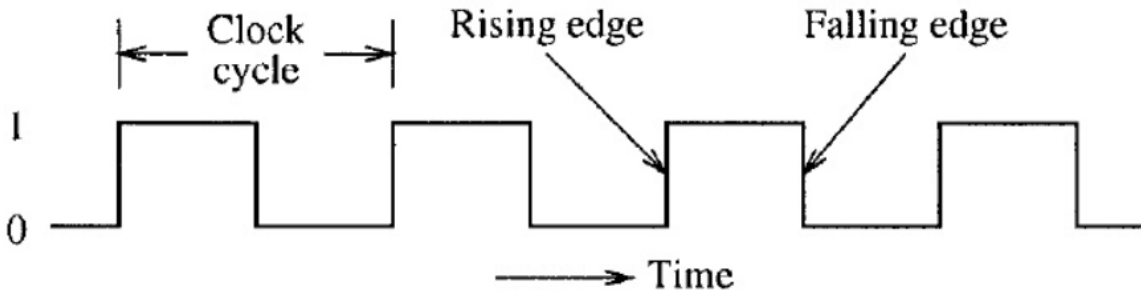


Figure 4.4.: A picture containing text, font, diagram, line Description automatically generated

Figure 4.4: Clock period

Using this information, we can create a delay function that consumes as many seconds as we desire, by passing 1,000,000 to the ___delay_cycles() function.

```
void delay_s(uint16_t seconds){
    volatile uint16_t i;
    for(i = 0; i < seconds; i++){
        __delay_cycles(1000000);
    }
}
```

We can use this to create a delay that will increment our state machine's input value in seconds.

4.6.3.4. Creating State Functions

We now have everything we need to implement the traffic light diagram in 4.3: functions to set the LEDs and a function to create a delay in seconds.

All we have to do next is create the state machine itself. We will begin by creating the functions that will represent our states:

```
uint16_t runRedState(uint16_t* seconds){
    uint16_t nextState = 2;
    // set output
    setRedLed(true);
    setGreenLed(false);
    setYellowLed(false);
    // next state logic
    if(*seconds > 50){
        *seconds = 0;
        nextState = 0;
    }
    return nextState;
}
uint16_t runYellowState(uint16_t* seconds){
    uint16_t nextState = 1;
    // set output
    setRedLed(false);
    setGreenLed(false);
    setYellowLed(true);
    // next state logic
    if(*seconds > 20){
        *seconds = 0;
        nextState = 2;
    }
    return nextState;
}
uint16_t runGreenState(uint16_t* seconds){
    uint16_t nextState = 0;
    // set output
    setGreenLed(true);
    setRedLed(false);
```

```
    setYellowLed(false);
    // next state logic
    if(*seconds > 50){
        *seconds = 0;
        nextState = 1;
    }
    return nextState;
}
```

Note that these functions are constructed in the same functionalized format presented in the previous section: each function contains the behavior of the state, each function is passed the state inputs, and returns what the next state should be on the following iteration based on the current inputs.

4.6.3.5. Implementing the State Machine

Next, we will construct the state machine within our main function and while(1) loop:

```
int main(void) {
    uint16_t seconds = 0;
    uint16_t currentState = 0;
    uint16_t nextState = currentState;
    init();
    while(1) {
        switch(currentState){
            case 0: // Green State
            nextState = runGreenState(&seconds);
            break;
            case 1: // Yellow State
            nextState = runYellowState(&seconds);
            break;
```

```
            case 2: // Red State
            nextState = runRedState(&seconds);
            break;
            default:
            break;
        }
        // Update state
        currentState = nextState;
        // Update seconds
        delay_s(1);
        seconds++;
    }
}
```

Note that we use two variables for tracking the state each iteration: currentState and nextState. Each loop, currentState is what the switch statement selects, but is updated with nextState by the end of the loop. The nextState variable is updated by the return from each state function call. Our inputs to each state are tracked by a seconds variable that is incremented after a 1 second delay each iteration.

4.6.3.6. Enumerating States

While the implementation we just described is a complete solution, there are several improvements that can be mode. The first aspect that can be improved is the usage of an integer to represent the states. Rather than using an integer, we can create an enumeration to represent the states:

```
enum{
    GREEN, YELLOW, RED
} state;
```

The enum keyword will enumerate the values encapsulated by the following curly braces. GREEN will take on the value of 0, Yellow will take on 1, and RED will take on 2; The same integer values we had used prior. However, with the usage of the enum will allow us to use these terms rather than the integer values of the states. We can also use enum as a custom datatype by including the keyword "typedef".

```
typedef enum{
    GREEN, YELLOW, RED
} state_t;
```

The rest of the state machine can be updated as such:

```
int main(void) {
    volatile uint16_t seconds = 0;
    state_t currentState = 0;
    state_t nextState = currentState;
    init();
    while(1) {
        switch(currentState){
            case GREEN:
            nextState = runGreenState(&seconds);
            break;
            case YELLOW:
            nextState = runYellowState(&seconds);
            break;
            case RED:
            nextState = runRedState(&seconds);
            break;
            default:
            break;
        }
        // Update state
```

```
        currentState = nextState;
        // Update seconds
        delay_s(1);
        seconds++;
    }
}
```

In the same way, the state functions can also be updated:

```
state_t runGreenState(uint16_t seconds){
    state_t nextState = GREEN;
    // set output
    setGreenLed(true);
    setRedLed(false);
    setYellowLed(false);
    // next state logic
    if(*seconds > 50){
        *seconds = 0;
        nextState = YELLOW;
    }
    return nextState;
}
```

With this improvement, the code becomes much easier to read and write, as the programmer no longer needs to remember which integer value is associated with each state. By using the enum as a custom datatype, it becomes more clear when we are evaluating or returning a state rather than an integer value.

4.6.3.7. Function Tables

Another improvement that can be made is utilization of a function table. A function table is an array of pointers to functions. A function table has

the following structure:

```
(int) *(func_table[3])(int, int) = {func0, func1, func2};
```

Where func0, func1, and func2 have the following prototypes:

```
int func0(int a, int b);
int func1(int a, int b);
int func2(int a, int b);
```

We can observe that each of these functions have the same parameters and return types, which is required if we are to point to each of these in a function table. Function tables can only hold with identical parameters and return types. We can see the function table is the array named "func_table" which is declared with three indices. The table is dereferenced with the "*" keyword, preceded by the return type in parenthesis, and followed by the parameter types in parenthesis. In this example, the table is initialized by declaring the function names within curly braces.

To use a function table, we simply call it like any other function with the addition of the index contained within brackets:

```
int x = func_table[1](5, 10);
```

In this example, we index element 1 of the function table, therefore calling func1 and passing it 5 and 10 as integer parameters. The integer result of the function is returned and stored in x.

We can create a function table to store our state functions:

```
state_t (*state_table[3])(uint16_t*) = {runGreenState, runYellowState, runRe
```

Then, inside the main loop, we can simply use our "currentState" variable to index the correct state function:

```
while(1) {
    nextState = state_table[currentState](&seconds);
    // Update state
    currentState = nextState;
    // Update seconds
    delay_s(1);
    seconds++;
}
```

Since the state enum values correlate to integers, as long as the sequence of states listed in the enum matches the sequence they are listed in the state table, the correct state will be called from the state table based on the currentState enum value as the index.

However, using a state table can introduce some new hazards that must be accounted for, such as the potential to call a function from an index not in the table (e.g., calling index 4 from an array/function table with only 3 elements). This needs to be checked for in the while loop to ensure that the index is within range of the state table. This is usually managed by adding an additional enum to the state enumeration which indicates the maximum index:

```
typedef enum{
    GREEN, YELLOW, RED, MAX_STATES
} state_t;
```

The MAX_STATES value will take on the number of states that precede it. In this case, MAX_STATES will be assigned the value of 3, which is the number of states in our state machine, and therefore the number of indices in the state table. We can update the rest of the code to utilize this value:

```
state_t (*state_table[MAX_STATES])(uint16_t*) = {runGreenState, runYellowStat
while(1) {
    if(currentState < MAX_STATES)
    nextState = state_table[currentState](&seconds);
    // Update state
    currentState = nextState;
    // Update seconds
    delay_s(1);
    seconds++;
}
```

Now if we add additional states, only the enum and the state table need to be updated, as the length of the array and the if condition guarding the function table call are all controlled by the MAX_STATES enum value.

4.7. Clock Sources and Signals

When we created the delay function earlier in this chapter, we created a delay using knowledge of the clock frequency. It was stated that from the documentation we knew that the default clock speed was 1 MHz. To be able to derive this information from the documentation, we need to understand how clocking works in the MSP430G2553, but also how it works in general for most microcontrollers.

Microcontrollers very commonly have multiple clocks. This is because the microcontroller might have different timing requirements for various aspects of the systems in which they are integrated. For instance, the system clock may need to run at a fast speed for processing requirements, while the analog to digital converter peripheral might need to sample a signal at a slower rate. While it's possible to have one clock be able to control all of these aspects, often many clocks are made available for better flexibility and precision.

The MSP430G2553 has multiple clock sources. A clock source is where the oscillation that drives the clock signals originates. These sources are given names associated with what generates them:

- DCOCLK – internal digitally controlled oscillator
- VLOCLK – Very low power, low-frequency oscillator with a ~12kHz operating frequency
- LFXT1CLK – Low-frequency / high-frequency oscillator that can be used with low-frequency watch crystals or external clock sources of 32768 Hz or with standard resonators in the 400-kHz to 16-MHz range

Each of these clock sources have different ranges and are designed for specific purposes. These clock sources can then be configured to be connected to various clock signals. Clock signals refer to the internal circuitry that carries and connects the clock sources to different peripherals and components.

- MCLK – Main clock, which drives the processor
- SMCLK – Subsystem main clock, which can be used to drive peripherals
- ACLK – Auxiliary clock, which also is used to drive peripherals

Clock signals must be configured to be driven by a clock source, as shown in Figure 4.5. Once a clock signal has been assigned a source, it can then be configured to drive the processor or some other peripheral.

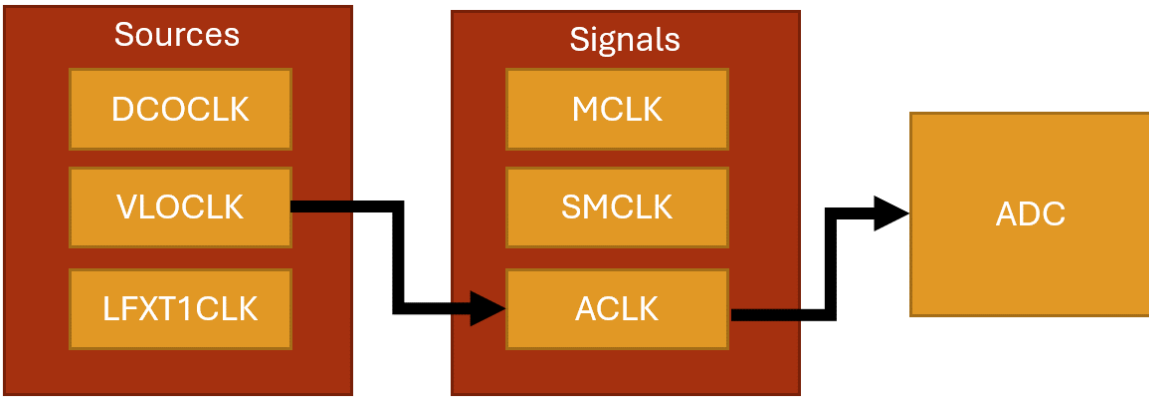


Figure 4.5.: A diagram of a signal Description automatically generated

Figure 4.5: Clock sources and signals

The clocking system is configured through the use of control registers, just as peripherals are. Except this time, we will observe that the control registers bits don't correspond to enabling features on pins, but rather individual bits and bit ranges will be responsible for enabling, disabling, or selecting different features and settings for the system.

4.7.1. Example: Using the VLO Clock Source

When writing embedded applications that has strict timing requirements, which is in general most applications, we will often utilize the clocking system to configure the system and peripheral clocks ourselves, so we know exactly which clock rates are being specified. Using the MSP-EXP430G2ET, we'll create an initialization function for the clock to configure the main clock signal to utilize a specific clock source.

In this example, we will change the main clock signal from its default 1 MHz supplied by the DCO clock source to a 12 kHz VLO clock source.

To change the clock source, we must configure a few registers: BCSCTL2, BCSCTL3, and IFG1.

BSCTL3

The BCSCTL3 (Basic Clock Module Control Register 3) configures frequency range, low frequency clock select, capacitor values, and also contains fault flags. According to Figure X, we will need to write 0b10 to bits 5-4 of this register to select VLOCLOCK as the low frequency oscillator (LFXT1CLK) clock source. This clock source can be configured to be driven by internal clock sources such as the VLO, but it can also be configured to utilize an external clock source, such as a quartz crystal oscillator, provided one was attached to specific pins on the microcontroller. External oscillators also depend on external capacitors, which is why this same register has options for configuring them. This register also specifies the frequency range of an external oscillator, but since we aren't using one, we can ignore this configuration as well.

Figure 5-13. BCSCTL3 Register

7	6	5	4	3	2	1	0
XT2Sx		LFXT1Sx		XCAPx		XT2OF	LFXT1OF
rw-0	rw-0	rw-0	rw-0	rw-0	rw-1	r-0	r-(1)

Table 5-5. BCSCTL3 Register Field Descriptions

Bit	Field	Type	Reset	Description
7-6	XT2Sx	R/W	0h	XT2 range select. These bits select the frequency range for XT2. 00b = 0.4- to 1-MHz crystal or resonator 01b = 1- to 3-MHz crystal or resonator 10b = 3- to 16-MHz crystal or resonator 11b = Digital external 0.4- to 16-MHz clock source
5-4	LFXT1Sx(1)	R/W	0h	Low-frequency clock select and LFXT1 range select. These bits select between LFXT1 and VLO when XTS = 0, and select the frequency range for LFXT1 when XTS = 1. MSP430G22x0: The LFXT1Sx bits should be programmed to 10b during the initialization and start-up code to select VLOCLK (for more details refer to Digital I/O chapter). The other bits are reserved and should not be altered. When XTS = 0: 00b = 32768-Hz crystal on LFXT1 01b = Reserved 10b = VLOCLK (Reserved in MSP430F21x1 devices) 11b = Digital external clock source When XTS = 1 (Not applicable for MSP430F20xx, MSP430G2xx1, MSP430G2xx2, MSP430G2xx3): 00b = 0.4- to 1-MHz crystal or resonator 01b = 1- to 3-MHz crystal or resonator 10b = 3- to 16-MHz crystal or resonator 11b = Digital external 0.4- to 16-MHz clock source LFXT1Sx definition for MSP430AFE2xx devices: 00b = Reserved 01b = Reserved 10b = VLOCLK 11b = Reserved

Figure 4.6.: A screenshot of a computer Description automatically generated

3-2	XCAPx[2]	R/W	1h	Oscillator capacitor selection. These bits select the effective capacitance seen by the LFXT1 crystal when XTS = 0. If XTS = 1 or if LFXT1Sx = 11 XCAPx should be 00. This bit is reserved in the MSP430AFE2xx devices. 00b = Approximately 1 pF 01b = Approximately 6 pF 10b = Approximately 10 pF 11b = Approximately 12.5 pF
1	XT2OF[3]	R	0h	XT2 oscillator fault. Does not apply to MSP430x2xx, MSP430x21xx, or MSP430x22xx devices. 0b = No fault condition present 1b = Fault condition present
0	LFXT1OF[2]	R	1h	LFXT1 oscillator fault. This bit is reserved in the MSP430AFE2xx devices. 0b = No fault condition present 1b = Fault condition present

Figure 4.7.: A white sheet with black text Description automatically generated with medium confidence

Table 4.1: The BCSCTL3 Register

IFG1

When we configure a new clock source in BCSCTL3, a fault will be generated, and we won't be able to use this clock until we clear it. The IFG1 register has a single bit at position 1 that will flag oscillator faults. We simply write a 0 to the bit 1 position in this register to clear it before doing any other configurations.

Table 5-7. IFG1 Register Field Descriptions

Bit	Field	Type	Reset	Description
7-2				These bits may be used by other modules. See device-specific da sheet.
1	OFIFG(1)	R/W	1h	Oscillator fault interrupt flag. Because other bits in IFG1 may be used for other modules, it is recommended to set or clear this bit using BIS.B or BIC.B instructions, rather than MOV.B or CLR.B instructions. 0b = Interrupt not pending 1b = Interrupt pending

Figure 4.8.: A screenshot of a computer Description automatically generated

0				This bit may be used by other modules. See device-specific data sheet.

Table 4.2: Select Bits of the IFG1 Register

BCSCTL2

The BCSCTL2 (Basic Clock Module Control Register 2) configures which clock sources drive which clock signals. The clock that drives our microcontroller's processor is the MCLK. We want to write 0b11 to bits 7-6 in this register to set the MCLK to be sourced by the VLOCLK, as seen in Table 4.3. The rest of the bits in this register configure other clock signal sources and select clock dividers, which can be used to slow down a clock signal. Since we are not configuring any other clocks or dividers, we'll ignore the rest and leave them at their default values.

Figure 5-12. BCSCTL2 Register

7	6	5	4	3	2	1	0
SELMx		DIVMx		SELS	DIVSx		DCOR
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

Table 5-4. BCSCTL2 Register Field Descriptions

Bit	Field	Type	Reset	Description
7-6	SELMx	R/W	0h	Select MCLK. These bits select the MCLK source. 00b = DCOCLK 01b = DCOCLK 10b = XT2CLK when XT2 oscillator present on-chip. LFXT1CLK or VLOCLK when XT2 oscillator not present on-chip. 11b = LFXT1CLK or VLOCLK
5-4	DIVMx	R/W	0h	Divider for MCLK 00b = /1 01b = /2 10b = /4 11b = /8
3	SELS	R/W	0h	Select SMCLK. This bit selects the SMCLK source. 0b = DCOCLK 1b = XT2CLK when XT2 oscillator present. LFXT1CLK or VLOCLK when XT2 oscillator not present
2-1	DIVSx	R/W	0h	Divider for SMCLK 00b = /1 01b = /2 10b = /4 11b = /8
0	DCOR(1) (2)	R/W	0h	DCO resistor select. Not available in all devices. See the device-specific data sheet. 0b = Internal resistor 1b = External resistor

Figure 4.9.: A screenshot of a computer Description automatically generated

Table 4.3: Select Bits of the BCSCTL2 Register

Writing the Code

In many embedded projects, it is common practice to include a clock initialization routine that configures all the clock signals and sources initially, so for timing purposes we have full control and knowledge of the system's clock speeds. Here we'll write a function that initializes the main clock (MCLK) signal using the VLO oscillator as the clock source using the configuration just described in the sections above.

```
void initClocks(){
    BCSCTL3 |= 0b00100000; // Set VLOCLK to LFXT1
    IFG1 &= ~0b00000010; // Clear OSCFault flag
    BCSCTL2 |= 0b11000000; // Set MCLK to VLOCLK or LFXT1CLK
}
```

This provides the configurations needed, however, it is largely unreadable code, unless for some reason you have memorized all the bits in each register and what options they control. We can help alleviate this by utilizing bit macros provided in the <msp430.h> library. We have previously seen that this library provides bit macros for various bit masks, such as BIT1 or BIT6, but it also will provide specific bit masks for setting configurations in specific registers. For example, this library provides the following macro:

```
**#define** LFXT1S_2 (0x20) /* Mode 2 for LFXT1 : VLO */
```

Which sets 0b11 in the 5-4 bit positions for selecting the VLO clock as the LFXT1 source. Ultimately, our function could be re-written using these macros and appear as follows:

```
void initClocks(){
    BCSCTL3 |= LFXT1S_2; // LFXT1 = VLO
    IFG1 &= ~OFIFG; // Clear OSCFault flag
    BCSCTL2 |= SELM_3 | DIVM_0; // MCLK = LFXT1/1
}
```

It may be debatable whether this is truly more readable than the previous function, but most embedded systems programmers would agree that it is. There is one aspect of this code that is particularly interesting, which is the use of logical or with the DIVM_0 macro. The DIVM_0 macro appears as follows in the <msp430.h> header file:

```
**#define** DIVM_0 (0x00) /* MCLK Divider 0: /1 */
```

We see that it is macro'd to 0 to select a clock division of 1, which is correct because we wanted to use the 12 kHz with no division in this example. Therefore, this bit mask does *absolutely nothing* to bits assigned to this register! So why would we bother including this? It may not do anything to the data assigned to the register, but it does *communicate* to other programmers that it was intended to use a clock divider of 1 in this initialization routine. Seeing as this functionally does nothing, the compiler will optimize this operation away, so including it doesn't decrease our program's execution time anyways.

4.8. Interrupts

Now that the problem of creating a delay for a specific period of time has been solved, a new problem has been introduced. While we are in the delay function, we can no longer be reactive to any external events, such as button press es. If the user presses the button and releases it while we are in the middle of a long delay function, we will have no way to detect that button press! Figure 4.7 illustrates this condition with a diagram.

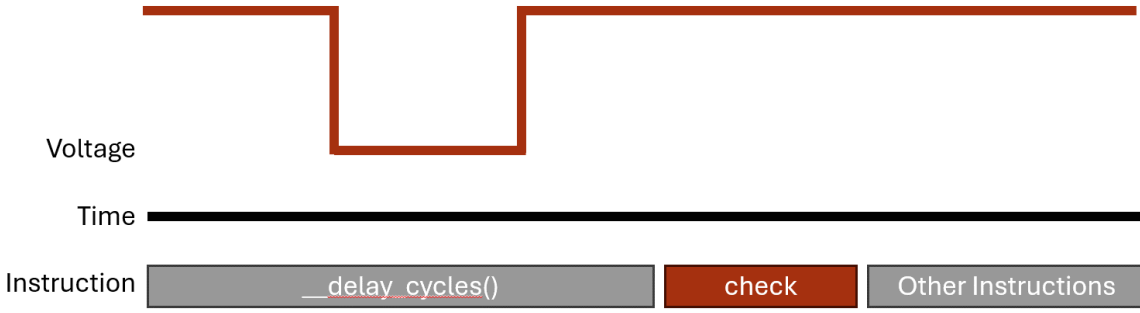


Figure 4.10.: A computer screen shot of a red rectangle Description automatically generated

Figure 4.7: Missing a button press while in a delay function

We need some way to instantly react to events that occur at real time in an embedded system. These events can come from user interaction through buttons or switches, but they can also come from external sensors or even conditions in the program. Whatever the source of the event, we often cannot predict when these events will occur, so we need the ability to pause whatever the program is doing and handle the event. The way we achieve this in almost every processor architecture is through “interrupts”.

Interrupts are triggered by random, external events and they allow us to pause the computer’s program execution and temporarily run some other code to handle the event. The code we call to handle the event is known as the **interrupt handler** or also the **interrupt service routine (ISR)**. Once the ISR has concluded, the program will pick back up where it left off before the interrupt occurred.

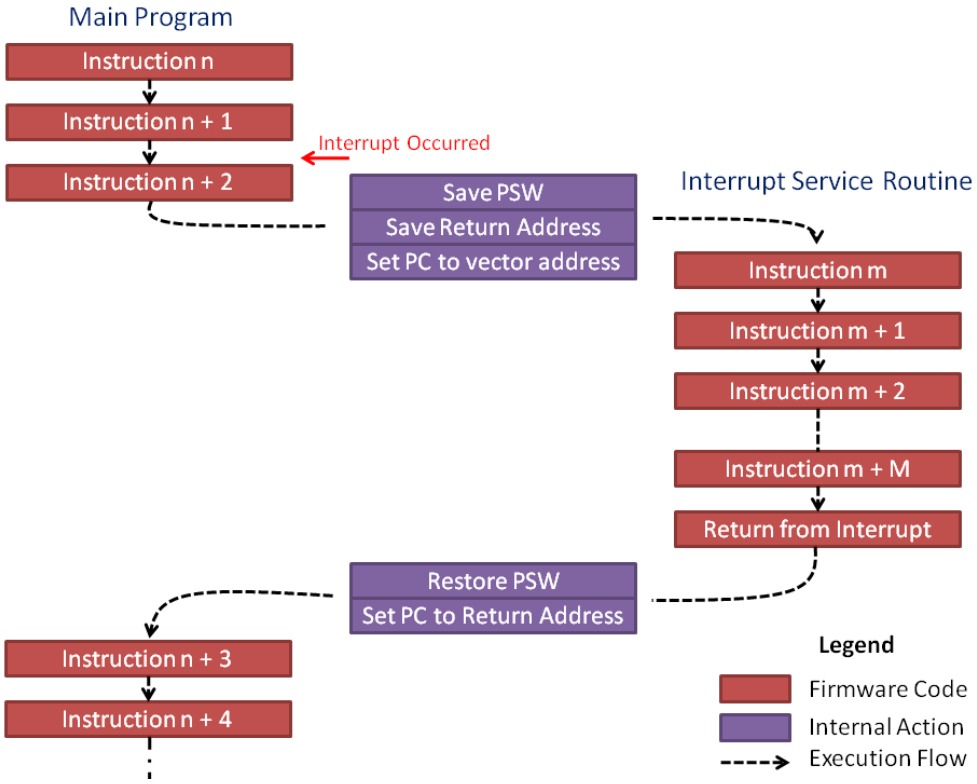


Figure 4.11.: Diagram Description automatically generated

Figure 4.8: Pausing the code to handle the interrupt

How Interrupts Work

An interrupt is triggered by an event that the hardware acknowledges. The hardware will save the current state of the processor (what line of code it was executing at the time and various other conditions and properties) then jump to a memory location that will lead to a user-defined function.

There are two ways interrupts are implemented: vectored and non-vectored. We'll start by discussing the vectored implementation, as it is the method used in our MSP430 microcontroller. In vectored interrupts, a section of memory is blocked off to serve as the interrupt vector table. The table has a start address, and from that address are indexed memory locations associated with each interrupt source. Each of these indexed memory locations can be filled with the address of the function that will be called when the interrupt event occurs (the interrupt service routine!).

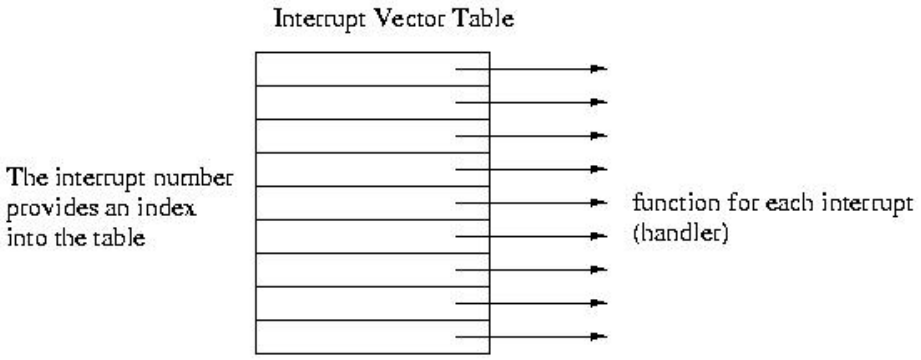


Figure 4.12.: A picture containing text, receipt, screenshot, font Description automatically generated

Figure 4.9: Interrupt Vector Table

For instance, say we have an interrupt that triggers when a button is pressed (the voltage on one of the GPIO changes from a 1 to a 0). This hardware event may be associated with, say, index 45 of the interrupt vector table. The processor will pause, retrieve the address of the ISR at memory location *<vector table base address> + 45*, then start executing the code at that function's memory location.

Non-vectored interrupts function in a similar way but have less hardware support. When an interrupt trigger occurs, the processor will pause and

then go to a single interrupt ISR which will then determine in the software what type of event occurred and what function to call. This eliminates the need for a dedicated vector table in the microcontroller's memory map, but it also is a little slower to access the ISR because it has to run additional lines of code. In vectored interrupts, the interrupt condition has a known memory location where it's ISR address is stored, and it will jump to that location without any prior computations.

4.8.1. Example: Traffic Light Crosswalk Button

Let's say we wish to add a crosswalk button to our traffic light state machine. The crosswalk button should add an additional state to the application which sets the red light and a blue light which represents a "walk sign" that gives time for pedestrians to cross the street. This blue LED is connected to P2.0 as seen in the schematic in Figure 4.10.

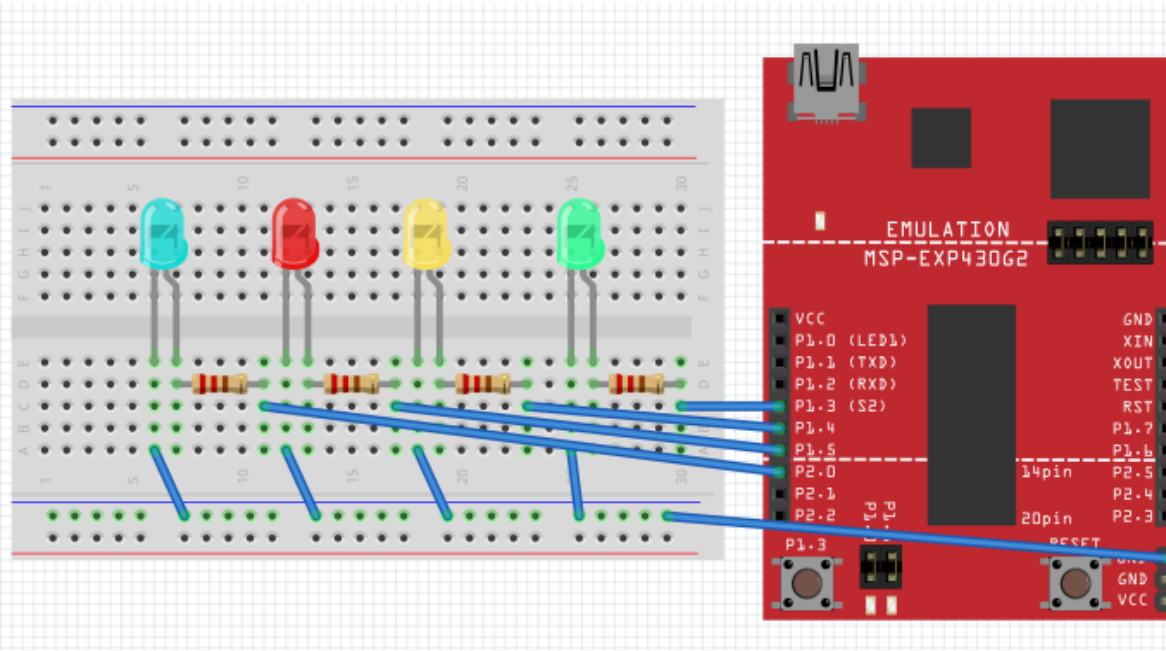


Figure 4.13.: A circuit board with wires and wires Description automatically generated

Figure 4.10: Schematic including additional blue LED to represent the crosswalk light

We'll use the button attached to P1.X as our crosswalk request button. If the crosswalk request button is pressed at any time, it will cause the state machine to transition to a crosswalk state after the next yellow light, then transition to the red light state after the crosswalk state ends. The updated state machine diagram is shown in Figure 4.11.

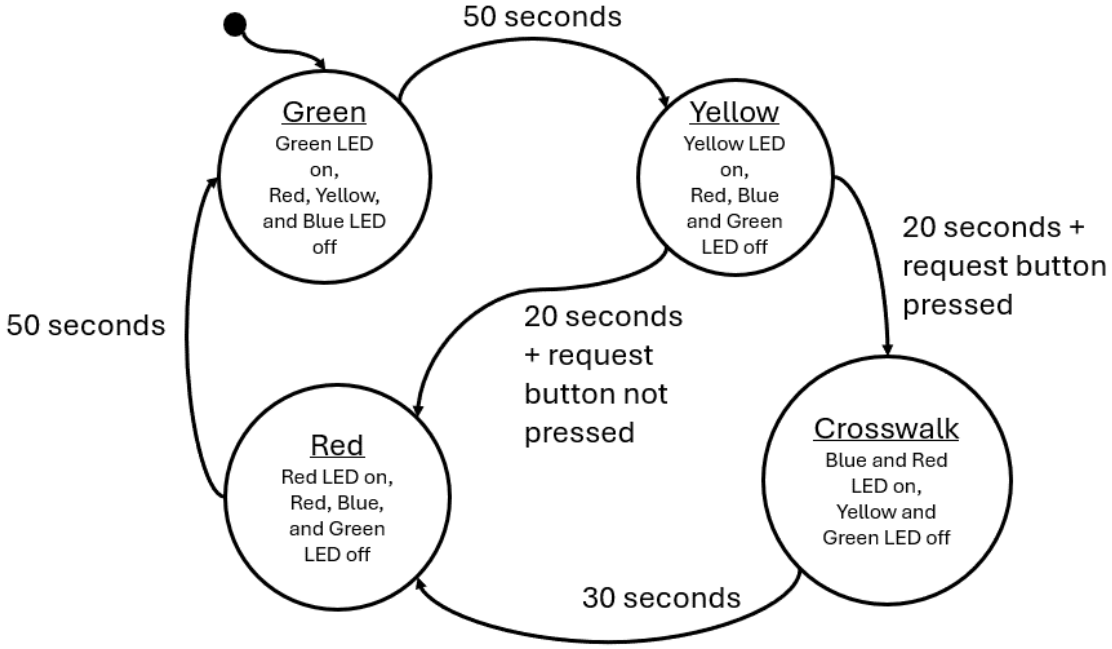


Figure 4.14.: A diagram of a crossword puzzle Description automatically generated

Figure 4.11: Traffic Light with Crosswalk State Machine Diagram

Since our state machine depends on one second delays, it is possible that the user presses the button and releases it during the delay period. In order to catch this button press, we will want to attach it to an interrupt that will acknowledge the button press no matter where we are during the code's execution.

Interrupts are usually configured through the associated peripheral's registers. For GPIO, we can enable interrupts using registers that enable

various interrupt conditions. For GPIO, each port has several registers that enable and configure the interrupt conditions.

PxIE

The first register is the PxIE register, which enables interrupts on the pins 0 through 7 of the port. For example, writing a 1 to bit 3 in this register will enable an interrupt on pin 3 of the GPIO port. Conversely, writing a 0 to bit 3 will disable the interrupt on pin 3. This register's description from the user's manual can be found in Figure 4.11.

12.4.10 PxIE Register

Port x Interrupt Enable Register

Figure 12-10. PxIE Register

7	6	5	4	3	2	1	0
PxIE							
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

Table 12-13. PxIE Register Description

Bit	Field	Type	Reset	Description
7-0	PxIE	RW	0h	Port x interrupt enable 0b = Corresponding port interrupt disabled 1b = Corresponding port interrupt enabled

Figure 4.15.: A screenshot of a computer Description automatically generated with medium confidence

Figure 4.11: The PxIE register

PxIES

The next register is the PxIES register. This register selects the interrupt source. Interrupts are often triggered by various conditions related to the

peripheral that's generating them. In the case of GPIO, interrupts can be configured to trigger when the voltage on the pin transitions from a digital high to digital low voltage, or vice versa. The PxIES register allows us to select which condition we want an individual pin to generate an interrupt. For example, writing a 1 to bit 3 of this register will enable high-to-low interrupts on pin 3, and writing a 0 to bit 3 will enable low-to-high interrupts on pin 3. This register's description from the user's manual can be found in Figure 4.12.

12.4.9 PxIES Register

Port x Interrupt Edge Select Register

Figure 12-9. PxIES Register

7	6	5	4	3	2	1	0
PxIES							
rw	rw	rw	rw	rw	rw	rw	rw

Table 12-12. PxIES Register Description

Bit	Field	Type	Reset	Description
7-0	PxIES	RW	Undefined	Port x interrupt edge select 0b = PxIFG flag is set with a low-to-high transition 1b = PxIFG flag is set with a high-to-low transition

Figure 4.16.: A screenshot of a computer Description automatically generated with medium confidence

Figure 4.12: The PxIES register

With these two registers, we can configure an interrupt on a GPIO pin. In our example, we want an interrupt to be triggered anytime a button is pressed. We have a button that is connected to ground on P2.3, so we'll configure this button with a pull-up resistor and a falling-edge triggered interrupt. That way, when the user presses the button, it will go from a digital high voltage provided by the pull-up resistor to a digital low voltage

when the button connects to ground. Initialization of this register may look like the following:

```
// Button Configuration (P2.3 as input with pull-up resistor)
P2DIR &= ~BIT3;
P2REN |= BIT3;
P2OUT |= BIT3;
// Enable high-to-low interrupts on P2.3
P2IES |= BIT3;
P2IE |= BIT3;
// Enable global interrupts
__enable_interrupt();
```

An interesting point regarding this code example is the "___enable_interrupt" line. This function is an internal compiler function for the MSP430 that enables all interrupts. Very commonly, microcontrollers will include a way to globally enable or disable interrupts. This feature exists so that if you have some important chunk of code that needs to execute without the possibility of a random interrupt breaking it's flow, you can turn off all interrupts prior (using "___disable_interrupt()") and then enable them when you're done. By default, global interrupts are disabled, so we need to turn them on before using them!

Now that we've configured the interrupt, how do we use an interrupt service routine? For the MSP430 compiler, we have to define a function and then associate it with the interrupt condition we want it to be called from. An example interrupt for reading a button press may appear as follows:

```
#pragma vector=PORT2_VECTOR
__interrupt void Port2_ISR(){
    buttonPressed = true;
    // Clear bit3 flag
    P2IFG &= ~BIT3;
}
```

This function has a lot of new things, so let's take each aspect of the function bit by bit. First of all, notice that **the function takes no parameters and returns void**. This is true for all interrupt service routines. Since they are called when an event occurs, they can't pass data and they can't return data; all due to the unpredictability of when they might be called. The only way to get data in and out of an ISR is through global variables. In this example, "buttonPressed" is a global Boolean variable that is set by the ISR. This data will be evaluated later in the main loop.

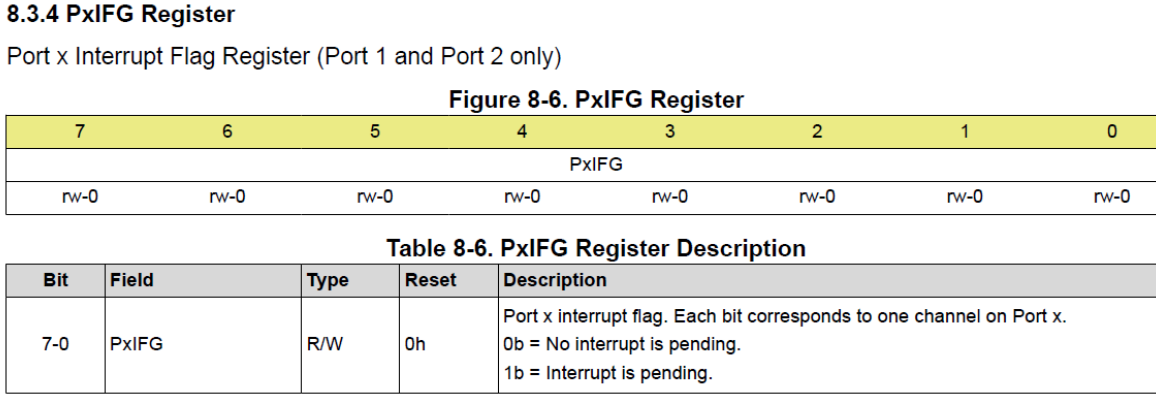
The next item to note is the #pragma line. Since this line of code begins with a hash symbol, that indicates that it is a preprocessor directive. The #pragma directive provides additional information not the compiler beyond what is contained with the language itself. The pragma directive can be used to force dependencies or throw custom warnings and errors when compiling. For this specific MSP430 compiler, it's being used to associate the following function with its position within the interrupt vector table. The MSP430 compiler understands that "vector=PORT2_VECTOR" as an association with interrupts generated by the GPIO Port2 peripheral. This also means that if we had multiple interrupts on port 2, they would all have to share this same interrupt service routine.

Preceding the function return type we see the line "__interrupt". Again, this is a compiler-specific keyword that simply indicates that the following function is an interrupt service routine.

Finally, the line of code "P2IFG &= ~BIT3". This line of code is clearing a flag bit that is set by the interrupt hardware. Many microcontrollers will include a flag register that will set bits that indicate the source of the interrupt. This register needs to be cleared at the end of each ISR call – otherwise, it will re-trigger the interrupt service routine, creating an infinite loop!

PxIFG

This PxIFG register is the "Port X Interrupt Flag" Register. When an interrupt is triggered on any pin on Port X, it's corresponding bit in the PxIFG register will be set. For example, if I have configured Port 2 Pin 3 to trigger an interrupt, if a signal occurs on that pin that would generate an interrupt, the P2IFG register would have bit 3 set.



8.3.4 PxIFG Register

Port x Interrupt Flag Register (Port 1 and Port 2 only)

Figure 8-6. PxIFG Register

7	6	5	4	3	2	1	0
PxIFG							
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

Table 8-6. PxIFG Register Description

Bit	Field	Type	Reset	Description
7-0	PxIFG	R/W	0h	Port x interrupt flag. Each bit corresponds to one channel on Port x. 0b = No interrupt is pending. 1b = Interrupt is pending.

Figure 4.17.: A screenshot of a computer Description automatically generated

Figure 4.13: The PxIFG register

This register's purpose allows us to determine which pin called the interrupt. For instance, let's say that Port 2 Pin 2 was also configured to generate an interrupt. Since all pins on Port 2 share an ISR, when the ISR is called, how can we tell which pin was responsible? This register is how! If I check the P2IFG register during the call and observe that only pin 2 is set and pin 3 is not, then I will know that pin 2 made the call. This is important for situations where we might have 2 buttons configured on the port. We might want each button to trigger different tasks (e.g., one starts a timer on a stopwatch and another resets it). This way, when the button is pressed, we can use P2IFG inside of an "if" statement within the ISR to select different actions based on which pin triggered the interrupt.

Additionally, this pin should be cleared before exiting the routine. If it is not, the hardware will cause the ISR to be called again. If any of the bits in this register are set, the ISR will be called. So therefore, after performing whatever operations we want during the ISR, we should write a 0 back to the bit that called the ISR. In our traffic light example, we see the inverse bit mask being applied to the P2IFG register before exiting the routine.

Updating the State Machine

Finally, we would write a new function for the crosswalk state and include it in our state machine.

```
state_t runCrosswalkState(uint16_t seconds){
    state_t nextState = CROSSWALK;
    // set output
    setGreenLed(false);
    setRedLed(true);
    setYellowLed(false);
    setBlueLed(true);
    // next state logic
    if(*seconds > 30){
        *seconds = 0;
        nextState = RED;
    }
    return nextState;
}
```

In addition to creating a new state for the crosswalk, we'll need to update the other states to include controls for the blue LED we added for the crosswalk and the yellow state needs to include logic to transition to either the red or crosswalk state (by evaluating the "buttonPressed" global variable) after 20 seconds:

```
uint16_t runRedState(uint16_t* seconds){
    state_t nextState = RED;
    // set output
    setRedLed(true);
    setGreenLed(false);
    setYellowLed(false);
    setBlueLed(false);
    // next state logic
    if(*seconds > 50){
        *seconds = 0;
        nextState = GREEN;
    }
    return nextState;
}
uint16_t runYellowState(uint16_t* seconds){
    state_t nextState = YELLOW;
    // set output
    setRedLed(false);
    setGreenLed(false);
    setYellowLed(true);
    // next state logic
    if(*seconds > 20){
        *seconds = 0;
        if(buttonPressed){
            buttonPressed = 0; // clear button flag
            nextState = CROSSWALK;
        } else {
            nextState = RED;
        }
    }
    return nextState;
}
uint16_t runGreenState(uint16_t* seconds){
```

```
    state_t nextState = GREEN;
    // set output
    setGreenLed(true);
    setRedLed(false);
    setYellowLed(false);
    setBlueLed(false);
    // next state logic
    if(*seconds > 50){
        *seconds = 0;
        nextState = YELLOW;
    }
    return nextState;
}
```

Additionally, we want to include the crosswalk state in the enumeration, so it can be called by the state table in our main loop:

```
typedef enum{
    GREEN, YELLOW, RED, CROSSWALK, MAX_STATES
} state_t;
```

Important Observations

An important observation about interrupt service routines is that we cannot return information from the ISR function itself. It must be a void function. Likewise, we can't pass variables into the function either. So how do we get data in and out of the routine? The answer is global variables. We must declare global variables to make data available to the routine or to other functions outside of the ISR's scope. We see this in the traffic light example above, as the global boolean variable declared "buttonPress".

It's also important to note that the ISR doesn't do much outside of setting the "buttonPress" variable to true. This is because we want to keep ISRs

as light and fast as possible. In this example, we only had one interrupt configured, but what happens if we have multiples? If an interrupt occurs when we are already servicing a different interrupt, we will have to wait until we finish to then service the next one. Therefore, we defeat the purpose of having an interrupt if we can't quickly react to external events! The way we prevent this from being an issue is by keeping the routines as simple as possible. In this case, we simply set a Boolean "flag" variable that records that the interrupt occurred, but we perform its associated task later in the main while loop.

Another solution used to handle multiple interrupts occurring at once are priority levels. Many microcontrollers will assign priority to certain interrupts so we can interrupt our interrupts! The MSP430G2553 assigns interrupt priority to interrupts according to their position within the interrupt vector table in memory. Interrupts with higher address values have higher priority.

Part III.

Peripherals

5. Timers

5.1. Learning Objectives

In this chapter we will learn:

- What is a timer?
- How do we create precise time delays?
- How to use Pulse Width Modulation?

5.2. What is a Timer?

Timers are devices used to measure… time! This is critical in many embedded systems applications as we are often interacting with other connected devices where precise timing is important for controlling and interacting with those devices. For example, consider a microwave oven driven by an embedded system. The purpose of the microwave is to activate the magnetron for a specified length of time to heat up your food. This example is fairly obvious, but we'll see that there are many, more subtle yet extremely critical applications of timers.

5.3. Counters

A timer is really an extended concept of another device called a "counter"! Counters are devices which simply store a value and increment or decrements that value on each clock edge.

Figure 5.1 describes a simple 8-bit up-counter circuit. This particular counter architecture includes an incrementor and a register, although there are many varieties of up counter implementations (For example, up-counters can commonly be made from chaining T-Flip Flops together). This up-counter increments the stored value in its register by 1 on each rising clock edge.

We can visualize the value of this simple up-counter by graphing the stored count value over time. Figure 5.2 illustrates this graph by plotting the count value on the Y-axis and clock cycles along the X-axis.

We see in Figure 5.2, that the count value increments on each complete period of the clock. For the first 255 clock cycles, the count value equals the number of clock cycles that have passed. However, once we reach 256 clock cycles, the timer's register overflows and it resets back to 0. This is because 255 is the maximum value that can be stored in an 8-bit value. After the overflow occurs, the counter starts counting up from 0 all over again.

Counters come with many different features and variations. The two largest variations are "up" and "down" counters. As the name suggests, a down-counter decrements the stored value rather than incrementing as the up-counter does. Counters also often include the ability to load a specific value into the counting register, which is especially important for the down counter, as it is often desirable to have a circuit count down from a specified value. Figure 5.3 illustrates what the architecture for a down counter with these features may look like.

Figure 5.3: An 8-bit down counter architecture

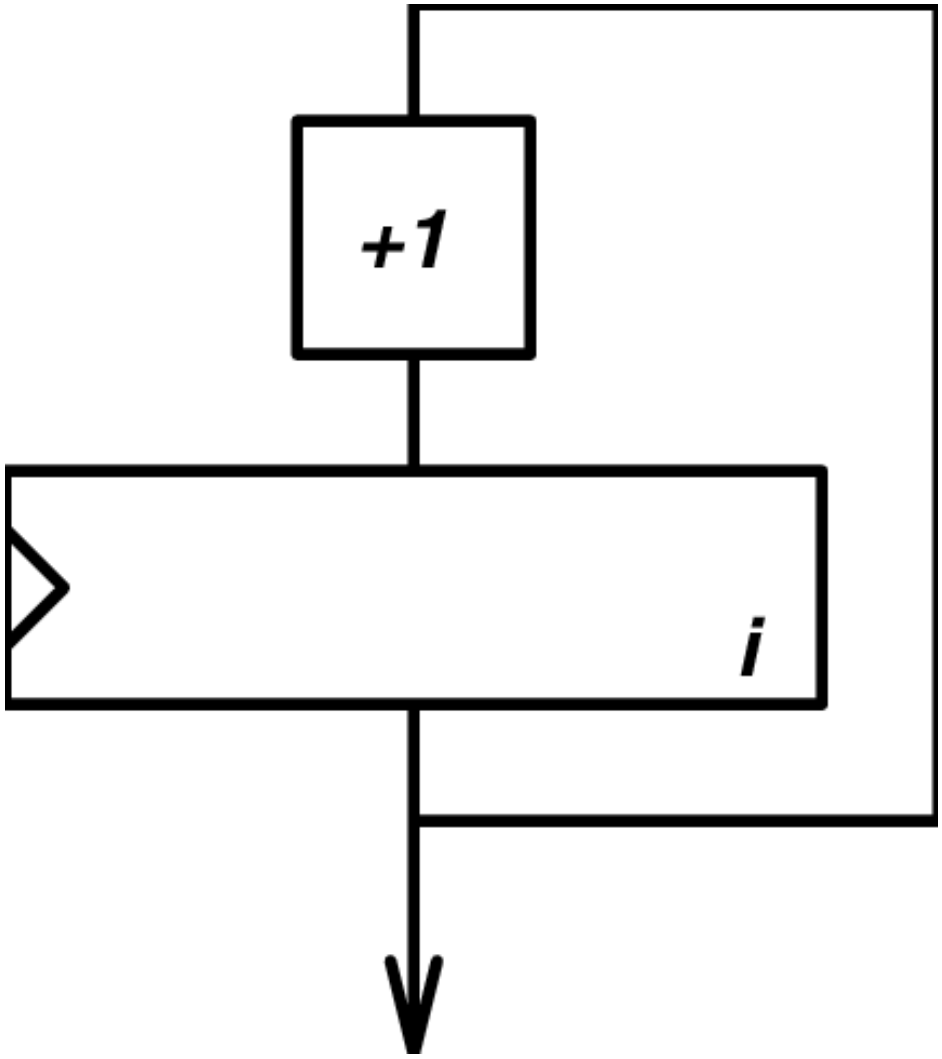


Figure 5.1.: A Simple 8-bit Counter Circuit

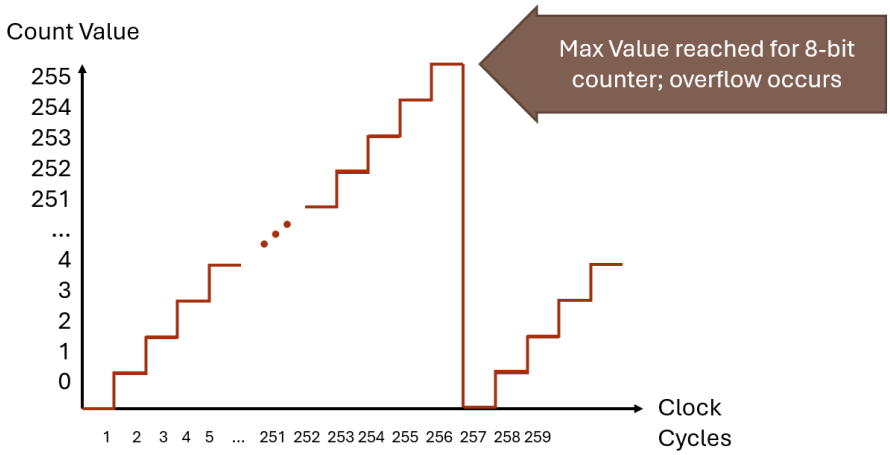


Figure 5.2.: An 8-bit counter's stored value over clock cycles

5.4. From Counters to Timers

So what makes a timer different from a counter? A timer is simply a counter where the stored count value represents a number of elapsed increments of time. A counter's value naturally represents an amount of time because the count value increments on each clock edge. If we know the length of the clock signal period, then the value of the counter is a number of periods, which have some known length of time.

For instance, if a counter started counting up from 0, and then read the counter after some time, and the count value was 147, we could determine how much time has elapsed since the counter started from 0. If the value is 147, then 147 clock cycles have passed since it started. If the period of clock is 12 milliseconds, then by multiplying 12 times 147, we can find that 1764 milliseconds have passed.

We can formally define this relationship with the equation below:

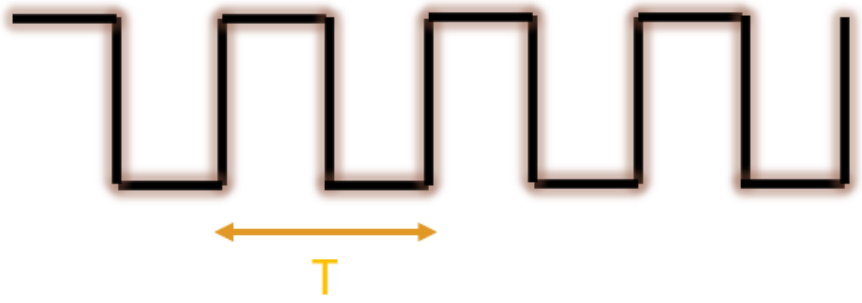


Figure 5.3.: Period of a clock signal

$$\frac{Desired\ Delay\ Time}{Clock\ Period} = Number\ of\ Counts$$

5.5. Categories of Timers

All timers perform the same task: using a counter to count clock cycles. However, timers can be designed with specific applications in mind. The timer described in the previous chapter is generally referred to as a "general purpose timer". These timers are used for a variety of purposes, such as creating a pulse with a specific period to trigger an ultrasonic sensor or measuring time between two digital signals. However, these timers may prove to be too inaccurate when it comes to measure actual time, like a wrist watch.

General Purpose Timers

Watchdog Timers

Real Time Clock

The TI MSP430G2553 Timer Peripherals

5.6. Recap

<RECAP HERE>

5.7. References

1. P.-L. Ageneau, “Chaplier Fou,” Aug. 25, 2023. [Online]. Available: https://chapelierfou.org/blog/a-usb-furby.html

6. Analog-to-Digital Conversion (ADC)

> **Draft status**
>
> This chapter has not yet been written. It will cover the MSP430G2553's analog-to-digital converter (ADC10), including configuring the ADC, sampling analog inputs, and reading conversion results.

7. Serial Communication

7.1. Learning Objectives

In this chapter we will learn:

- Why embedded systems use communication protocols rather than directly connecting every data bit
- How a two-controller vacuum robot motivates timing, clock signals, serial links, and bus ownership
- The differences between parallel and serial communication
- The meaning of synchronous, asynchronous, simplex, half-duplex, and full-duplex communication
- How UART frames data and how baud rate determines timing
- How to configure and use the MSP430G2553 USCI_A0 peripheral for UART communication
- How SPI exchanges data through clocked shift registers
- How I²C uses addressing, acknowledgments, open-drain signaling, and repeated START conditions
- How CAN provides robust multi-controller communication in noisy systems
- How to choose an appropriate serial protocol for an embedded application

7.2. Embedded Systems Need to Communicate

An embedded application often contains several microcontrollers instead of one large processor that performs every task. Different controllers can be located near the sensors and actuators they manage, can execute independently, and can be selected to match the timing, cost, and power requirements of a particular subsystem. The resulting devices must still cooperate. Dividing the work among processors therefore creates a new engineering problem: how do those processors exchange information?

7.2.1. One Vehicle, Many Microcontrollers

A modern vehicle provides a familiar example. Separate electronic control units may manage fuel injection and ignition timing, the transmission, anti-lock braking, airbag deployment, electric power steering, climate control, infotainment, navigation, and driver-assistance features. These are not isolated products that happen to share the same chassis. For example, a stability-control system may need wheel-speed information from the braking system, steering-angle information from another controller, and a torque-reduction response from the engine controller.

The practical difficulty is not deciding whether one controller can produce a high or low voltage on a GPIO pin. We have already seen that a microcontroller can do this. The difficulty is deciding how those voltages should represent useful information, how the receiving controller knows when to read them, and what happens when several devices need to communicate. To develop the solution gradually, we will begin with a smaller and more concrete embedded system.

7.2.2. A Vacuum Robot as a Running Example

Consider an autonomous vacuum robot. The robot has a front bump sensor that indicates when the chassis contacts an obstacle. It has drive motors that move the wheels and additional motors that operate the vacuum mechanism. It also has wheel encoders, which produce pulses as the wheels rotate so the electronics can estimate wheel speed and distance traveled.

Suppose the design uses two microcontrollers. Microcontroller 1 reads the bump sensor and executes the high-level behavior of the robot. Its program may implement a state machine that drives forward, stops when the bumper is pressed, backs away, turns, and resumes cleaning. Microcontroller 2 operates the motor-driver circuit and reads the wheel encoders. Its program can concentrate on generating motor-control signals and monitoring wheel motion.

Now suppose Microcontroller 1 decides that the left wheel should run at a particular speed. The desired speed exists as a number inside Microcontroller 1, but the motor hardware is controlled by Microcontroller 2. Likewise, Microcontroller 2 may measure the actual wheel speed and need to send that measurement back. The motor controller and supervisory controller therefore need a way to transmit numbers between them.

This is the central question that motivates the remainder of the chapter: if one microcontroller has an 8-bit value that the other microcontroller needs, how can we move that value between the devices? Rather than beginning with the name of an established protocol, we will build an answer using GPIO concepts that are already familiar.

Figure 7.1.: A vacuum robot combines wheel encoders with multiple subsystems that must exchange information.

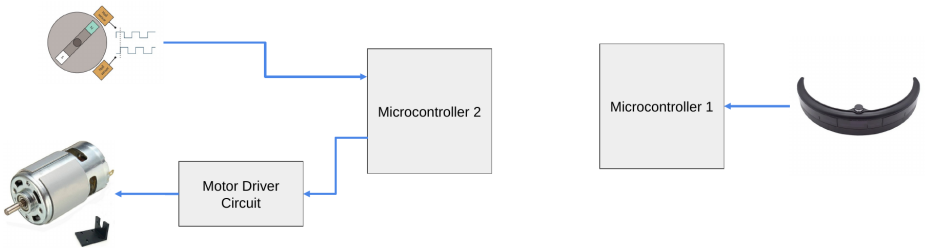


Figure 7.2.: The two-controller vacuum robot separates autonomy and bump sensing from motor control and encoder feedback.

7.3. Building a Digital Communication Link

7.3.1. First Attempt: One GPIO Wire for Every Bit

An 8-bit value contains eight individual bits. The most direct solution is therefore to connect eight GPIO output pins on the transmitting microcontroller to eight GPIO input pins on the receiving microcontroller. The devices must also share a ground reference, although the ground connection is omitted from the simplified diagrams so that the data signals remain easy to see.

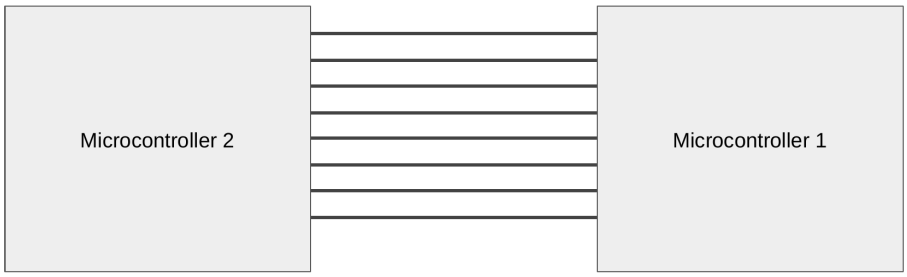


Figure 7.3.: Eight connections allow two microcontrollers to exchange the eight bits of a byte at the same time.

Each signal carries one bit position. The first wire carries bit 0, the next carries bit 1, and the remaining wires carry bits 2 through 7. The transmitter sets each output high or low according to the value of that bit, and the receiver reads all eight GPIO inputs to reconstruct the complete byte. Because all eight bits are present simultaneously, this arrangement is called parallel communication.

7.3.2. Sending the Number 21

Consider the decimal number 21. Its eight-bit binary representation is 00010101. Bit 0 is 1, bit 1 is 0, bit 2 is 1, bit 3 is 0, bit 4 is 1, and bits 5 through 7 are 0. The transmitting microcontroller drives the bit-0, bit-2, and bit-4 wires high while holding the other five wires low. When the receiving microcontroller reads the eight inputs, it sees the pattern 00010101 and reconstructs decimal 21.

```
21 decimal = 0b00010101
bit 7 bit 6 bit 5 bit 4 bit 3 bit 2 bit 1 bit 0
0 0 0 1 0 1 0 1
```

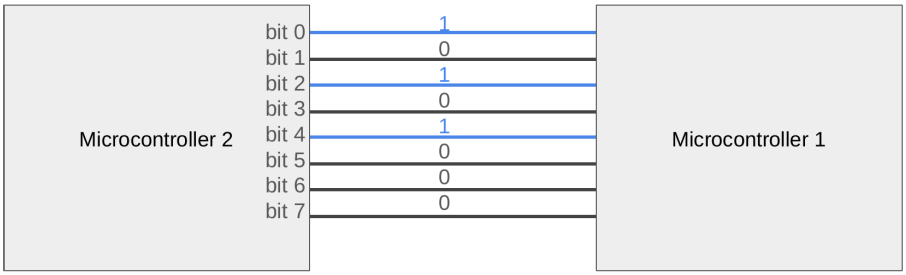


Figure 7.4.: Decimal 21 is transmitted by driving bit positions 0, 2, and 4 high on an eight-wire parallel link.

If the number represents a wheel-speed command, the motor controller can use it immediately. At first, this seems to solve our problem: eight

output pins connect to eight input pins, and the transmitted byte appears on the receiver. However, the diagram captures only a single instant. A real robot sends a sequence of commands over time, and that introduces a timing problem.

7.3.3. When Does One Value End and the Next Begin?

Suppose the transmitting controller sends the sequence 21, 21, 10. The first two values are identical, so none of the eight data wires changes when the transmitter advances from the first 21 to the second 21. The receiver can observe that the wires currently represent 21, but the wires alone do not reveal whether that 21 is the first message, the second message, or a value that has simply remained unchanged.

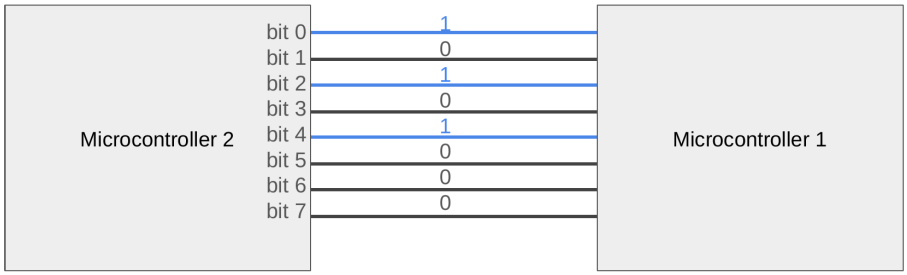


Figure 7.5.: An unchanged parallel pattern does not reveal whether a repeated value is a new message or the previous value held longer.

The same observation could correspond to several different message sequences. A receiver might interpret one long interval as 21, 21, 21, 21, 10, 10, while the transmitter intended only 21, 21, 10. This is not a problem with binary representation: both controllers can correctly identify the voltage pattern. It is a problem of synchronization. We must define when each new value becomes available and when the receiver should sample it.

7.3.4. Attempt 1: Agree on a Sampling Interval

One possible solution is for both microcontrollers to operate on an agreed schedule. For example, the transmitter could place a new byte on the eight GPIO outputs every 10 milliseconds, and the receiver could read the eight GPIO inputs every 10 milliseconds. If both controllers begin at the same time and maintain compatible timing, the receiver observes the first 21, then the second 21, and then 10.

This arrangement does not require a dedicated timing wire, but the agreement is fragile. The two microcontrollers use separate clock sources, and neither clock is perfectly accurate. If one device runs slightly faster, its sampling points gradually move relative to the other device's updates. Eventually the receiver may sample the same byte twice, skip a byte, or sample while the transmitter is changing the outputs.

An interrupt, another software task, or an unexpectedly long computation can also cause one device to miss its scheduled update. The fundamental limitation is that the transmitter and receiver infer one another's timing rather than directly observing a shared timing signal. Communication based on an agreed timing interval without a transmitted clock is asynchronous.

7.3.5. Attempt 2: Add a Clock Signal

A more direct solution is to add one additional wire that carries a clock signal. The transmitting controller first places a valid byte on the eight data lines and then changes the clock from low to high. The receiving controller watches the clock and reads all eight data lines whenever it observes that rising edge. A new rising edge means a new byte is available even if the new byte has the same value as the previous one.

The sequence 21, 21, 10 is now unambiguous. The receiver samples 21 at the first clock edge, 21 again at the second edge, and 10 at the third edge. The transmitter can speed up, slow down, or pause between transfers

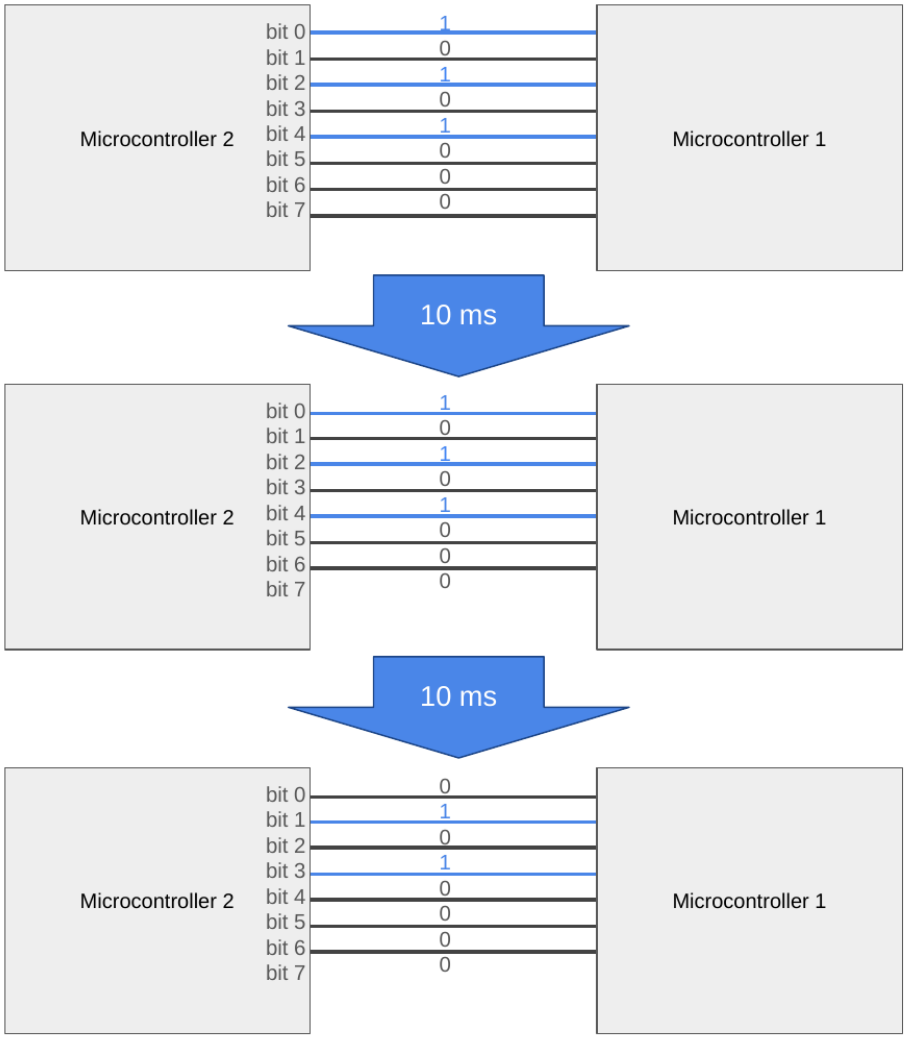


Figure 7.6.: The sequence 21, 21, 10 is transmitted in successive 10-millisecond intervals.

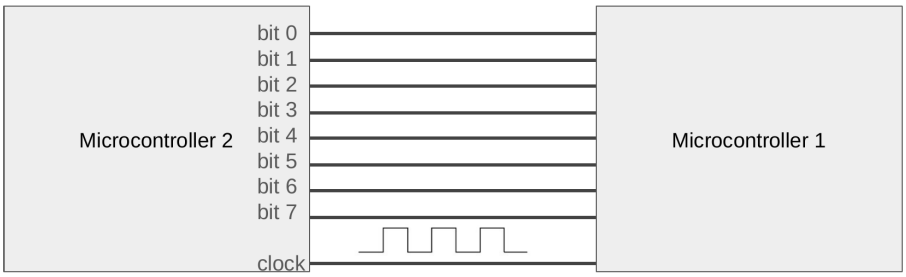


Figure 7.7.: Adding a ninth clock wire tells the receiver exactly when to sample the eight parallel data lines.

because the receiving controller responds to the clock edges rather than assuming that the next byte will appear after a fixed delay.

7.3.6. Synchronous and Asynchronous Communication

These two approaches introduce an important distinction. Synchronous communication includes a timing signal, usually called a clock, that tells the receiver when data should be sampled. Asynchronous communication does not transmit a separate clock. Instead, both devices must agree on timing parameters and use their local clocks to estimate when the data should be read.

Synchronous interfaces can often operate more reliably at high speeds because the transmitter explicitly identifies the sampling instants. This does not mean asynchronous interfaces are unreliable. Practical asynchronous protocols add synchronization markers and limit how long the receiver must remain aligned without resynchronizing. UART, which we will study shortly, is an example of a carefully structured asynchronous interface.

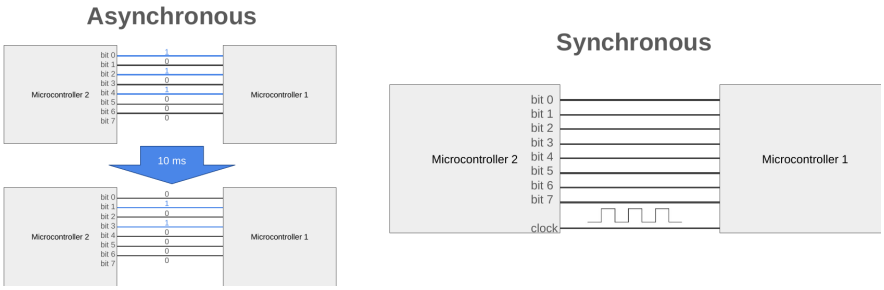


Figure 7.8.: An agreed 10-millisecond asynchronous schedule contrasts with a synchronous link that carries a clock.

7.4. From Parallel Communication to Serial Communication

7.4.1. The Cost of a Wide Parallel Bus

Our clocked parallel interface transfers an entire byte on each clock edge, but the connection requires eight data pins plus one clock pin on each microcontroller. If the robot also needs bump sensors, motor-control outputs, encoder inputs, indicators, and other peripherals, dedicating nine pins to a single communication channel is expensive. Wider data transfers consume even more pins: a 16-bit interface would require sixteen data wires plus timing and control signals.

The wiring also increases connector size, printed-circuit-board routing complexity, and cable cost. At higher speeds, the individual wires do not necessarily deliver their signals to the receiver at exactly the same instant. Differences in trace length and electrical loading produce timing skew, which can force the receiver to wait for the slowest signal before safely sampling the group.

7.4.2. Reuse One Wire for Every Bit

Instead of providing a separate wire for each bit, suppose we place one bit at a time on a single data wire. The transmitter drives the first bit onto the wire and produces a clock edge. The receiver samples the wire and stores that bit. The transmitter then places the next bit on the same wire, generates another clock edge, and repeats the process until all eight bits have been transferred.

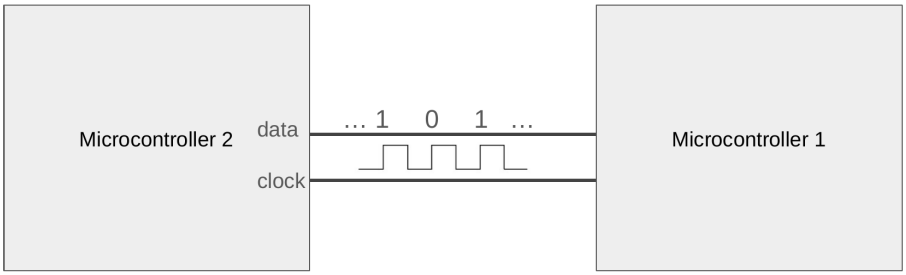


Figure 7.9.: Replacing eight parallel data wires with one data wire and one clock wire creates a synchronous serial link.

The receiving controller reconstructs the original byte by collecting the bits in their agreed order. For decimal 21, an interface that sends the most-significant bit first transmits 0, 0, 0, 1, 0, 1, 0, 1. An interface that sends the least-significant bit first transmits 1, 0, 1, 0, 1, 0, 0, 0. Either order is valid as long as both devices follow the same convention.

This is serial communication: the individual bits of a value move sequentially over a shared data path rather than simultaneously across separate wires. The transfer now requires eight clock edges instead of one, but the number of data wires falls from eight to one. The resulting savings in pins, wiring, board area, and connector size explain why serial communication dominates modern embedded systems.

7.4.3. Familiar Examples of Parallel and Serial Interfaces

Older desktop computers often included a parallel printer port, which transferred several data bits simultaneously, and a serial port, which transferred bits sequentially using asynchronous signaling such as UART with RS-232 electrical levels. Universal Serial Bus, or USB, is another familiar serial interface, although its signaling and packet structure are substantially more complex than the UART, SPI, and I²C examples in this chapter.

The important distinction is not whether a connector is physically large or small. Parallel communication dedicates multiple data wires to the bits of a value. Serial communication reuses a smaller number of data wires over successive bit periods. Whether the timing is synchronous or asynchronous is a separate design choice.

7.5. Who Is Allowed to Transmit?

7.5.1. Simplex Communication

So far, our serial example sends data from one microcontroller to the other. One device acts as the transmitter, labeled TX, and the other acts as the receiver, labeled RX. If information is only allowed to move in this one direction, the connection is simplex. A sensor that only reports measurements to a controller can operate this way.

Simplex communication is not sufficient for the vacuum robot if the supervisory controller needs to send motor-speed commands while the motor controller also needs to return encoder measurements. We must extend the connection so both devices can transmit.

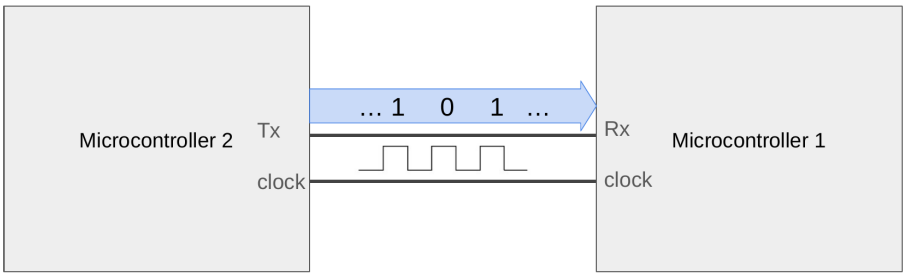


Figure 7.10.: A simplex connection assigns one transmitter and one receiver, so data travels in only one direction.

7.5.2. Sharing a Bidirectional Data Wire

One option is to let both devices use the same data wire. A microcontroller configures its GPIO pin as an output when it is transmitting and as an input when it is receiving. After one message is complete, the devices can reverse their roles. The wire is therefore capable of carrying information in either direction, but only when both controllers agree about which one currently owns it.

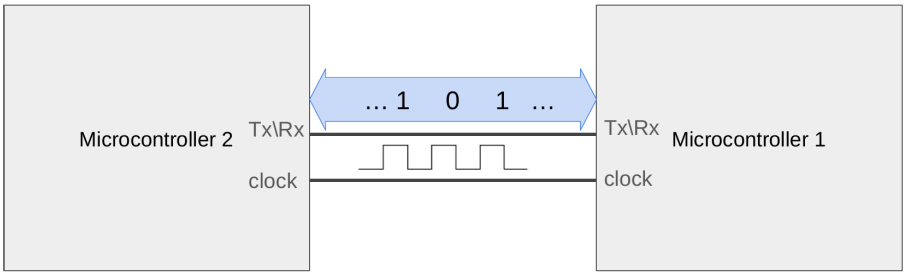


Figure 7.11.: A shared data wire can be bidirectional when each microcontroller changes its GPIO direction at the correct time.

7.5.3. What Happens If Both Devices Transmit?

Sharing one actively driven signal creates an immediate concern: what if both microcontrollers attempt to transmit at the same time? Suppose one controller drives the line high while the other controller drives the same line low. The outputs are now attempting to impose incompatible voltages on the same conductor. The resulting condition is called bus contention.

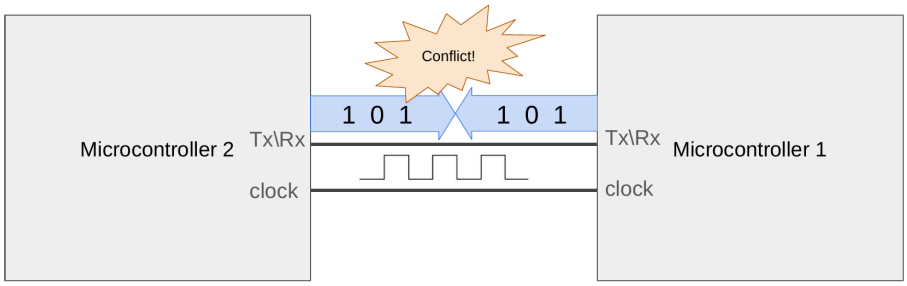


Figure 7.12.: If two push-pull outputs drive conflicting values on a shared wire, the bus experiences contention.

Bus contention can produce an undefined logic level, corrupted data, excessive current, and potentially damaged output drivers. A shared interface therefore needs a rule that prevents simultaneous push-pull transmission. Software can assign turns, hardware can select one device at a time, or the electrical interface can use a signaling method such as the open-drain outputs used by I²C.

7.5.4. Half-Duplex Communication

When both devices can communicate but must take turns on the same data path, the interface is half-duplex. Microcontroller 1 may transmit a motor command, release the line, and then allow Microcontroller 2 to transmit its encoder feedback. Both directions are supported, but they cannot carry independent messages simultaneously.

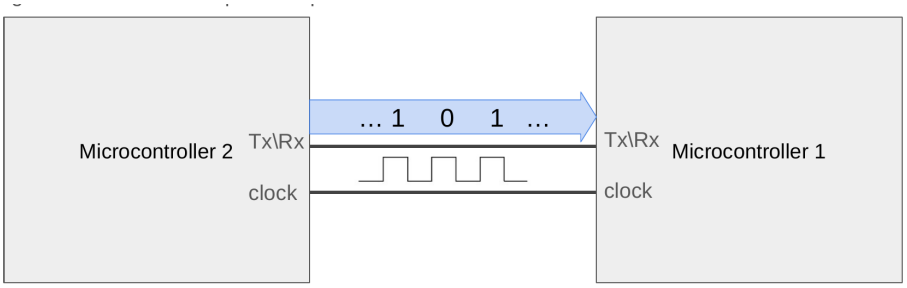


Figure 7.13.: A half-duplex connection supports both directions, but the devices must take turns using the shared data wire.

A half-duplex protocol must define when a device is allowed to begin, how the other device recognizes the end of the current message, and who controls the line during responses. These rules may be implemented through timing, addressing, arbitration, chip-selection signals, or electrical conventions. The reduced pin count is useful, but coordinating bus ownership adds complexity.

7.5.5. Full-Duplex Communication

The alternative is to add a second data wire. One wire carries information from Microcontroller 1 to Microcontroller 2, and the other carries information from Microcontroller 2 to Microcontroller 1. Each output drives only its dedicated direction, so both devices can send data at the same time without competing for the same conductor. This arrangement is full-duplex communication.

These classifications reappear throughout the remainder of the chapter. UART normally uses a dedicated transmit wire and a dedicated receive wire, allowing full-duplex operation. SPI also has separate controller-output and controller-input data paths, so it can shift bits in both directions on each clock cycle. I²C shares one bidirectional data line and therefore

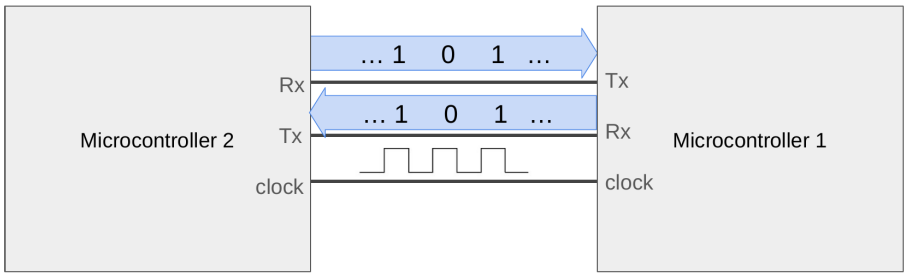


Figure 7.14.: A full-duplex connection uses separate transmit and receive paths so both devices can communicate simultaneously.

operates as a half-duplex interface. CAN likewise uses a shared bus, but its electrical signaling and arbitration rules determine which message wins when multiple devices begin transmitting.

7.6. Asynchronous Serial Communication

The clocked serial link in the previous example requires one data wire plus one clock wire for a single direction. We can reduce the required wiring further if the transmitter and receiver agree on how long each bit remains valid and add a recognizable marker that announces the beginning of a message. The receiver can then reconstruct the sampling instants from its own local clock. This idea leads directly to UART.

7.7. Universal Asynchronous Receiver/Transmitter

A Universal Asynchronous Receiver/Transmitter, normally shortened to UART, is a hardware peripheral that converts bytes in the processor into timed serial frames and converts received frames back into bytes. The word universal reflects that the peripheral can be configured for several

frame formats and rates. UART describes the digital communication mechanism inside the devices. It does not by itself specify a connector or a long-distance electrical standard.

A UART link normally uses one transmit signal, one receive signal, and a shared ground. Because no clock is transmitted, the devices must agree on the baud rate, number of data bits, parity setting, and number of stop bits. A common configuration is written as 9600 8N1: 9600 baud, eight data bits, no parity, and one stop bit.

7.7.1. Baud Rate and Bit Time

Baud rate is the number of symbols transmitted each second. A simple UART represents one bit with each symbol, so its baud rate is numerically equal to its bit rate. At 9600 baud, one bit occupies

```
bit time = 1 / 9600 = 104.17 microseconds
```

A complete frame contains more than the eight payload bits. With 8N1 formatting, one start bit and one stop bit are added, giving ten transmitted bits per byte. The maximum payload rate is therefore 960 bytes per second, not 1200 bytes per second. This distinction matters when estimating whether an interface is fast enough for a stream of measurements.

7.7.2. Framing a UART Message

The UART signal remains high while the link is idle. To begin a frame, the transmitter drives the signal low for one bit period. This start bit creates an edge that allows the receiver to synchronize its local timing. The data bits follow, usually least-significant bit first. An optional parity bit may be added, followed by one or more high stop bits. The stop bit returns the signal to the idle condition and provides separation before the next frame.

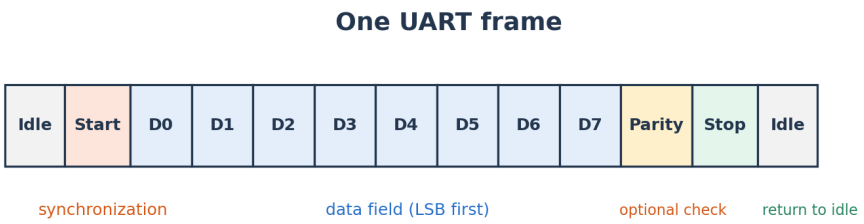


Figure 7.15.: A configurable UART frame surrounds the data field with synchronization and optional error-checking bits.

7.7.2.1. Data Bits

Most current embedded applications use eight data bits because a byte is the natural unit of memory and text encodings such as ASCII fit within a byte. UART hardware can also support shorter data fields for older or specialized systems. The transmitter and receiver must use the same length. If one side expects seven data bits while the other sends eight, every following field will be interpreted at the wrong location.

7.7.2.2. Start and Stop Bits

The start bit is always the opposite of the idle level. The receiver detects the high-to-low transition and waits approximately half a bit period before confirming the start bit near its center. It then samples each data bit one bit period apart. After the final data or parity bit, the receiver expects a high stop bit. If the signal is low where the stop bit should occur, the UART reports a framing error. A framing error commonly indicates mismatched baud rates, incorrect frame settings, electrical noise, or a wire connected to the wrong signal.

7.7.2.3. Parity

Parity provides simple error detection. With even parity, the transmitter selects the parity bit so the total number of 1 bits in the data and parity field is even. Odd parity makes the total odd. If one bit changes during transmission, the receiver computes a different parity and reports an error. Parity cannot correct the damaged data, and it cannot detect every multi-bit error. Many short, reliable UART links omit parity and rely on a higher-level checksum when stronger protection is needed.

7.7.3. A 9600 8N1 Example

Suppose the vacuum robot reports that a command completed successfully by transmitting the two-character message OK. The ASCII value of uppercase O is decimal 79, or hexadecimal 0x4F; its eight-bit binary representation is 01001111. Uppercase K is decimal 75, or hexadecimal 0x4B; its binary representation is 01001011. UART creates a separate frame for each character rather than transmitting the letters as one unstructured voltage sequence.

```
'O' = 79 decimal = 0x4F = 0b01001111
'K' = 75 decimal = 0x4B = 0b01001011
```

The figure below shows a frame carrying the character O. The start bit is low, the eight data bits are transmitted least-significant bit first, and the stop bit is high. Every bit occupies approximately 104 microseconds at 9600 baud. The entire ten-bit frame therefore requires approximately 1.04 milliseconds, and the two-character message requires approximately 2.08 milliseconds before any optional spacing between frames.

The receiver does not need a clock edge for every bit. It assumes that the edges between bit cells occur at multiples of the configured bit time after the start edge. Sampling near the center of each cell gives the largest

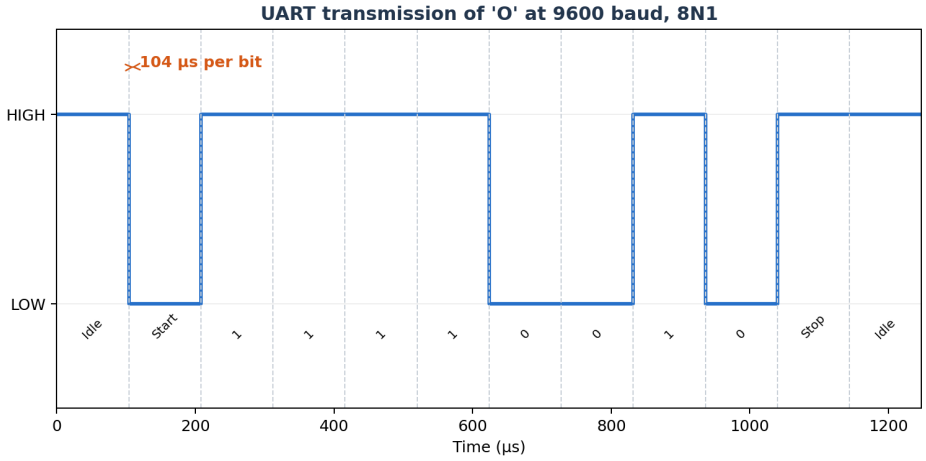


Figure 7.16.: A 9600-baud 8N1 UART frame carrying the ASCII character 'O'.

tolerance to jitter and clock mismatch. The frame is deliberately short so timing error does not accumulate indefinitely.

7.7.4. UART Wiring and Voltage Levels

The transmit output of one device must connect to the receive input of the other. Connecting TX to TX causes both outputs to drive the same line and leaves neither receiver connected to useful data. The devices also need a common ground so their voltage thresholds have the same reference. The following diagram shows the normal cross-connection.

The MSP430G2553 uses logic-level signals near its supply voltage. These signals are often informally called TTL serial, although a 3.3-V CMOS output is not literally the older 5-V TTL standard. Logic-level UART should only connect directly to a device with compatible voltage thresholds.

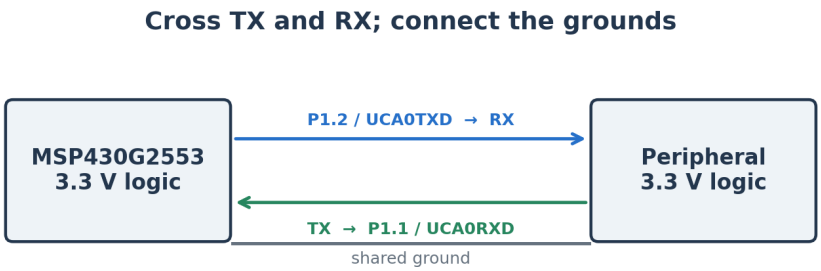


Figure 7.17.: A logic-level UART connection crosses transmit and receive and shares a ground reference.

A 5-V output can damage a 3.3-V-only input unless a level shifter or voltage divider is used.

RS-232 is a different electrical standard that also carries asynchronous serial frames. It uses positive and negative voltages and reverses the logical polarity relative to a typical microcontroller UART. An RS-232 cable must not be connected directly to an MSP430 pin. A transceiver such as the MAX3232 translates between the voltage domains. Likewise, the USB connector on a computer does not carry UART voltage levels. A USB-to-UART bridge is required to create a virtual serial port.

7.8. The MSP430G2553 Serial Communication Hardware

The MSP430G2553 contains a Universal Serial Communication Interface, abbreviated USCI. This name is easy to confuse with the simpler USI peripheral found on some other MSP430 devices. The distinction matters:

the MSP430G2553 uses USCI, and its USCI_A0 module includes UART hardware. A plain USI module supports synchronous interfaces but does not provide UART.

The device contains two related modules. USCI_A0 supports UART and SPI. USCI_B0 supports SPI and I²C. Each module can operate in only one mode at a time, so using USCI_A0 for UART means that same module cannot simultaneously provide an additional SPI bus. The following diagram summarizes the relationship between the CPU, the USCI modules, and the pins.

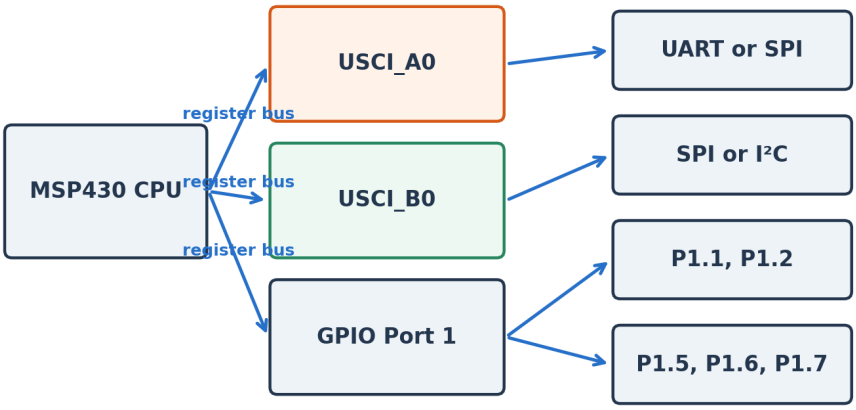


Figure 7.18.: The MSP430G2553 provides UART/SPI through USCI_A0 and SPI/I²C through USCI_B0.

7.8.1. Finding the UART Pins

Peripheral hardware is connected internally to selected package pins. The MSP430G2553 pinout identifies P1.1 as UCA0RXD and P1.2 as UCA0TXD. The same pins can also serve as GPIO or SPI signals, so software must

select the peripheral function before UART data can reach the pins. The P1SEL and P1SEL2 registers control this routing.

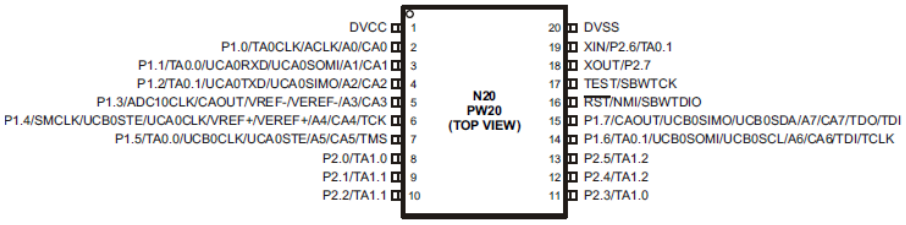


Figure 7.19.: The 20-pin MSP430G2553 pinout identifies the alternative peripheral functions available on each GPIO pin.

```
P1SEL |= BIT1 | BIT2;
P1SEL2 |= BIT1 | BIT2;
```

Setting both selection bits for P1.1 and P1.2 connects those pins to USCI_A0. From that point, the UART peripheral rather than the ordinary GPIO data registers controls the pins. When debugging a serial connection, checking the pin-selection registers is just as important as checking the UART registers.

7.8.2. USCI_A0 UART Registers

The UART configuration is distributed across several memory-mapped registers. UCA0CTL0 selects the frame format. UCA0CTL1 selects the clock source and contains the UCSWRST software-reset bit. UCA0BR0, UCA0BR1, and UCA0MCTL form the baud-rate generator. UCA0TXBUF and UCA0RXBUF provide the software-visible transmit and receive buffers. The IE2 and IFG2 special-function registers contain interrupt-enable and status flags associated with USCI_A0.

Table 15-6. USCI_Ax Control and Status Registers

Address	Acronym	Register Name	Type	Reset	Section
60h	UCA0CTL0	USCI_A0 control 0	Read/write	00h with PUC	Section 15.4.1
61h	UCA0CTL1	USCI_A0 control 1	Read/write	01h with PUC	Section 15.4.2
62h	UCA0BR0	USCI_A0 baud-rate control 0	Read/write	00h with PUC	Section 15.4.3
63h	UCA0BR1	USCI_A0 baud-rate control 1	Read/write	00h with PUC	Section 15.4.3
64h	UCA0MCTL	USCI_A0 modulation control	Read/write	00h with PUC	Section 15.4.5
65h	UCA0STAT	USCI_A0 status	Read/write	00h with PUC	Section 15.4.6
66h	UCA0RXBUF	USCI_A0 receive buffer	Read	00h with PUC	Section 15.4.7
67h	UCA0TXBUF	USCI_A0 transmit buffer	Read/write	00h with PUC	Section 15.4.8
5Dh	UCA0ABCTL	USCI_A0 auto baud control	Read/write	00h with PUC	Section 15.4.11
5Eh	UCA0IRTCTL	USCI_A0 IrDA transmit control	Read/write	00h with PUC	Section 15.4.9
5Fh	UCA0IRRCTL	USCI_A0 IrDA receive control	Read/write	00h with PUC	Section 15.4.10
1h	IE2	SFR interrupt enable 2	Read/write	00h with PUC	Section 15.4.12
3h	IFG2	SFR interrupt flag 2	Read/write	0Ah with PUC	Section 15.4.13

Figure 7.20.: The USCI_A0 control, status, baud-rate, and data registers used by UART mode.

7.8.2.1. Frame Format: UCA0CTL0

The UCA0CTL0 register controls parity, bit order, data length, stop bits, and operating mode. Standard 8N1 UART is represented by the reset values for these format fields: parity disabled, least-significant bit first, eight data bits, one stop bit, asynchronous mode. Explicitly writing zero to the register makes this intent clear.

7.8.2.2. Clock Source and Reset: UCA0CTL1

USCI configuration should be changed while the module is held in software reset. UCSWRST is set before modifying the mode and baud-rate registers. UCSSEL_2 selects the subsystem master clock, SMCLK, as the bit-clock source. Clearing UCSWRST after configuration allows the peripheral state machine to operate.

Figure 15-12. UCAxCTL0 Register

7	6	5	4	3	2	1	0
UCPEN	UCPAR	UCMSB	UC7BIT	UCSPB	UCMODEx		UCSYNC
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

Table 15-7. UCAxCTL0 Register Field Descriptions

Bit	Field	Type	Reset	Description
7	UCPEN	R/W	0h	Parity enable 0b = Parity disabled 1b = Parity enabled. Parity bit is generated (UCAxTXD) and expected (UCAxRXD). In address-bit multiprocessor mode, the address bit is included in the parity calculation.
6	UCPAR	R/W	0h	Parity select. UCPAR is not used when parity is disabled. 0b = Odd parity 1b = Even parity
5	UCMSB	R/W	0h	MSB first select. Controls the direction of the receive and transmit shift register. 0b = LSB first 1b = MSB first
4	UC7BIT	R/W	0h	Character length. Selects 7-bit or 8-bit character length. 0b = 8-bit data 1b = 7-bit data
3	UCSPB	R/W	0h	Stop bit select. Number of stop bits. 0b = One stop bit 1b = Two stop bits
2-1	UCMODEx	R/W	0h	USCI mode. The UCMODEx bits select the asynchronous mode when UCSYNC = 0. 00b = UART mode 01b = Idle-line multiprocessor mode 10b = Address-bit multiprocessor mode 11b = UART mode with automatic baud rate detection
0	UCSYNC	R/W	0h	Synchronous mode enable 0b = Asynchronous mode 1b = Synchronous mode

Figure 7.21.: UCA0CTL0 selects the UART frame format.

Figure 15-13. UCAxCTL1 Register

7	6	5	4	3	2	1	0
UCSSELx		UCRXEIE	UCBRKIE	UCDORM	UCTXADDR	UCTXBRK	UCSWRST
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-1

Table 15-8. UCAxCTL1 Register Field Descriptions

Bit	Field	Type	Reset	Description
7-6	UCSSELx	R/W	0h	USCI clock source select. These bits select the BRCLK source clock. 00b = UCLK 01b = ACLK 10b = SMCLK 11b = SMCLK
5	UCRXEIE	R/W	0h	Receive erroneous-character interrupt-enable 0b = Erroneous characters rejected and UCAxRXIFG is not set 1b = Erroneous characters received will set UCAxRXIFG
4	UCBRKIE	R/W	0h	Receive break character interrupt-enable 0b = Received break characters do not set UCAxRXIFG. 1b = Received break characters set UCAxRXIFG.
3	UCDORM	R/W	0h	Dormant. Puts USCI into sleep mode. 0b = Not dormant. All received characters will set UCAxRXIFG. 1b = Dormant. Only characters that are preceded by an idle-line or with address bit set will set UCAxRXIFG. In UART mode with automatic baud rate detection only the combination of a break and synch field will set UCAxRXIFG.
2	UCTXADDR	R/W	0h	Transmit address. Next frame to be transmitted will be marked as address depending on the selected multiprocessor mode. 0b = Next frame transmitted is data 1b = Next frame transmitted is an address
1	UCTXBRK	R/W	0h	Transmit break. Transmits a break with the next write to the transmit buffer. In UART mode with automatic baud rate detection 055h must be written into UCAxTXBUF to generate the required break/synch fields. Otherwise 0h must be written into the transmit buffer. 0b = Next frame transmitted is not a break 1b = Next frame transmitted is a break or a break/synch
0	UCSWRST	R/W	1h	Software reset enable 0b = Disabled. USCI reset released for operation. 1b = Enabled. USCI logic held in reset state.

Figure 7.22.: UCA0CTL1 selects the baud-rate clock and holds the USCI module in software reset during configuration.

7.8.3. Configuring the Baud-Rate Generator

The baud-rate generator divides a source clock into the timing used by the UART. Let fBRCLK be the selected clock frequency and fbaud be the desired baud rate. The ideal divider is

```
N = fBRCLK / fbaud
```

For a calibrated 1-MHz SMCLK and 9600 baud, N is 104.167. The integer portion, 104, is written into the combined UCA0BR1:UCA0BR0 divider. The remaining fraction cannot be represented by the integer divider, so UCA0MCTL periodically adjusts the timing. Texas Instruments tabulates modulation settings for common frequencies. At 1 MHz and 9600 baud, the recommended settings are UCA0BR0 = 104, UCA0BR1 = 0, and UCA0MCTL = UCBRS0.

The calibrated-clock constants stored in the device information memory are used to configure the digitally controlled oscillator. Production code should verify that these calibration constants are valid before using them, because an erased information segment contains 0xFF rather than usable values.

7.8.4. Example: Transmitting a Character

The following program configures USCI_A0 for 9600-baud 8N1 communication and transmits the character ‘A’ approximately once per second. The transmit function waits for UCA0TXIFG before writing UCA0TXBUF. The flag indicates that the transmit buffer can accept another byte; it does not necessarily mean the final stop bit has already left the pin.

```
#include <msp430.h>
static void uart_init(void)
{
```

Table 15-4. Commonly Used Baud Rates, Settings, and Errors, UCOS16 = 0

BRCLK Frequency [Hz]	Baud Rate [Baud]	UCBRx	UCBRSx	UCBRFx	Maximum TX Error [%]		Maximum RX Error [%]	
32,768	1200	27	2	0	-2.8	1.4	-5.9	2.0
32,768	2400	13	6	0	-4.8	6.0	-9.7	8.3
32,768	4800	6	7	0	-12.1	5.7	-13.4	19.0
32,768	9600	3	3	0	-21.1	15.2	-44.3	21.3
1,048,576	9600	109	2	0	-0.2	0.7	-1.0	0.8
1,048,576	19200	54	5	0	-1.1	1.0	-1.5	2.5
1,048,576	38400	27	2	0	-2.8	1.4	-5.9	2.0
1,048,576	56000	18	6	0	-3.9	1.1	-4.6	5.7
1,048,576	115200	9	1	0	-1.1	10.7	-11.5	11.3
1,048,576	128000	8	1	0	-8.9	7.5	-13.8	14.8
1,048,576	256000	4	1	0	-2.3	25.4	-13.4	38.8
1,000,000	9600	104	1	0	-0.5	0.6	-0.9	1.2
1,000,000	19200	52	0	0	-1.8	0	-2.6	0.9
1,000,000	38400	26	0	0	-1.8	0	-3.6	1.8
1,000,000	56000	17	7	0	-4.8	0.8	-8.0	3.2
1,000,000	115200	8	6	0	-7.8	6.4	-9.7	16.1
1,000,000	128000	7	7	0	-10.4	6.4	-18.0	11.6
1,000,000	256000	3	7	0	-29.6	0	-43.6	5.2
4,000,000	9600	416	6	0	-0.2	0.2	-0.2	0.4
4,000,000	19200	208	3	0	-0.2	0.5	-0.3	0.8
4,000,000	38400	104	1	0	-0.5	0.6	-0.9	1.2
4,000,000	56000	71	4	0	-0.6	1.0	-1.7	1.3
4,000,000	115200	34	6	0	-2.1	0.6	-2.5	3.1
4,000,000	128000	31	2	0	-0.8	1.6	-3.6	2.0
4,000,000	256000	15	5	0	-4.0	3.2	-8.4	5.2
8,000,000	9600	833	2	0	-0.1	0	-0.2	0.1
8,000,000	19200	416	6	0	-0.2	0.2	-0.2	0.4
8,000,000	38400	208	3	0	-0.2	0.5	-0.3	0.8
8,000,000	56000	142	7	0	-0.6	0.1	-0.7	0.8
8,000,000	115200	69	4	0	-0.6	0.8	-1.8	1.1
8,000,000	128000	62	4	0	-0.8	0	-1.2	1.2
8,000,000	256000	31	2	0	-0.8	1.6	-3.6	2.0
12,000,000	9600	1250	0	0	0	0	-0.05	0.05

Figure 7.23.: Common USCI UART divider and modulation settings from the MSP430 family documentation.

```
    BCSCTL1 = CALBC1_1MHZ;
    DCOCTL = CALDCO_1MHZ;
    P1SEL |= BIT1 | BIT2;
    P1SEL2 |= BIT1 | BIT2;
    UCA0CTL1 = UCSWRST | UCSSEL_2;
    UCA0CTL0 = 0; // 8 data, no parity, 1 stop
    UCA0BR0 = 104; // 1 MHz / 9600 baud
    UCA0BR1 = 0;
    UCA0MCTL = UCBRS0;
    UCA0CTL1 &= ~UCSWRST;
}
static void uart_send(char value)
{
    while ((IFG2 & UCA0TXIFG) == 0) {
        // Wait until UCA0TXBUF is available.
    }
    UCA0TXBUF = value;
}
int main(void)
{
    WDTCTL = WDTPW | WDTHOLD;
    uart_init();
    while (1) {
        uart_send('A');
        __delay_cycles(1000000);
    }
}
```

The processor does not manually toggle P1.2 for every bit. Software writes one byte to UCA0TXBUF, and the USCI hardware moves that byte into an internal shift register when the transmitter is ready. The shift register generates the start bit, data bits, and stop bit with the configured timing. This division of responsibility is one of the primary benefits of a hardware

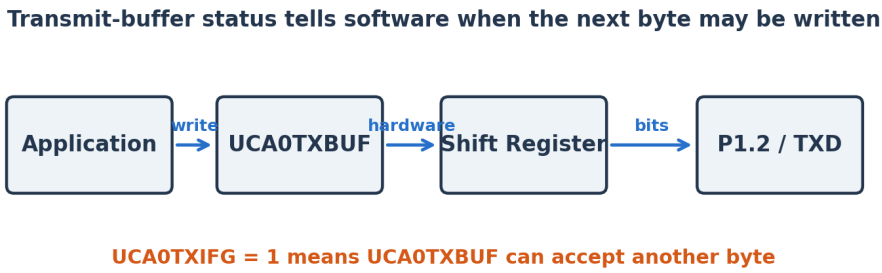


Figure 7.24.: Writing UCA0TXBUF begins a hardware-managed transfer through the transmit shift register.

peripheral.

7.8.5. Receiving Data by Polling

The receive side works in the opposite direction. USCI_A0 monitors P1.1, recognizes a valid start bit, samples the data, checks the frame, and places the completed byte in UCA0RXBUF. UCA0RXIFG is set when the byte is ready. Reading UCA0RXBUF clears the receive flag. A simple blocking receive function is shown below.

```
static char uart_receive(void)
{
    while ((IFG2 & UCA0RXIFG) == 0) {
        // Wait for a complete frame.
    }
    return UCA0RXBUF;
}
```

Polling is appropriate when the program has nothing useful to do until a byte arrives. It becomes wasteful when the processor should handle other work or enter a low-power mode. If a byte arrives while UCA0RXBUF still holds unread data, the previous value can be overwritten and an overrun error can occur. The program must service the receiver quickly enough or use an interrupt and a software buffer.

7.8.6. Interrupt-Driven UART Echo

An echo program sends every received character back to the transmitter. It is useful because one short test exercises the pin routing, baud-rate settings, receive hardware, interrupt system, and transmit hardware. The receive interrupt allows the main program to sleep until a character arrives.

```
#include <msp430.h>
int main(void)
{
    WDTCTL = WDTPW | WDTHOLD;
    BCSCTL1 = CALBC1_1MHZ;
    DCOCTL = CALDCO_1MHZ;
    P1SEL |= BIT1 | BIT2;
    P1SEL2 |= BIT1 | BIT2;
    UCA0CTL1 = UCSWRST | UCSSEL_2;
    UCA0CTL0 = 0;
    UCA0BR0 = 104;
    UCA0BR1 = 0;
    UCA0MCTL = UCBRS0;
    UCA0CTL1 &= ~UCSWRST;
    IE2 |= UCA0RXIE;
    __bis_SR_register(LPM0_bits | GIE);
    while (1) {
        // Execution resumes here only if the ISR wakes the CPU.
    }
```

```
}
#pragma vector=USCIAB0RX_VECTOR
__interrupt void usci_rx_isr(void)
{
    char received = UCA0RXBUF;
    while ((IFG2 & UCA0TXIFG) == 0) {
        // Wait for the transmit buffer.
    }
    UCA0TXBUF = received;
}
```

The MSP430G2553 combines the USCI_A0 and USCI_B0 receive sources into the USCIAB0RX_VECTOR. A larger program should inspect the flags to determine which module caused the interrupt. This example only enables the UCA0 receive interrupt, so reading UCA0RXBUF is sufficient. An application that receives a continuous stream would normally place bytes into a circular buffer and let the main program process them outside the interrupt service routine.

7.8.7. Using the LaunchPad USB Connection

The MSP430 LaunchPad includes a debugger or emulator section that communicates with the host computer through USB. In addition to programming and debugging the target, the emulator can bridge the target UART to a virtual serial port. P1.1 and P1.2 must be routed through the board jumpers in the hardware-UART orientation. The host then opens the serial port using a terminal application and selects the same baud rate, data length, parity, and stop-bit settings as the MSP430.

A useful troubleshooting order is to first confirm that the correct serial port appears on the computer, then confirm the terminal settings, then verify the LaunchPad jumper orientation, and finally inspect the MSP430 clock, pin-selection, and USCI registers. Randomly changing the baud rate

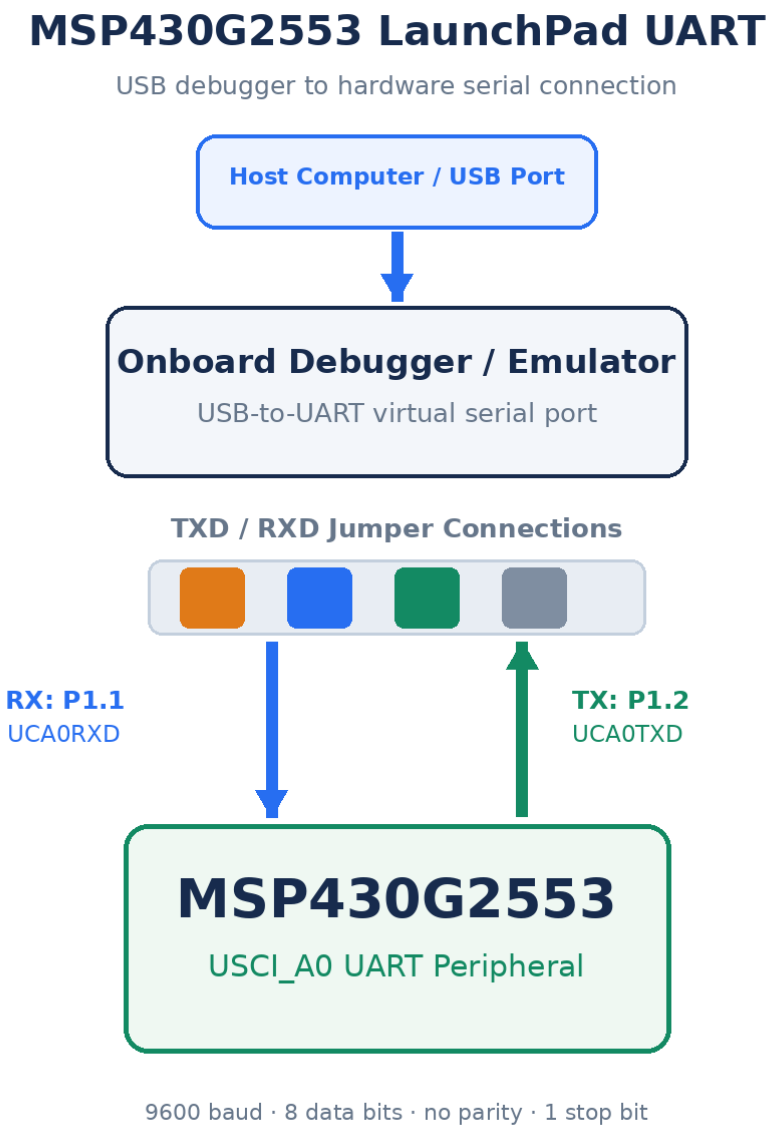


Figure 7.25.: The LaunchPad debugger bridges USCI_A0 UART signals to a virtual serial port on the host computer.

rarely fixes a connection whose actual problem is swapped wires or the wrong virtual port.

7.8.8. Common UART Failure Modes

- No characters appear: verify common ground, TX-to-RX wiring, pin selection, peripheral reset state, and terminal port.
- Unreadable characters appear consistently: verify baud rate, SMCLK frequency, data length, parity, and stop-bit settings.
- Only the first character is correct: check whether software waits for UCA0TXIFG before each write.
- Characters are occasionally missing: service UCA0RXBUF sooner or add an interrupt-driven software buffer.
- The signal is electrically wrong: confirm compatible logic levels and do not connect RS-232 voltages directly.

7.9. Synchronous Serial Communication

UART removes the clock wire by requiring both devices to agree on a baud rate and frame format. That is convenient for a terminal or another point-to-point connection, but the two devices still depend on separate local clocks. A synchronous serial interface instead returns to the clocked arrangement introduced with the vacuum robot: one device produces a timing signal that tells the receiving device exactly when to sample each bit.

Because the clock is transmitted along with the data, a synchronous controller can deliberately pause between bits, change its clock frequency between transactions, and coordinate an entire exchange without asking the target to infer the bit timing. SPI and I²C both use this principle, but

they make different choices about wiring, device selection, direction, and bus ownership.

7.10. Serial Peripheral Interface

Serial Peripheral Interface, or SPI, is a synchronous protocol commonly used between a controller and nearby peripherals such as displays, analog-to-digital converters, flash memory, radio modules, and microSD cards. Unlike UART, SPI transmits a clock signal. The controller decides when each bit occurs and can change the clock rate between transactions as long as the peripheral limits are respected.

7.10.1. SPI Signals

A conventional four-signal SPI connection contains a serial clock, a controller-output data signal, a controller-input data signal, and a chip-select signal. Signal names vary among manufacturers. SIMO and SOMI mean slave-in/master-out and slave-out/master-in in older terminology; MOSI and MISO are also common. Newer documentation may use controller and peripheral language. Regardless of the label, one data wire carries bits from the controller and the other carries bits back.

SPI is inherently a shift-register exchange. Every clock cycle shifts one bit out of each device and one bit into each device. To read a register, the controller must still transmit something—often an address followed by dummy bytes—to generate the clock cycles that return the requested data. This behavior can be surprising after using UART, where receiving does not require simultaneous transmission.

Notice that the target cannot decide to send an arbitrary number of bits whenever it wants. The controller owns the clock, so it must know when a response is expected and generate enough clock pulses to receive that

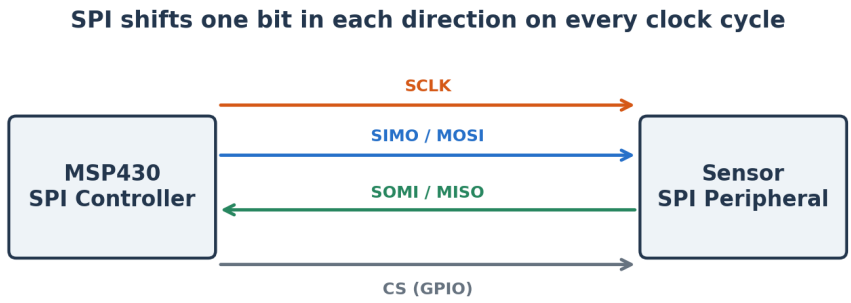


Figure 7.26.: A four-signal SPI connection provides a clock, two data directions, and a chip-select signal.

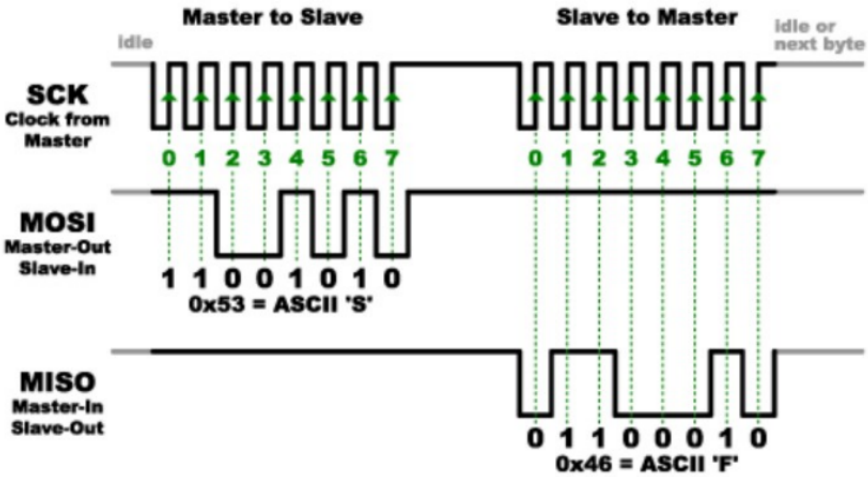


Figure 7.27.: SPI timing shows the clock, controller-output data, and controller-input data during successive byte transfers.

response. A sensor datasheet normally specifies whether the first transferred byte is an address, a command, a dummy value, or returned data.

7.10.2. Clock Polarity and Phase

SPI does not define one universal sampling edge. Clock polarity determines whether the clock idles low or high. Clock phase determines whether data is captured on the first or second active edge. The four combinations are often called SPI modes 0 through 3. Both devices must use compatible polarity and phase settings. If they disagree, the receiver may sample while the transmitter is changing the bit, producing shifted or unstable data.

A peripheral datasheet normally shows a timing diagram that identifies the idle level, capture edge, maximum clock frequency, setup time, and hold time. Those requirements should determine the USCI configuration. Selecting a mode because it worked for a different sensor is not a reliable method.

7.10.3. Selecting One of Several Peripherals

The clock and data wires may be shared among several SPI peripherals. The controller activates one device by driving its chip-select signal to the active level, commonly low. All other chip-select signals remain inactive so those devices ignore the clock and release their output connections. Each additional peripheral generally needs another GPIO chip-select pin.

Some devices support daisy chaining, in which the output of one shift register feeds the input of the next. Daisy chaining reduces the number of select signals but changes the transaction format and increases the number of clock cycles needed to reach a particular device. It should only be used when the peripheral explicitly supports it.

are binary decoder chips that can multiply your SS outputs.

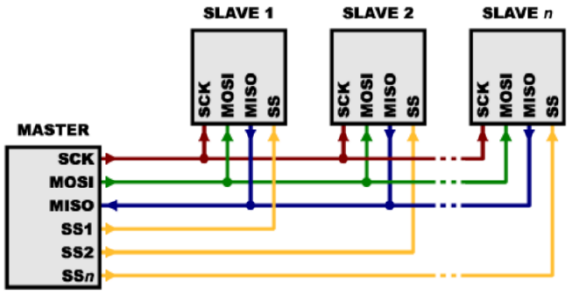


Figure 7.28.: Several SPI peripherals share clock and data signals while using separate active-low select lines.

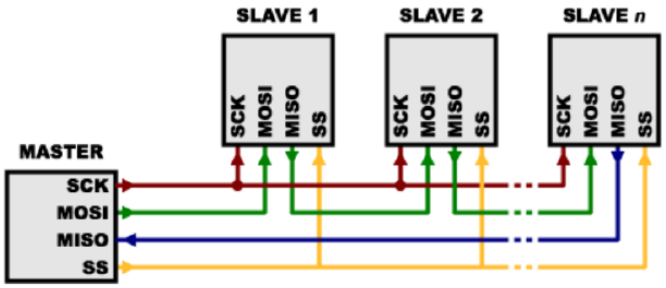


Figure 7.29.: An alternative SPI arrangement connects compatible peripherals in a chain and uses one shared select signal.

7.10.4. SPI on the MSP430G2553

Both USCI_A0 and USCI_B0 can operate in SPI mode. If USCI_A0 is already providing the UART connection to the computer, USCI_B0 is the natural choice for SPI. In USCI_B0 SPI mode, P1.5 provides UCB0CLK, P1.6 provides UCB0SOMI, and P1.7 provides UCB0SIMO. These same P1.6 and P1.7 pins are used for I²C, so USCI_B0 cannot provide SPI and I²C simultaneously.

The following initialization configures USCI_B0 as an 8-bit, most-significant-bit-first SPI controller. UCCKPH and UCCKPL must be selected for the actual peripheral; the example uses UCCKPH with the default low clock polarity. P2.0 is used as a software-controlled active-low chip select.

```
static void spi_init(void)
{
    P1SEL |= BIT5 | BIT6 | BIT7;
    P1SEL2 |= BIT5 | BIT6 | BIT7;
    P2DIR |= BIT0; // P2.0 is chip select
    P2OUT |= BIT0; // deselect the peripheral
    UCB0CTL1 = UCSWRST | UCSSEL_2;
    UCB0CTL0 = UCCKPH | UCMSB | UCMST | UCSYNC;
    UCB0BR0 = 2; // 500-kHz clock from 1-MHz SMCLK
    UCB0BR1 = 0;
    UCB0CTL1 &= ~UCSWRST;
}
static unsigned char spi_transfer(unsigned char value)
{
    while ((IFG2 & UCB0TXIFG) == 0) {
    }
    UCB0TXBUF = value;
    while ((IFG2 & UCB0RXIFG) == 0) {
    }
```

```
    return UCB0RXBUF;
}
```

A complete transaction drives P2.0 low, calls spi_transfer for every command, address, or data byte, waits until the final transfer is complete, and then drives P2.0 high. The returned value from each call is the byte shifted in during the same eight clock cycles. The meaning of those bytes comes from the peripheral datasheet, not from SPI itself.

7.10.5. When SPI Is a Good Choice

- Use SPI when a nearby peripheral needs high throughput or predictable clocked timing.
- SPI is attractive for displays, fast ADCs, external flash memory, microSD cards, and many wireless modules.
- The protocol is simple in hardware and supports simultaneous shifting in both directions.
- The tradeoff is pin count: clock, two data signals, and usually one chip-select signal per peripheral.
- SPI provides no universal addressing, acknowledgment, or error detection; those features must come from the peripheral-specific command format.

7.10.6. Real-World Example: Wi-Fi and microSD

A Wi-Fi module may need to move command packets and network data faster than a low-rate UART can comfortably support. SPI provides a controller-driven clock and efficient full-duplex transfers while keeping the interface simple. A microSD card can also operate through SPI mode. The

card command protocol defines initialization, block addressing, responses, and data tokens on top of the basic SPI signals.

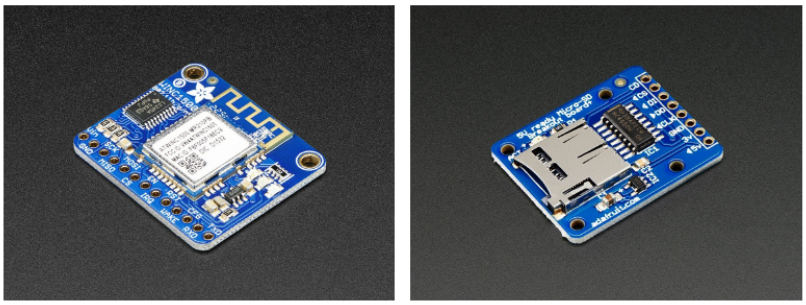


Figure 7.30.: Wi-Fi modules and microSD breakout boards illustrate practical high-throughput SPI peripherals.

These applications reveal an important distinction: a serial bus defines how bits move, while a device protocol defines what the bits mean. Correct SPI polarity and timing are necessary, but they are not enough. Software must also implement the command sequence required by the Wi-Fi module or storage card.

7.11. Inter-Integrated Circuit

The Inter-Integrated Circuit bus, written I²C and pronounced “I-squared-C,” is a synchronous, addressed, half-duplex protocol. It uses only two shared signal wires: SCL for the clock and SDA for data. A controller initiates communication, supplies the clock, and selects a target by transmitting its address. Multiple targets can share the same two wires as long as their addresses do not conflict and the total electrical loading remains within specification.

7.11.1. Open-Drain Signaling and Pull-Up Resistors

I²C devices do not actively drive the bus high. Their outputs behave as open-drain switches that can either pull a signal low or release it. External pull-up resistors return the released wires to a high voltage. This arrangement allows several devices to share the bus safely: if any device pulls a line low, the bus is low. No device attempts to force the line high against another device that is pulling low.

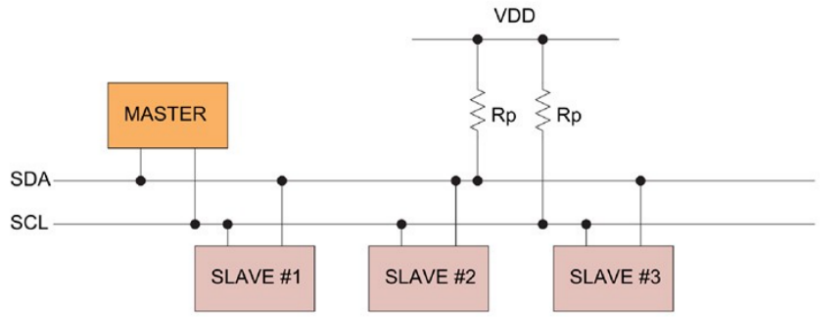


Figure 7.31.: An I²C bus connects one controller and several targets through shared SCL and SDA signals with external pull-up resistors.

The pull-up resistance and bus capacitance determine how quickly a released line rises. A large resistance reduces current but slows the edge. A small resistance improves rise time but increases current whenever a device pulls the line low. Long wires, many devices, and large connector structures add capacitance and can prevent the bus from meeting its timing requirement even when the register configuration is correct.

7.11.2. Following an I²C Transaction

An I²C transaction is more structured than a basic UART character or SPI shift-register exchange. The controller must announce the beginning of the transaction, identify the target device, indicate the direction of transfer, check whether each byte was accepted, and release the bus at the end. The waveform below shows these stages together; the next subsections explain them one at a time.

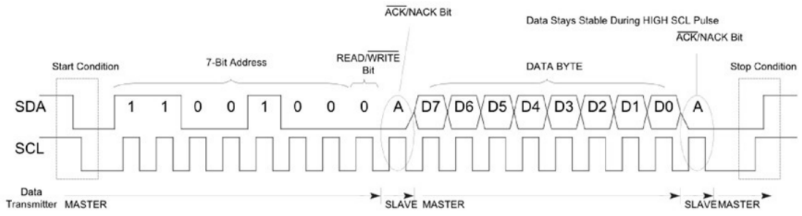


Figure 7.32.: I²C timing identifies START, the 7-bit address, the direction bit, acknowledgments, data, and STOP.

7.11.2.1. START Condition

An I²C transfer begins with a START condition. While SCL remains high, the controller pulls SDA from high to low. Every device on the shared bus can recognize this unusual transition because ordinary data bits are changed while the clock is low. START therefore tells the targets that an address frame is about to follow.

7.11.2.2. Address Frame and Direction Bit

After START, the controller transmits the seven-bit target address, most-significant bit first. An eighth bit specifies the direction: 0 requests a

write from controller to target, while 1 requests a read from target to controller. A device whose configured address matches the transmitted address participates in the remainder of the transfer; devices with other addresses ignore it.

A seven-bit address creates 128 possible bit patterns, but some values are reserved, so not all 128 represent ordinary usable peripheral addresses. The practical number of devices on a bus is also constrained by duplicate fixed addresses and by the total capacitance contributed by devices, traces, and cables.

7.11.2.3. Acknowledgment Bit

Every eight transmitted bits are followed by a ninth clock pulse reserved for acknowledgment. The transmitting device releases SDA, allowing the receiving device to pull it low if the byte was accepted. A low SDA value during this clock pulse is an acknowledgment, or ACK. If SDA remains high, the result is a negative acknowledgment, or NACK.

A NACK may mean that no device recognized the address, that a target is busy or unable to accept another byte, or that a controller has reached the final byte of a read. A program should not assume that an I²C target is present merely because it requested a transaction; acknowledgment is the evidence that another device actually participated.

7.11.2.4. Data Frames

After the address is acknowledged, the controller continues producing SCL pulses for the data bytes. During a write, the controller places each data bit on SDA. During a read, the addressed target places its data on SDA while the controller still generates the clock. Each group of eight data bits is followed by another acknowledgment clock. The physical bus is shared, so only one device supplies a given data bit at a time.

7.11.2.5. STOP and Repeated START

The controller normally ends a transaction by generating STOP: SDA changes from low to high while SCL remains high. This transition releases the bus and tells the targets that the message is complete. The controller may instead issue another START without first generating STOP. This repeated START keeps control of the bus while beginning a new address phase, which is useful when one logical operation contains both a write and a read.

7.11.3. Writing a Target Register

Many I²C sensors organize their settings and measurements as internal registers. To write a register, the controller addresses the target in the write direction, transmits the internal register address, and then transmits the new value. Each byte is acknowledged. For example, writing 0x00 to register 0x6B of an MPU6050 wakes the device from its default sleep state.

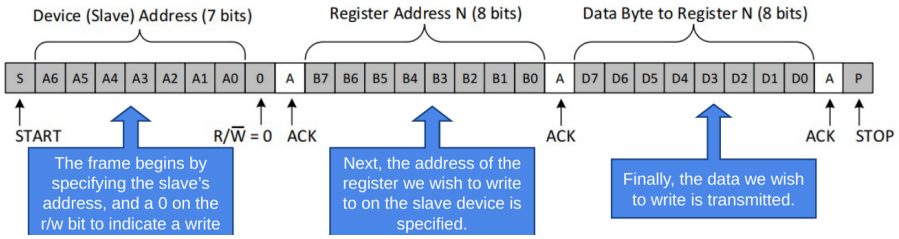


Figure 7.33.: A register-write transaction separates the target address, internal register address, data byte, and acknowledgments.

```
START | 0x68 + WRITE | ACK | 0x6B | ACK | 0x00 | ACK | STOP
```

7.11.4. Reading a Target Register

A register read usually begins as a write because the controller must first tell the target which internal register is desired. After transmitting the register address, the controller issues a repeated START and sends the target address again with the read bit set. The target then returns one or more data bytes. The controller acknowledges every byte it wants to continue receiving and NACKs the final byte before generating STOP.

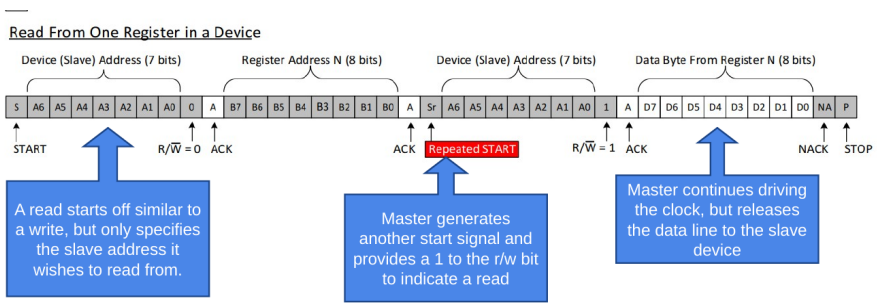


Figure 7.34.: A register-read transaction includes the internal register address, repeated START, direction change, returned data, and final NACK.

The repeated START is not an arbitrary complication. If the controller issued STOP between the register-address phase and the read phase, another controller could claim the bus or the target could interpret the operations as unrelated transactions. Keeping the bus active makes the two phases one atomic operation.

7.11.5. I²C on the MSP430G2553

USCI_B0 provides the I²C hardware. P1.6 is UCB0SCL and P1.7 is UCB0SDA. Both P1SEL and P1SEL2 must select the peripheral functions. In controller mode, UCMST enables controller operation, UCMODE_3

selects I²C, and UCSYNC selects synchronous operation. UCB0BR0 and UCB0BR1 divide SMCLK to create SCL. With a 1-MHz SMCLK and a divider of ten, the nominal SCL frequency is 100 kHz.

```
static void i2c_init(void)
{
    P1SEL |= BIT6 | BIT7;
    P1SEL2 |= BIT6 | BIT7;
    UCB0CTL1 = UCSWRST | UCSSEL_2;
    UCB0CTL0 = UCMST | UCMODE_3 | UCSYNC;
    UCB0BR0 = 10; // 100-kHz SCL from 1-MHz SMCLK
    UCB0BR1 = 0;
    UCB0CTL1 &= ~UCSWRST;
}
```

Unlike UART, the target address is part of each I²C transaction. The unshifted 7-bit address is written to UCB0I2CSA. The USCI hardware places the address and direction bit on the bus when software requests a START condition.

7.11.6. Real-World Example: An Impact-Detecting Hard Hat

Consider a smart construction hard hat designed to report a potentially dangerous impact. When an object strikes the helmet, the shell experiences a sudden change in acceleration. A small embedded controller can monitor that acceleration and decide whether the event exceeds a threshold that should trigger an alarm or wireless notification.

The controller could measure an analog acceleration signal directly, but many practical sensing devices include their own measurement electronics and expose the result through a digital interface. The system then needs a serial connection between the main microcontroller and the sensor. I²C is attractive because it uses only two signal wires, allows the controller to

configure the sensor through internal registers, and supports additional sensors on the same bus.

7.11.6.1. What the Accelerometer Measures

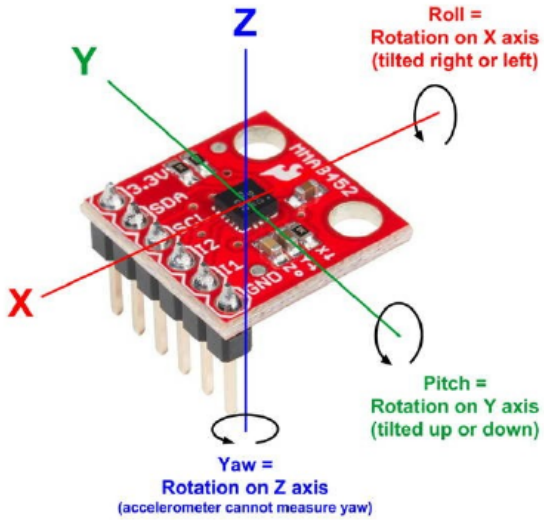
An accelerometer measures acceleration along one or more physical axes. A three-axis device reports separate X, Y, and Z values, allowing software to estimate both the direction and magnitude of an event. In a simplified physical model, a small internal mass responds to acceleration; the sensor converts the resulting displacement or force into an electrical measurement, which on-chip circuitry digitizes.

7.11.7. Example: Reading the MPU6050 Accelerometer

The MPU6050 is an inertial measurement unit containing a three-axis accelerometer and a three-axis gyroscope. It commonly uses the 7-bit address 0x68 when its AD0 address pin is low. The acceleration measurements begin at internal register 0x3B, with the high and low bytes of the X-axis measurement stored in consecutive registers.

For the impact-detecting hard hat, the MSP430 first configures the sensor and then reads consecutive acceleration registers. A practical application could periodically measure all three axes, combine them into an overall acceleration magnitude, and compare the result with a threshold. The chapter example concentrates on the serial transaction itself: select the measurement register, issue a repeated START, and retrieve the high and low bytes.

The following example shows the central transaction logic. Error handling is intentionally omitted so the bus sequence remains visible. Production code should detect NACK, arbitration, and bus-stuck conditions and should use timeouts rather than wait forever.



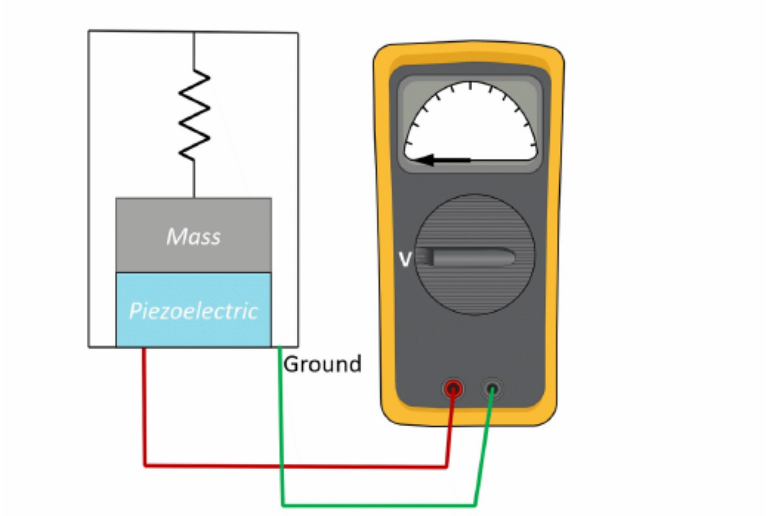


Figure 7.35.: Three-axis accelerometer orientation is related to a simplified mass-and-sensor measurement mechanism.

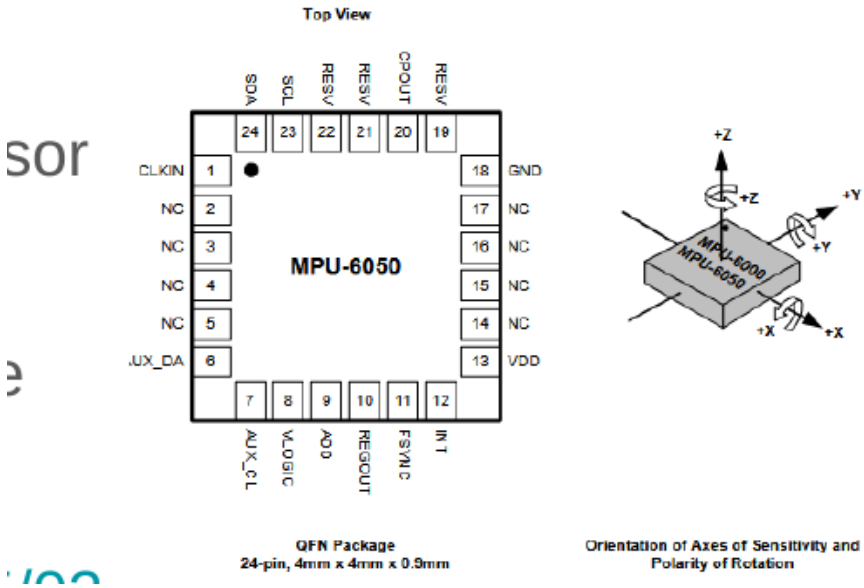


Figure 7.36.: The MPU6050 package includes serial-interface pins and defined measurement axes.

```
#define MPU6050_ADDRESS 0x68
#define ACCEL_XOUT_H 0x3B
static int i2c_read_accel_x(void)
{
    unsigned char high;
    unsigned char low;
    while (UCB0CTL1 & UCTXSTP) {
    }
    UCB0I2CSA = MPU6050_ADDRESS;
    // Write the internal register address.
    UCB0CTL1 |= UCTR | UCTXSTT;
    while ((IFG2 & UCB0TXIFG) == 0) {
    }
    UCB0TXBUF = ACCEL_XOUT_H;
    while ((IFG2 & UCB0TXIFG) == 0) {
    }
    // Change direction with a repeated START.
    UCB0CTL1 &= ~UCTR;
    UCB0CTL1 |= UCTXSTT;
    while (UCB0CTL1 & UCTXSTT) {
    }
    while ((IFG2 & UCB0RXIFG) == 0) {
    }
    // Tell USCI that the next byte will be the final byte.
    UCB0CTL1 |= UCTXSTP;
    high = UCB0RXBUF;
    while ((IFG2 & UCB0RXIFG) == 0) {
    }
    low = UCB0RXBUF;
    return (int)((high << 8) | low);
}
```

The two received bytes are combined into one signed 16-bit quantity.

Because high and low are promoted to int before the shift and OR, the resulting bit pattern can be cast to a signed 16-bit type in a program that includes stdint.h. The raw value must then be scaled according to the accelerometer range configured in the MPU6050.

7.11.8. Real-World Example: A Mattress Sensor Array

A pressure-mapping mattress may contain dozens or hundreds of pressure and temperature sensors. Giving every sensor a dedicated group of wires would be impractical. I²C allows many addressed devices to share SCL and SDA, so the wiring does not grow by one complete bus for every sensor. Local sensor hubs can also collect data from small regions and expose one address to the main controller.

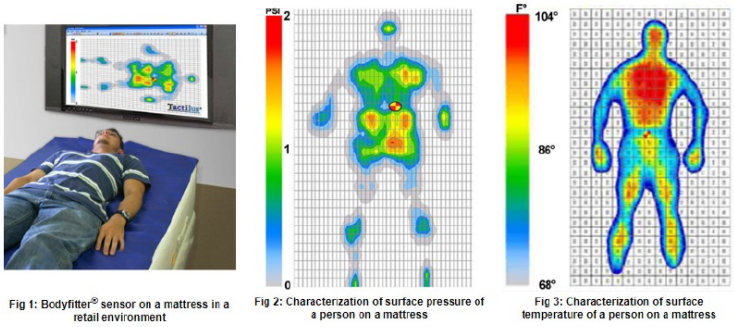


Figure 7.37.: A pressure-mapping mattress generates pressure and temperature images from a dense sensor array.

This example also reveals an I²C limitation. Adding devices and cable length increases capacitance, which slows the rising edges produced by the pull-up resistors. A large physical array may need bus buffers, multiple

segments, slower clocking, or local controllers rather than one enormous bus.

7.11.9. Real-World Example: An I/O Expander

A design sometimes needs more digital inputs and outputs than the microcontroller package provides. An I²C I/O expander uses one bus address and a small set of internal registers to control additional pins. Two microcontroller signals can therefore control banks of LEDs, read a keypad, monitor switches, or operate relays.

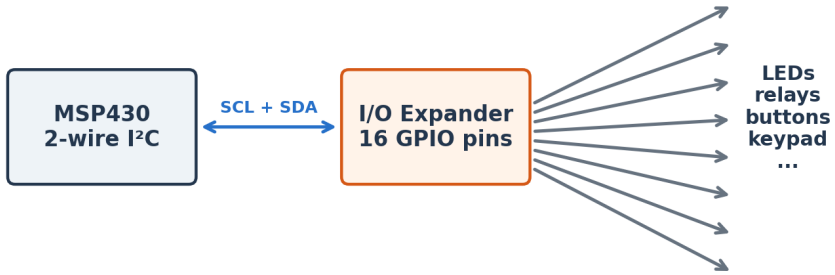


Figure 7.38.: An I²C I/O expander trades two bus signals for a larger bank of remotely controlled GPIO pins.

The added pins are not identical to native GPIO. Every change requires an I²C transaction, so the latency is longer and the maximum update rate is lower. A target cannot initiate an I²C transaction without a controller clock, so many expanders include a separate interrupt output to notify the microcontroller that an input changed.

Figure 7.39.: A 16-bit I²C I/O expander adds many digital signals using a two-wire bus.

7.11.10. When I²C Is a Good Choice

- Use I²C for configuration registers and moderate-rate sensors located near the controller.
- Addressing allows many targets to share two signal wires.
- Acknowledgments reveal whether an address or data byte was accepted.
- Open-drain signaling requires pull-up resistors and creates a rise-time limit.
- The shared half-duplex bus is generally slower and more software-intensive than a simple SPI exchange.
- Address conflicts occur when two targets use the same fixed address unless a multiplexer or alternate address is available.

7.12. Controller Area Network

Controller Area Network, or CAN, is an asynchronous multi-controller serial bus developed for reliable communication among distributed electronic control units. It is strongly associated with vehicles, but it is also used in industrial equipment, robotics, medical devices, and other systems where noise immunity and fault detection matter.

CAN is not a point-to-point character stream like UART. Every node monitors the same bus, and a frame carries an identifier that describes the message rather than a destination address. A node can publish a wheel-speed message, for example, and any interested node can receive it. Hardware acceptance filters allow a controller to ignore identifiers it does not need.

This returns us to the vehicle example that opened the chapter. The engine controller, braking controller, steering controller, and other subsystems

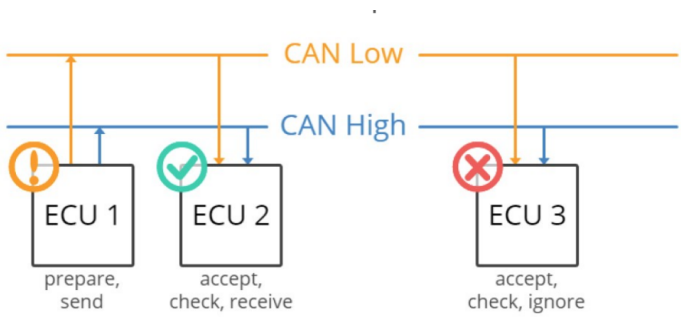


Figure 7.40.: Several vehicle control units share CAN_H and CAN_L while deciding which messages to accept.

do not need a separate dedicated wire for every value sent between every pair of devices. Instead, they publish structured messages on a shared communication network. CAN provides the physical signaling, message organization, arbitration, and error handling that make this arrangement practical.

7.12.1. Differential Signaling

A high-speed CAN physical layer uses two wires, CAN_H and CAN_L, arranged as a twisted pair. The receiver interprets the difference between the wire voltages. External noise that couples similarly into both wires changes their absolute voltages but leaves much of the difference unchanged. This common-mode rejection makes differential signaling valuable in electrically noisy environments.

When the bus is recessive, the two wires remain close to the same voltage, often approximately 2.5 V in a high-speed CAN implementation. A dominant bit drives CAN_H higher and CAN_L lower, increasing the voltage difference. Receivers identify the bit from this differential voltage rather

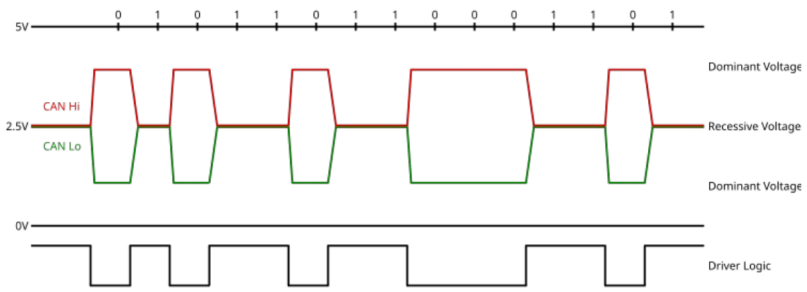


Figure 7.41.: A CAN waveform distinguishes dominant and recessive bits using the difference between CAN_H and CAN_L.

than treating either individual wire as an ordinary single-ended GPIO signal.

A CAN controller does not connect directly to the twisted pair. A transceiver converts the controller's digital transmit and receive signals into the differential bus voltages. Termination resistors are installed at both physical ends of the bus to absorb signal energy and reduce reflections. The bus should be wired as a line with short stubs rather than as an arbitrary star.

7.12.2. The Classical CAN Frame

A Classical CAN data frame includes a start-of-frame bit, an identifier, control information, zero to eight data bytes, a cyclic-redundancy check, acknowledgment, and end-of-frame bits. Standard CAN uses an 11-bit identifier; extended CAN uses a 29-bit identifier. The identifier labels the meaning and priority of the message rather than identifying the physical destination of the frame.

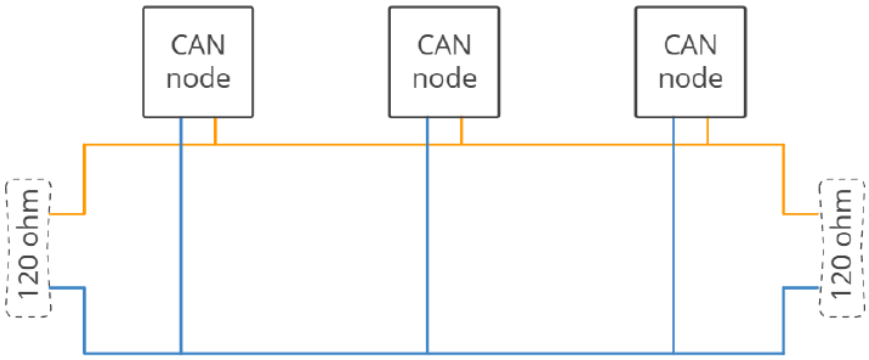


Figure 7.42.: Termination places a 120-ohm resistor at each physical end of a high-speed CAN bus.

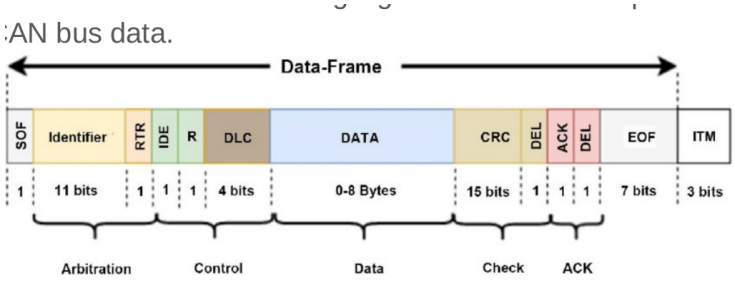


Figure 7.43.: A Classical CAN frame identifies the arbitration, control, payload, CRC, acknowledgment, and end-of-frame fields.

7.12.2.1. Start of Frame, Identifier, and Arbitration

A dominant start-of-frame bit announces that a node intends to transmit and provides a synchronization edge for the other nodes. The identifier follows. For example, identifier 0x100 might represent engine speed, 0x200 might represent wheel speed, and 0x300 might represent coolant temperature. All nodes observe the message, but each node uses its acceptance filters to decide whether the information is relevant.

Several nodes may begin transmitting at the same time. CAN resolves this using non-destructive bitwise arbitration: a dominant 0 overrides a recessive 1 on the shared bus. Each transmitter monitors the actual bus level. A node that sends recessive but observes dominant knows that another node has higher priority and immediately stops transmitting. The message with the lower numerical identifier continues without being corrupted.

	Start bit	ID bits										The rest of the frame
		10	9	8	7	6	5	4	3	2	1	0	
Node 15	0	0	0	0	0	0	0	0	1	1	1	1	
Node 16	0	0	0	0	0	0	0	1	Stopped Transmitting				
CAN data	0	0	0	0	0	0	0	0	1	1	1	1	

Figure 7.44.: CAN arbitration causes a node to stop transmitting when its recessive identifier bit loses to another node's dominant bit.

7.12.2.2. RTR, IDE, Reserved Bits, and Data Length

The remote transmission request, or RTR, distinguishes a frame carrying data from a request for data in Classical CAN. The identifier extension bit, IDE, distinguishes the standard 11-bit identifier format from the extended 29-bit format. A reserved bit, commonly labeled r0, is maintained according

to the Classical CAN specification. The four-bit data length code, DLC, indicates how many data bytes appear in the payload.

7.12.2.3. Data, CRC, Acknowledgment, and End of Frame

The data field carries zero to eight payload bytes in Classical CAN. The payload interpretation is defined by the application: a pair of bytes might represent wheel speed, temperature, motor current, or another measurement. The cyclic-redundancy check allows receivers to verify that the protected frame contents arrived without detectable corruption.

For acknowledgment, the transmitter releases an acknowledgment slot as recessive. Any receiver that successfully validated the frame can drive that slot dominant to confirm reception. A transmitter that observes no dominant acknowledgment knows that the frame was not accepted. The end-of-frame field consists of seven recessive bits, marking completion and allowing the bus to return to its idle state.

7.12.2.4. Bit Timing and Stuff Bits

Although CAN uses two physical wires, it does not carry a separate clock. Every node is configured for the same nominal bit rate, and edges in the transmitted signal allow receivers to keep their internal sampling clocks aligned. A long unbroken sequence of identical bits would remove those useful synchronization edges.

Bit stuffing solves that problem by inserting an opposite-polarity bit after a run of five identical transmitted bits in the stuffed portion of a Classical CAN frame. The receiver removes the inserted bit before interpreting the message. The detailed frame below shows how identifier, control, payload, CRC, acknowledgment, and physical-layer waveforms relate to one another.

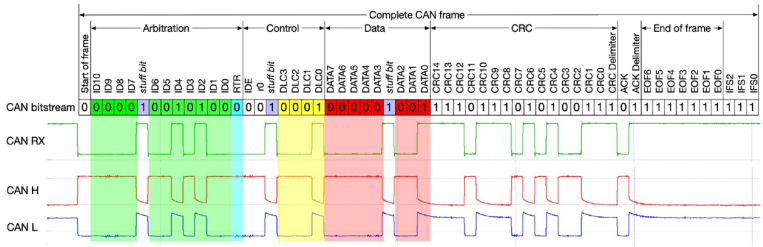


Figure 7.45.: A detailed CAN frame aligns logical fields, the received bitstream, and the differential CAN_H/CAN_L waveforms.

CRC checks, acknowledgment, transmit monitoring, format checks, and error counters work together to detect faults and prevent a failing node from permanently disrupting the network. These features make CAN more complex than UART, but they support the reliability needed when many distributed controllers share a noisy physical environment.

7.12.3. Using CAN with the MSP430G2553

The MSP430G2553 does not contain an integrated CAN controller. CAN can still be added with an external controller connected through SPI and a separate CAN transceiver connected to the differential bus. This solution uses more components and software than a microcontroller with built-in CAN, but it is appropriate when the MSP430 remains suitable for the rest of the application.

This example emphasizes the difference among three layers. SPI carries bytes between the MSP430 and the external CAN controller. The CAN controller creates, receives, filters, and checks CAN frames. The transceiver produces and senses the CAN_H and CAN_L voltages. Calling all three components “the CAN interface” can obscure which part is responsible for a failure.

7.13. Choosing a Serial Protocol

There is no universally best serial protocol. A designer should begin with the communication relationship and physical environment rather than personal familiarity. The number of devices, required throughput, distance, noise, pin budget, direction of communication, error-detection needs, and availability of compatible hardware all influence the decision.

7.13.1. Choose UART When

- Two devices need a simple point-to-point connection.
- The devices can agree on a fixed baud rate and frame format.
- Human-readable debugging through a terminal is valuable.
- A separate clock wire is undesirable and moderate throughput is sufficient.

7.13.2. Choose SPI When

- Peripherals are close to the controller and high throughput is important.
- The controller can dedicate clock, data, and chip-select pins.
- Full-duplex shifting or precise controller-generated timing is useful.
- The device command protocol supplies any needed addressing and checking.

7.13.3. Choose I²C When

- Several moderate-speed devices should share two wires.
- The targets provide unique or configurable addresses.
- The bus is short enough to satisfy capacitance and rise-time limits.
- Register-oriented sensor and configuration traffic dominates.

7.13.4. Choose CAN When

- Many controllers need to publish and receive messages on one robust network.
- The electrical environment is noisy or the wiring extends beyond one circuit board.
- Priority arbitration, hardware filtering, and strong fault detection are required.
- The design can include a CAN controller and physical-layer transceiver.

A real system often uses several protocols at once. A robot may use I²C for low-rate sensors, SPI for a display and flash memory, UART for a debug console, and CAN for communication between major control units. The protocols are complementary tools, not mutually exclusive architectural philosophies.

7.14. Recap

In this chapter we learned:

- Serial communication reduces pin and wiring requirements by transmitting bits over time.
- Synchronous protocols transmit a clock; asynchronous UART reconstructs timing from an agreed baud rate.
- UART frames data with start, data, optional parity, and stop bits.
- The MSP430G2553 uses USCI_A0 for UART and exposes UCA0RXD on P1.1 and UCA0TXD on P1.2.
- Polling is simple, while interrupts and software buffers allow the processor to perform other work.
- SPI exchanges bits in both directions under a controller-generated clock and selects targets with chip-select signals.
- I²C uses addressed, open-drain SCL and SDA signals with an acknowledgment after every byte.
- Repeated START conditions allow an I²C register read to change direction without releasing the bus.
- CAN combines differential signaling, message identifiers, arbitration, and extensive error detection.
- The MSP430G2553 needs an external CAN controller and transceiver to join a CAN network.
- Protocol selection depends on topology, speed, distance, robustness, pin budget, and device support.

7.15. Review Questions

1. How can the vacuum robot divide bump sensing, motor control, and encoder feedback between two microcontrollers?

2. Which eight GPIO levels represent decimal 21, and why can the receiver not distinguish the sequence 21, 21, 10 using data levels alone?

3. How does adding a clock wire solve the repeated-value problem, and why might designers then replace eight data wires with one?

4. What electrical problem occurs if two push-pull GPIO outputs drive opposing values on the same shared wire?

5. Why does a parallel bus become difficult to route and time as its width and speed increase?

6. How does a UART receiver determine when to sample a bit without a transmitted clock?

7. How long does one 8N1 frame require at 115200 baud? What is the maximum payload byte rate?

8. Why must TX and RX be crossed in a point-to-point UART connection?

9. What is the difference between UCA0TXIFG and transmission complete?

10. Why does an SPI controller transmit a dummy byte when it only wants to receive?

11. What happens if two SPI peripherals drive the shared input line at the same time?

12. Why can several I²C devices safely pull SDA low without shorting one output against another?

13. Describe the sequence required to read two bytes from an I²C target register.

14. Why does bus capacitance limit the physical size of an I²C network?

15. How could an MSP430 and MPU6050 cooperate to detect a potentially dangerous impact on a smart hard hat?
16. How does CAN arbitration allow the highest-priority frame to continue without corruption?
17. Which protocol would you choose for a terminal, a microSD card, a temperature sensor array, and communication among vehicle controllers? Explain each choice.

7.16. References

Texas Instruments. MSP430G2x53, MSP430G2x13 Mixed-Signal Microcontrollers Data Sheet, SLAS735.

Texas Instruments. MSP430F2xx, MSP430G2xx Family User's Guide, SLAU144K, revised August 2022. https://www.ti.com/lit/ug/slau144k/slau144k.pdf

Texas Instruments. MSP-EXP430G2 LaunchPad Experimenter Board User's Guide, SLAU318.

NXP Semiconductors. I²C-bus Specification and User Manual, UM10204. https://www.nxp.com/docs/en/user-guide/UM10204.pdf

Bosch. CAN Protocols and CAN Controller IP Documentation. https://www.bosch-semiconductors.com/products/ip-modules/can-protocols/

TDK InvenSense. MPU-6000 and MPU-6050 Product Specification and Register Map.